

컴퓨터구조 Computer Architecture

6장 - 기억 장치
김현석

학습목표

- 기억 장치의 의미와 읽기·쓰기 과정과 기억 장치 관련 용어·특징을 이해할 수 있다.
- RAM과 ROM의 특성을 이해하고 기억 장치 모듈을 설계할 수 있다.
- 캐시 기억 장치의 세 가지 매핑 방법과 교체 알고리즘을 이해하고 설명할 수 있다.
- 가상 기억 장치의 필요성과 매핑 방법을 이해하고 설명할 수 있다.
- 연관 기억 장치의 동작 원리를 이해하고 설명할 수 있다.
- SDRAM, DDR SDRAM, 플래시 메모리의 동작 원리를 이해하고 설명할 수 있다.

내용

- 01 기억 장치 시스템의 개요
- 02 주기억 장치
- 03 캐시 기억 장치
- 04 가상 기억 장치
- 05 연관 기억 장치
- 06 최신 기억 장치 기술

01 기억 장치 시스템의 개요

1 기억 장치의 종류와 특성

표 6-1 컴퓨터 기억 장치 시스템의 주요 특성 분류

기준	설명
위치	CPU, 컴퓨터의 내부와 외부
용량	워드 크기, 워드 개수
전송 단위	워드, 블록
성능	액세스 시간, 사이클 시간, 전송률
액세스 방법	순차, 직접, 임의, 연관
물리적인 유형	반도체, 자기, 광, 자기-광
물리적인 특성	휘발성과 비휘발성, 파괴적과 비파괴적

□ 위치에 따른 분류

- 위치는 컴퓨터 내부와 외부를 구분하는 기준이다.
- CPU 내부의 레지스터와 주기억 장치는 **내부 기억 장치**고, 자기 디스크와 자기 테이프는 **외부 기억 장치**다.

01 기억 장치 시스템의 개요

□ 용량에 따른 분류

- 용량(capacity)은 기억 장치가 저장할 수 있는 데이터의 총량으로 바이트나 워드로 나타낸다.
- 워드(word)는 시스템에 따라 8, 16, 32, 64비트로 길이가 다양한데 이는 CPU가 처리하는 명령어 길이나 내부에서 한 번에 연산할 수 있는 데이터 비트 수와 같다.
- 일반적으로 외부 기억 장치는 용량을 바이트로 표시한다.

□ 전송 단위에 따른 분류

- 내부 기억 장치에서 전송 단위는 기억 장치로 들어가고 나오는 데이터선의 수로, 워드 길이와 같거나 다를 수도 있다.
- 외부 기억 장치에서 전송 단위는 워드보다 큰 블록이다. 예를 들어 외부 기억 장치인 하드 디스크는 블록 크기가 512바이트 또는 1024바이트다.
- 외부 기억 장치에 연결되는 데이터 버스의 폭은 8비트, 16비트, 32비트이므로 블록 하나를 전송하려면 전송 동작이 여러 번 연속으로 이루어져야 한다.

01 기억 장치 시스템의 개요

□ 성능에 따른 분류

- **액세스 시간**(access time) : 주소나 제어(읽기/쓰기) 신호가 기억 장치에 도착한 순간부터 데이터가 저장되거나 읽히는 순간까지이다.
- **사이클 시간**(cycle time) : 액세스 시간과 다음 액세스를 시작하기 위해 필요한 동작에 걸리는 시간을 더한 것이다.
- **전송률**(transfer rate) : 전송률은 데이터가 기억 장치로 들어가거나 나오는 초당 비트 수로, 기억 장치의 전송 속도를 측정하는 기준이다. 대역폭(bandwidth)이라고도 한다. 데이터 버스의 폭이 크면 더 많은 비트가 한 번에 전송되므로 전송률이 높아진다.

01 기억 장치 시스템의 개요

□ 액세스 방법에 따른 분류

- **순차 액세스**(sequential access) : 임의 위치의 데이터를 처음부터 순서대로 읽으므로 데이터의 위치에 따라 액세스 시간이 크게 달라진다. (자기 테이프, 자기 드럼)
- **직접 액세스**(direct access) : 기억 장치의 각 FAT(또는 레코드, 블록) 근처로 먼저 이동한 위치부터 순서대로 읽으므로 데이터의 저장 위치에 따라 액세스 시간이 달라진다. (자기 디스크, CD-ROM, DVD)
- **임의 액세스**(random access) : 기억 장치의 장소마다 고유의 주소가 있어 **어떤 위치든 임의로 액세스할 수 있다**. 데이터의 위치에 상관없이 액세스 시간이 같다. (주 기억 장치를 구성하는 반도체 기억 장치)
- **연관 액세스**(associative access) : 임의 액세스 방식의 일종으로, 주소 대신 내용의 일부를 이용한다. 하드웨어로 구현한다. (캐시 기억 장치)

01 기억 장치 시스템의 개요

□ 물리적 유형에 따른 분류

- 기억 장치를 제작에 사용하는 물리적인 재료에 의해 분류할 수 있다. 반도체 기억 장치, 자기-표면 기억 장치, 광 저장 장치 등이 있다.

□ 물리적 특성에 따른 분류

- 전원이 끊기면 데이터의 소멸 여부에 따라 휘발성 기억 장치와 비휘발성 기억 장치로 분류
- RAM은 휘발성 기억 장치고, ROM, 플래시 메모리, 자기-표면 기억 장치, 광 저장 장치 등은 비휘발성 기억 장치다.
- 데이터를 읽으면 데이터 파괴 여부에 따라 파괴적 기억 장치와 비파괴적 기억 장치로 분류
- 자기 코어나 FRAM은 파괴적 기억 장치고, 반도체 기억 장치, 자기-표면 기억 장치, 광 저장 등은 비파괴적 기억 장치다.

01 기억 장치 시스템의 개요

2 계층적 기억 장치 시스템

❖ 기억 장치 시스템에서 액세스 속도, 가격, 용량의 관계

- 액세스 속도가 빨라질수록 비트당 가격은 높아진다.
- 용량이 증가할수록 비트당 가격은 감소하고, 액세스 속도도 낮아진다.

GB당 가격	저장 용량	액세스 시간
100만 달러	수십~수백 Byte	1ns 이하
10만 달러 1만 달러	L1 : 수십 KB L2 : 수 MB	1~10ns
1000달러	수백 MB	10~50ns
10달러	수백 GB	자기 : 10~50ms 광 : 100~500ms
1달러	수 TB	0.5s 이상

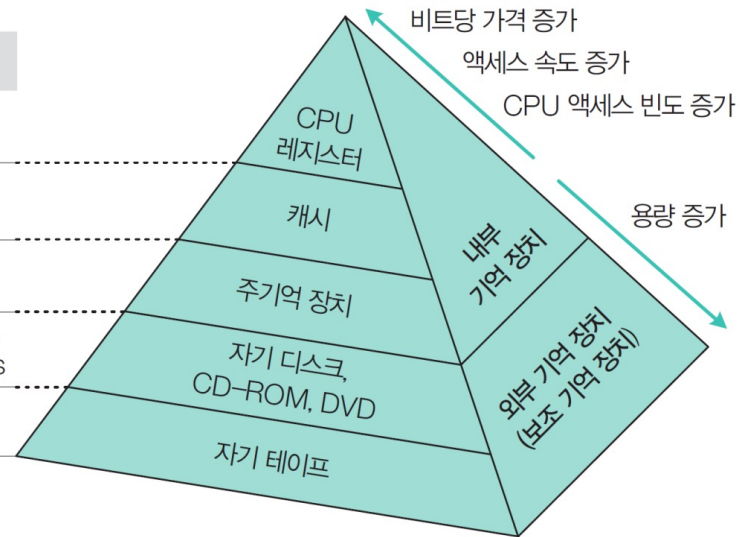


그림 6-1 기억 장치의 계층 구조

컴퓨터에서 여러 종류의 기억 장치를 계층적으로 구성하는 이유:
평균 액세스 속도를 빠르게 하면서 가격 대비 성능도 적절히 유지하기 위함

01 기억 장치 시스템의 개요

❖ 기억 장치의 계층 구조

① 레지스터	• 액세스 속도가 가장 빠르지만 비트당 가격도 가장 높아 CPU에 소량 존재한다. • RISC 계열의 CPU는 레지스터가 200개 이상이지만 보통은 수십 개 정도다.
② 캐시	• 캐시는 주기억 장치와 CPU의 속도 차를 줄이기 위해 레지스터와 주기억 장치 사이에서 CPU가 자주 사용하는 명령어나 데이터를 일시 저장하는 버퍼 역할을 한다. • 캐시는 SRAM으로 구성되는데, CPU 내부에 있으면 L1 캐시, 외부에 있으면 L2 캐시라고 하는데 요즘은 L2 캐시도 CPU 내부에 집적한다.
③ 주기억 장치	• DRAM으로 구성했다가 요즘은 SDR SDRAM이나 DDR SDRAM으로 구성한다. • 프로그램을 수행하려면 먼저 해당 프로그램의 명령어와 데이터를 주기억 장치에 적재해야 한다. • 내부 기억 장치는 고속 동작을 위해 보통 하나의 CPU 보드상에 위치한다.
④ 외부기억 장치	• 대규모 데이터를 영구 저장하기 위해 사용된다. CPU가 직접 액세스할 수 없고 제어기를 통해 액세스할 수 있다. • 보조 기억 장치라고도 한다. • 고속 액세스가 가능한 것은 자기 디스크이며, 요즘에는 플래시 메모리도 보편적으로 사용된다

02 주기억 장치

1 주기억 장치의 동작

- CPU와 주기억 장치 사이의 데이터 전송은 CPU 내부에 있는 레지스터 2개(MAR, MBR)와 제어 신호 3개(읽기, 쓰기, 칩 선택)를 통해 이루어진다.
- 메모리 주소 레지스터(MAR) : 메모리 액세스 시 특정 워드의 주소가 MAR에 전송
- 메모리 버퍼 레지스터(MBR) : 레지스터와 외부 장치 사이에서 전송되는 데이터의 통로

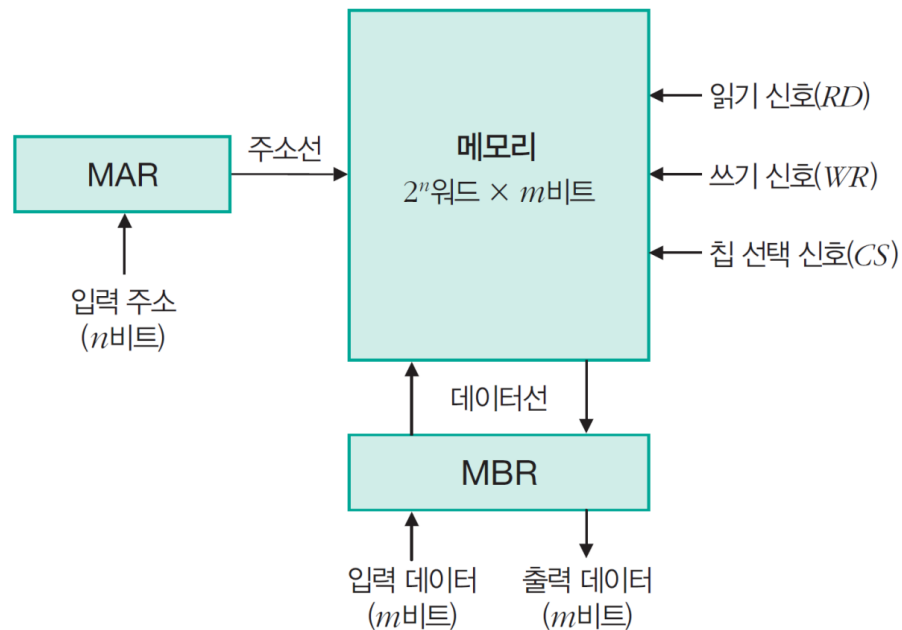


그림 6-2 주기억 장치의 동작 블록도

02 주기억 장치

□ 메모리 읽기 동작

- ① 읽으려는 메모리의 주소를 MAR로 전송한다.
- ② 칩 선택 신호와 읽기 신호를 활성화시키면 지정된 메모리의 워드가 MBR로 들어온다.

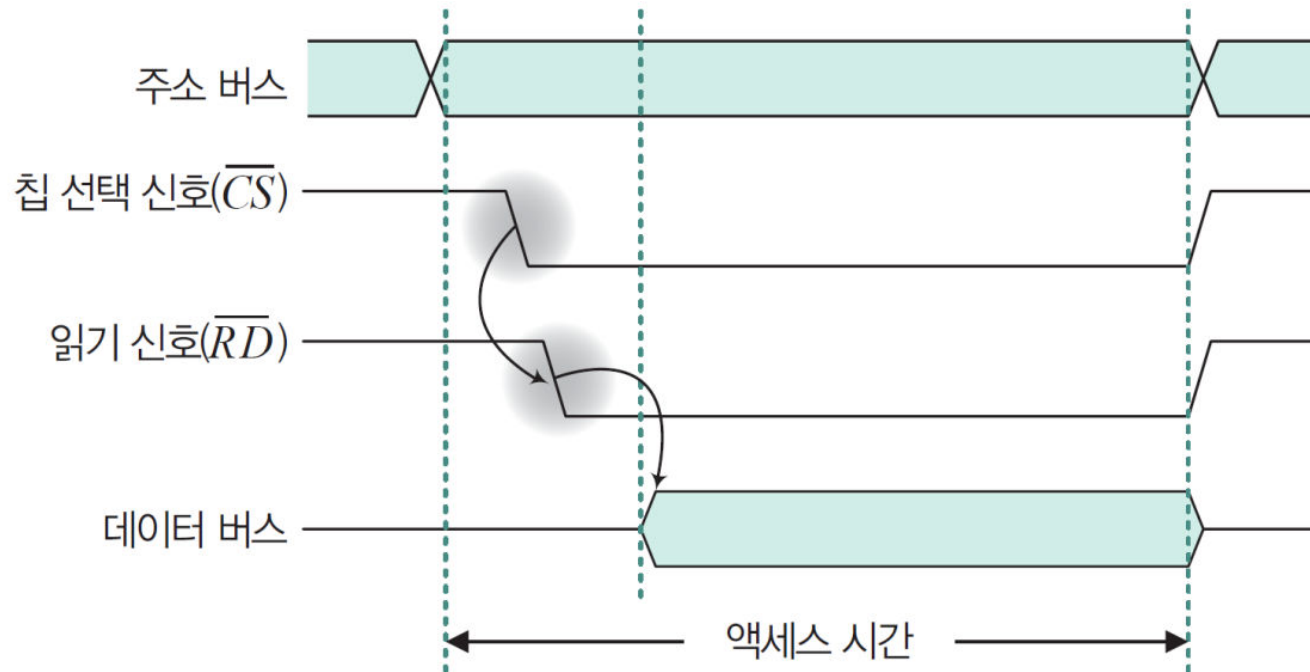


그림 6-3 메모리 읽기 동작의 타이밍도

02 주기억 장치

□ 메모리 쓰기 동작

- ① 지정된 메모리의 주소를 MAR로 전송하는 동시에 저장하려는 데이터의 워드를 MBR에 전송한다.
- ② 칩 선택 신호와 쓰기 신호를 활성화시킨다.

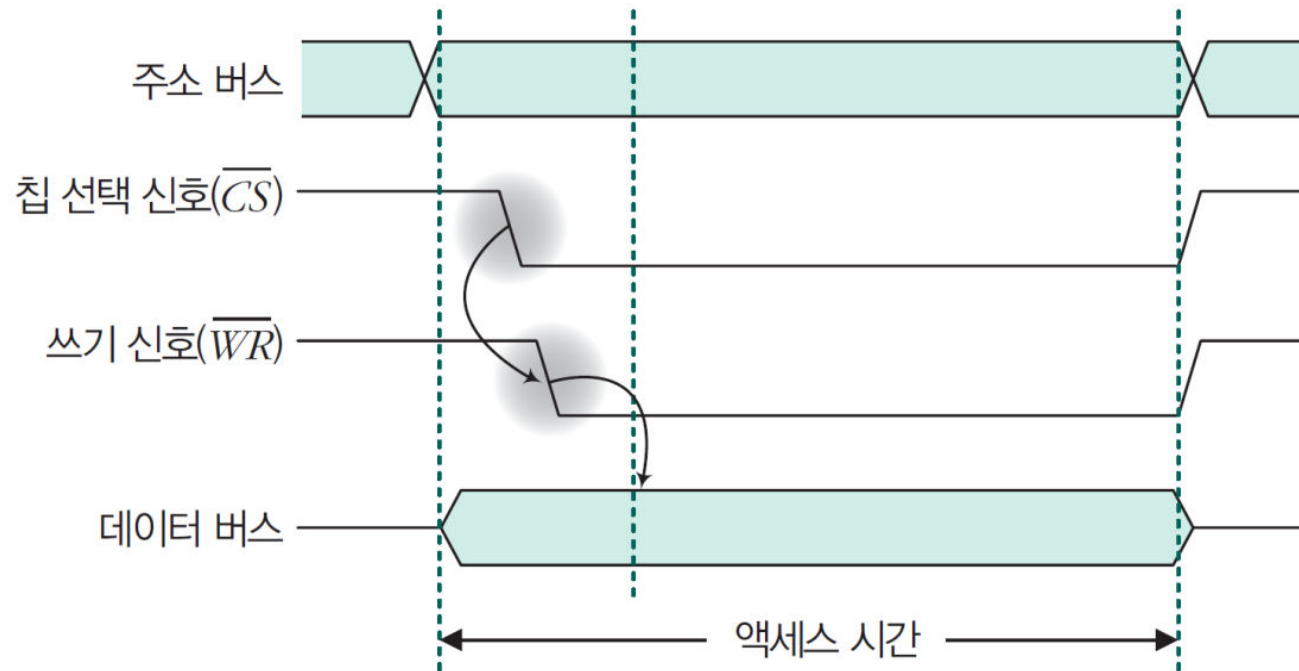


그림 6-4 메모리 쓰기 동작의 타이밍도

02 주기억 장치

■ 기억 장치의 용량 표현

- 기억 장치의 용량은 주소 버스의 길이와 지정된 주소에 들어 있는 데이터의 길이로 나타낸다.
- 주소 버스의 길이가 n 비트고, 워드당 비트 수가 m 일 때 용량은 다음과 같다.

$$\text{용량} = 2^n \times m$$

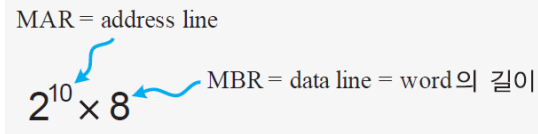
- MAR의 비트 수는 주소 버스의 길이 n 과 같으며, 워드 개수는 2^n 이다.
- MBR의 비트 수는 워드당 비트 수이므로 데이터 버스의 길이 m 과 같다.

예제 6-1 기억 장치의 용량이 1024×8 이라면 MAR과 MBR은 각각 몇 비트인가?

풀이

$1024 \times 8 = 2^{10} \times 8$ 이므로 MAR=10비트, MBR=8비트이다.

참고로 MAR = address line = address bus, MBR = data line = data bus = word의 길이이다.



End of Example

02 주기억 장치

□ 워드의 저장 방법

- 기억 장치에 저장되는 데이터를 구별하려면 주소와 데이터 단위를 정의해야 한다.
- 주소에는 0번지부터 고유의 일련번호를 부여한다.
- 각 주소에 데이터가 1바이트나 워드 단위로 저장되므로 바이트 주소와 워드 주소로 분류할 수 있다.
- 기억 장치에 바이트를 배열하는 방법을 **엔디안**(endian)이라고 한다.

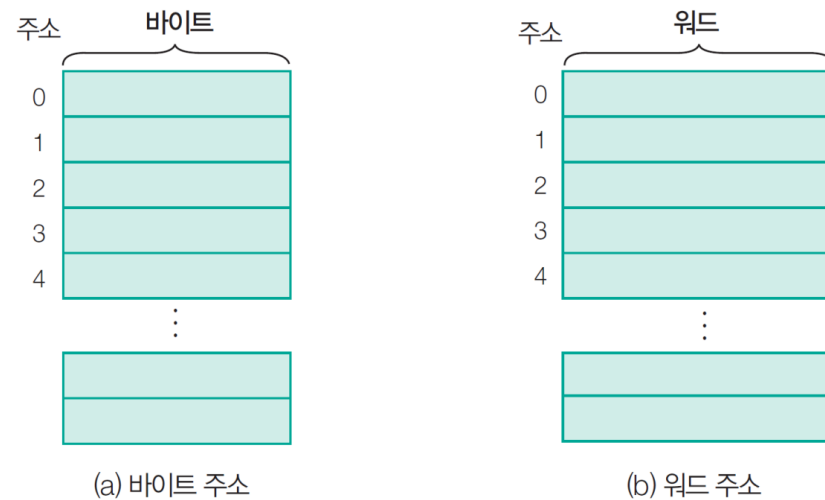
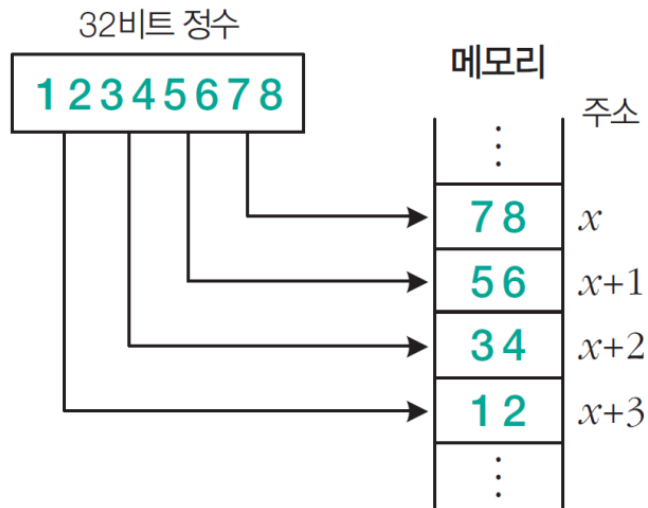


그림 6-5 바이트 주소와 워드 주소

02 주기억 장치

❖ 리틀 엔디안(little endian)

- 하위 바이트를 낮은 주소에 저장하는 방법이다. 오른쪽에서 왼쪽으로 저장한다.
- 이는 산술 연산이 주소가 낮은 쪽에서 높은 쪽으로 처리되는 순서와 같다.
- 홀수와 짝수를 검사할 때도 첫 바이트만 확인하면 되므로 빠르다.
- 리눅스, 인텔 계열의 CPU, AMD 계열의 CPU에서 사용하는 방식이다.



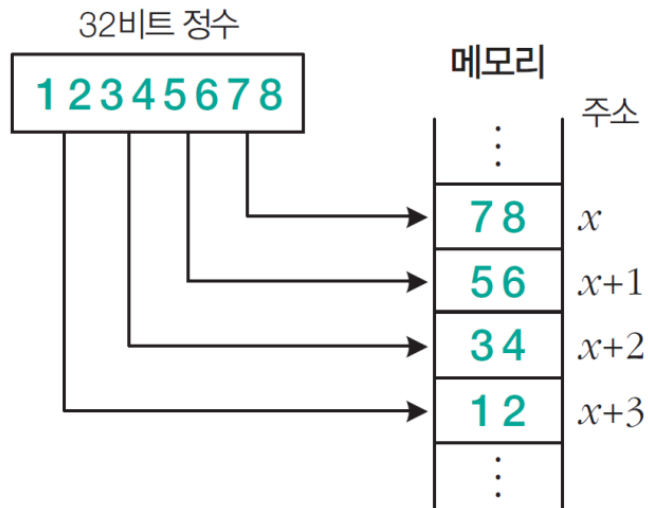
(a) 리틀 엔디안

그림 6-6 워드의 저장 방법(바이트 주소인 경우)

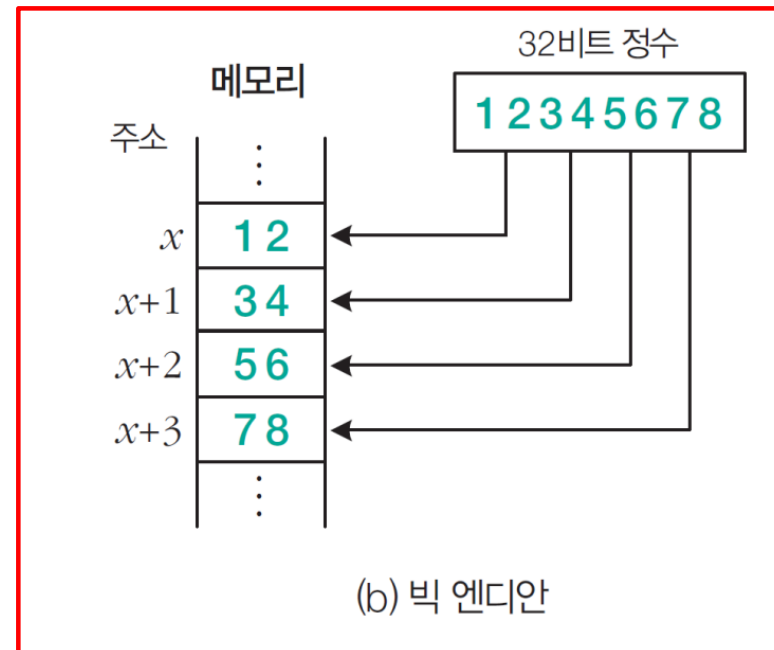
02 주기억 장치

❖ 빅 엔디안(big endian)

- 상위 바이트를 낮은 주소에 저장하는 방법으로 왼쪽에서 오른쪽으로 저장한다.
- 숫자를 읽고 쓰는 일반적인 방식과 같아 사람이 읽기 편하다.
- IBM이나 모토로라의 CPU가 사용하는 방식이다. 또 TCP/IP 전송도 이 방식을 사용한다.



(a) 리틀 엔디안



(b) 빅 엔디안

그림 6-6 워드의 저장 방법(바이트 주소인 경우)

❖ 바이 엔디안(biendian)

- 엔디안을 선택할 수 있도록 설계된 방법으로 ARM, PowerPC, DED Alpha, MIPS 등이 있다.

02 주기억 장치

2 반도체 기억 장치

- 반도체 메모리는 다양한 관점으로 분류할 수 있으나 대표적으로 쓰기 기능, 휘발성/비휘발성, 재사용 여부, 기억 방식 등에 따라 분류한다.
- 읽기와 쓰기를 모두 수행할 수 있는 메모리를 RWM(Read and Write Memory), 읽기만 가능한 메모리를 ROM이라고 한다. 일반적으로 RAM은 RWM 메모리를 가리킨다.

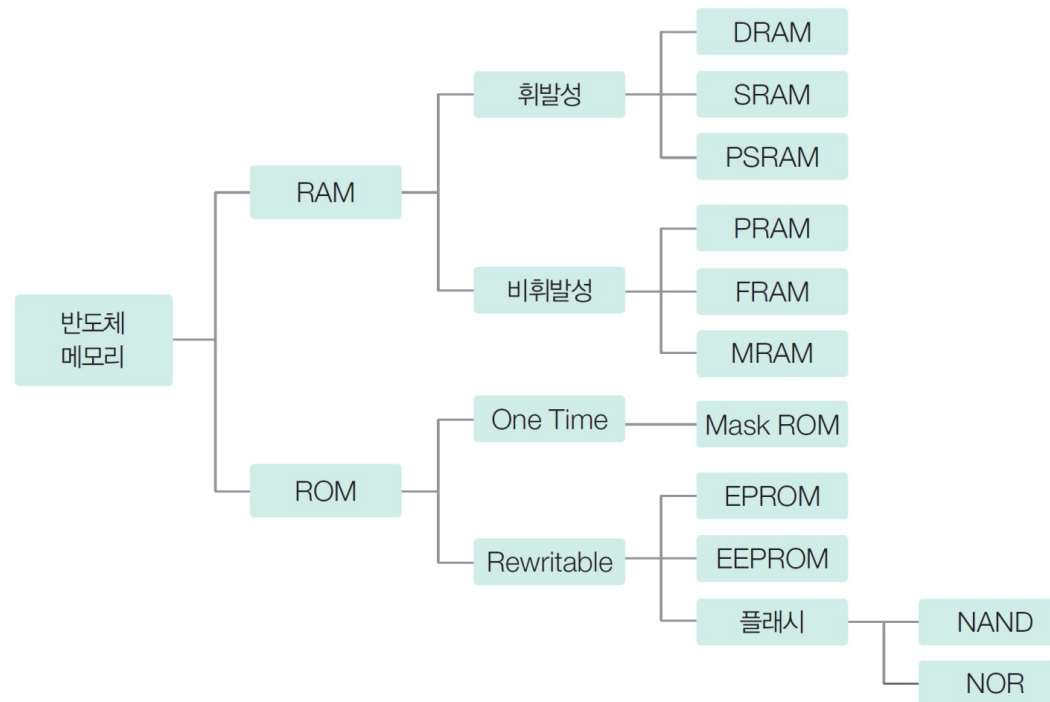


그림 6-7 반도체 메모리의 분류

02 주기억 장치

❖ RAM

- 전원이 꺼지면 저장 내용이 지워지는 휘발성 메모리

표 6-2 DRAM, SRAM, PSRAM의 특성

구분	특성
DRAM	2진 정보를 커패시터에 공급되는 전하 형태로 보관한다. 커패시터에 사용되는 전하는 시간이 경과하면 방전되므로 일정한 시간 안에 재충전 _{refresh} 해야 한다.
SRAM	2진 정보를 저장하는 플립플롭으로 구성되며, 저장된 정보는 전원이 공급되는 동안 보존된다.
PSRAM	DRAM과 SRAM의 장점을 취한 SRAM처럼 보이는 DRAM이다. 하드웨어적으로 충전하는 회로가 DRAM에 포함되어 있으므로 사용자는 SRAM처럼 사용할 수 있다.

❖ 차세대 메모리

- 비휘발성이고 고속으로 데이터를 액세스 가능

표 6-3 PRAM, FRAM, MRAM의 특성

구분	특성
PRAM	상 _{phase} 변화를 일으키는 물질을 이용한다. 고속이고 고집적화가 가능하지만 쓰기 시간이 길다.
FRAM	강유전체의 분극 특성을 이용한다. 고속이고 저전력으로 구성할 수 있으나 내구성이 취약하다.
MRAM	자성체를 이용한다. 고속이고 내구성이 좋으나 제조 비용이 상대적으로 높다.

02 주기억 장치

□ ROM(Read Only Memory)

- 저장된 데이터를 읽을 수는 있으나, 별도의 장치 없이는 변경할 수 없다.
- 변경할 필요가 없는 프로그램이나 데이터를 저장하는 데 사용된다.
- 컴퓨터 시스템에서는 RAM과 함께 주기억 장치의 일부분으로 ROM을 사용하고 있다.

- 시스템 초기화 프로그램 및 진단 프로그램
- 빈번히 사용되는 함수 및 서브루틴
- 제어 장치의 마이크로 프로그램

02 주기억 장치

❖ ROM의 기본 구조

- ROM은 AND 게이트와 OR 게이트로 구성된 조합 논리 회로다.
- AND 게이트는 디코더를 구성하고 OR 게이트는 디코더의 출력인 최소항을 합하므로 OR 게이트의 수는 ROM의 출력선의 수와 같다.

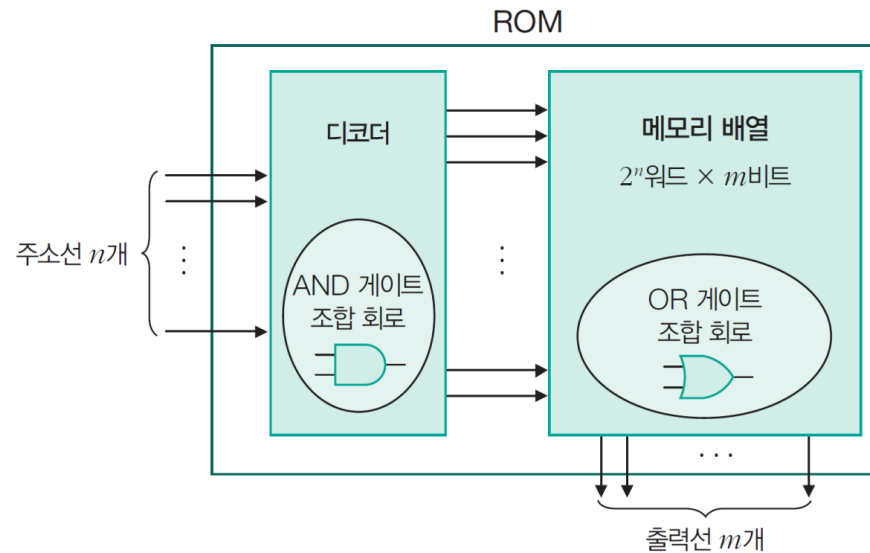


그림 6-8 ROM의 기본 구조

02 주기억 장치

❖ 32×4 ROM의 내부 논리 구조

- 5비트 주소가 입력되어 출력 32개 중 하나만 활성화된다.
- 디코더의 출력 32개가 각각 퓨즈로 연결되어 OR 게이트에 입력되므로 내부 퓨즈는 $32 \times 4 = 128$ 개다.
- 퓨즈를 통해 연결된 입력은 프로그램으로 절단할 수 있다.

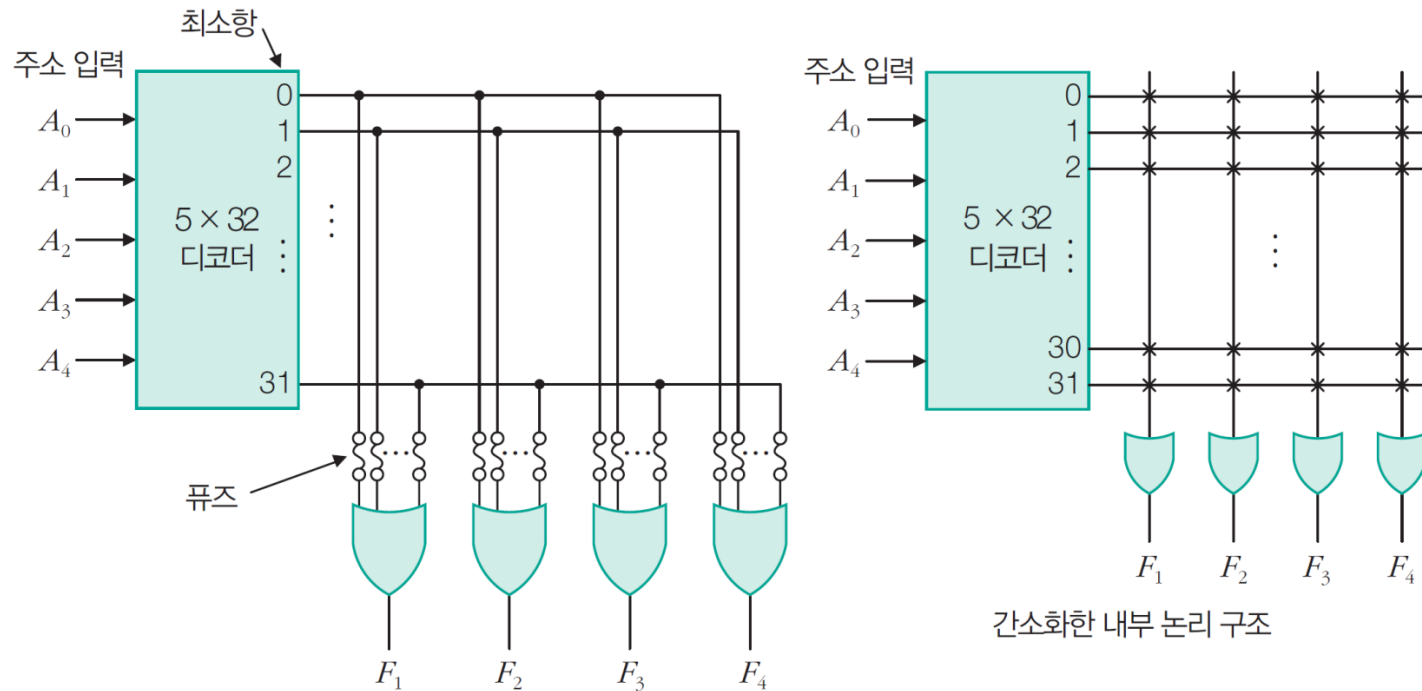


그림 6-9 32×4 ROM의 내부 논리 구조

02 주기억 장치

❖ 1Kbyte ROM 블록도

- 8비트로 구성된 기억 장소들이 1024($=2^{10}$)개가 배열된 경우이므로 주소선이 10개 필요
- $\overline{RD}$ 는 읽기 신호로, ROM은 읽기만 가능하기 때문에 $\overline{RD}$ 신호만 있으면 된다.
- 칩 여러 개로 구성된 기억 장치에서는 칩 선택 신호 $\overline{CS}$ 로 칩이 선택된다.
- $\overline{CS}$ 와 $\overline{RD}$ 신호가 활성화되면 주소가 지정하는 기억 장소에서 데이터를 읽어 데이터 버스에 실는다.
- 대부분의 ROM은 데이터 출력선이 8개인 구조를 사용하며, 용량도 바이트 단위로 표시한다.
- $\overline{CS}$ 와 $\overline{RD}$ 신호는 0일 때 활성화되는 active-low 신호를 가정했다.

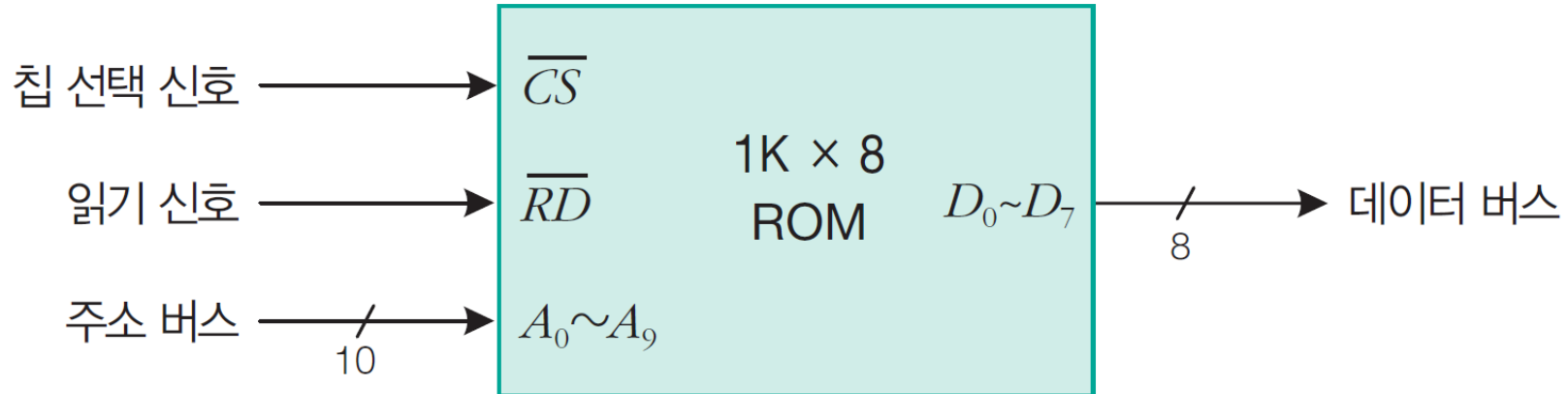
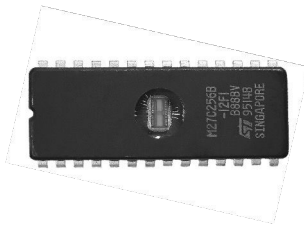


그림 6-10 1Kbyte ROM의 블록도

02 주기억 장치

❖ ROM의 종류

마스크 ROM (mask ROM)	• 제조 과정에서 데이터를 영구적으로 저장하며, 저장된 것은 절대 변경할 수 없다. • 동일한 형태가 대량으로 필요할 때는 Mask ROM이 경제적이다.
PROM (programmable ROM)	• 사용자가 ROM 라이터를 이용하여 프로그램을 할 수 있다. • 일단 프로그램을 하면 퓨즈의 연결 형태가 그대로 유지되며, 변경할 수 없다.
EPROM (erasable PROM)	• 퓨즈가 절단되어도 모든 퓨즈들이 절단되지 않은 초기 상태로 복원할 수 있는 ROM이다. • 복원하는 과정은 일정 시간 자외선을 쬔이면 된다.
EEPROM (electrically EPROM)	• EPROM과 같으나, 복원 과정에서 자외선 대신에 전기 신호를 사용하여 지우는 PROM이다.



EPROM(UVEPROM)



02 주기억 장치

❖ 플래시메모리(flash memory)

- 플래시메모리는 **블록 단위로 읽기 · 쓰기 · 지우기가 가능**한 EEPROM의 한 종류
- 비휘발성 ROM의 장점과 정보의 입출력이 자유로운 RAM의 장점을 동시에 지닌 반도체 메모리
- 속도가 빠르며 전력소모가 적고, CD나 DVD처럼 드라이브를 장착해야 하는 번거로움이 없다.
- 2001년부터 USB 드라이브, thumb 드라이브라는 이름으로 소개되었으며, 이후 디지털 캠코더, 휴대폰, 디지털 카메라 등의 휴대용 디지털 기기에 사용되면서 사용량이 급격히 증가하였다.
- 반도체 칩 내부의 전자회로 형태에 따라 **NAND 플래시**와 **NOR 플래시**로 나뉜다.
- NAND 플래시는 대용량화에 유리하고 쓰기 및 지우기 속도가 빠르다.
- NOR 플래시는 읽기 속도가 빠른 장점을 갖고 있다.

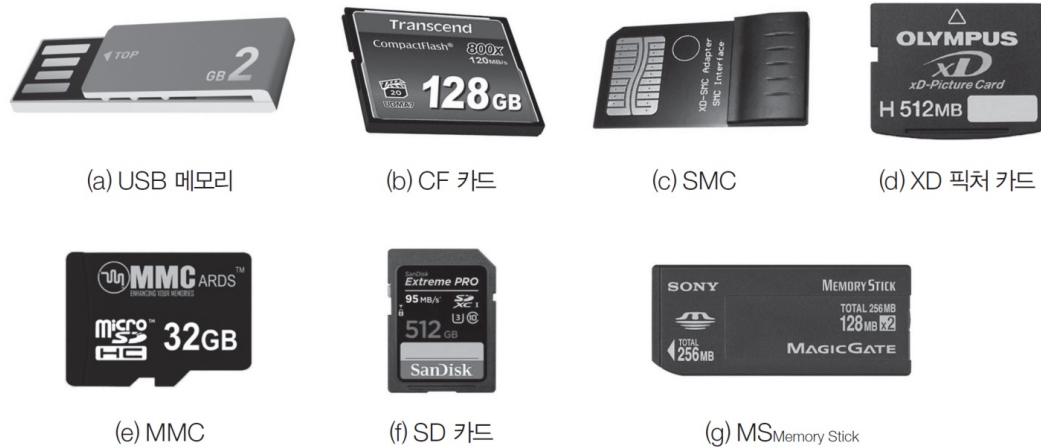


그림 6-11 플래시 메모리를 응용한 제품군

02 주기억 장치

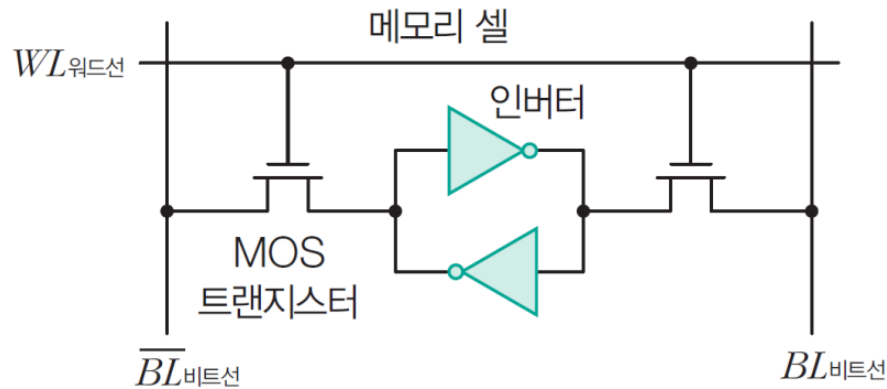
❑ RAM(random access memory)

- RAM은 휘발성이어서 사용하려면 전원을 계속 공급해야 하므로 일시적인 저장 장치로만 활용된다.
- RAM은 데이터의 읽기와 쓰기가 모두 가능하다.
- 임의 액세스 방식을 사용해 CPU가 지정하는 주소에 있는 정보를 직접 액세스할 수 있어 메모리의 위치에 관계없이 액세스 시간이 동일하다.
- **SRAM**은 플립플롭을 사용해 정보를 저장하지만, **DRAM**은 커패시터에 전하를 충전하는 방식으로 정보를 저장한다.

02 주기억 장치

❖ SRAM의 메모리 셀 구조 및 동작

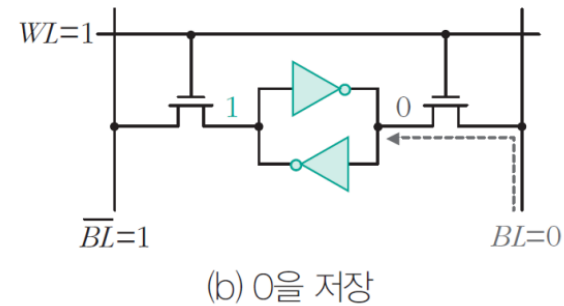
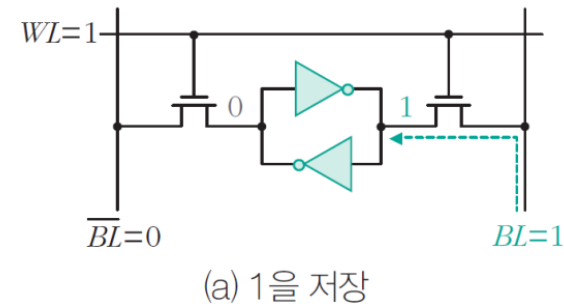
- 주소선으로 메모리 셀이 선택되면 해당 메모리 셀의 워드선(WL)에는 논리 1이 입력된다.
- 읽기와 쓰기는 비트선(BL)을 통해 이루어지므로 $WL=1$ 로 하면 MOS 트랜지스터 2개가 on이 되고, $BL=1, \overline{BL}=0$ 으로 하면 비트선 $BL=1$ 이 플립플롭으로 전송되어 비트 1이 저장된다.
- 또 $BL=0, \overline{BL}=1$ 하면 비트선 $BL=0$ 이 플립플롭으로 전송되어 비트 0이 저장된다.



(a) SRAM

그림 6-12 RAM의 메모리 셀 구조

SRAM 셀의 쓰기 동작



02 주기억 장치

❖ DRAM의 메모리 셀 구조 및 동작

- MOS 트랜지스터는 스위치로 동작하며, 커패시터에 저장된 전하의 유무에 따라 정보를 저장한다.
- 워드선은 주소선에 연결되고, 비트선은 데이터선에 연결된다.
- $WL=1$ 이면 MOS 트랜지스터는 on되는데, 이때 $BL=1$ 이면 커패시터에는 전하가 충전되어 논리 1이 저장된다. 반면 $BL=0$ 이면 커패시터에는 충전되지 않으므로 논리 0이 저장된다.
- 커패시터는 방전되므로 DRAM은 데이터의 저장 상태를 유지하기 위해 주기적으로 재충전(refresh)해야 한다.

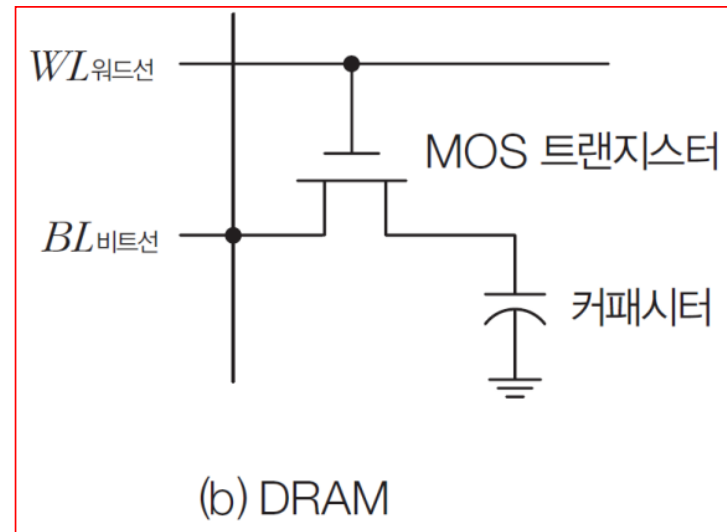


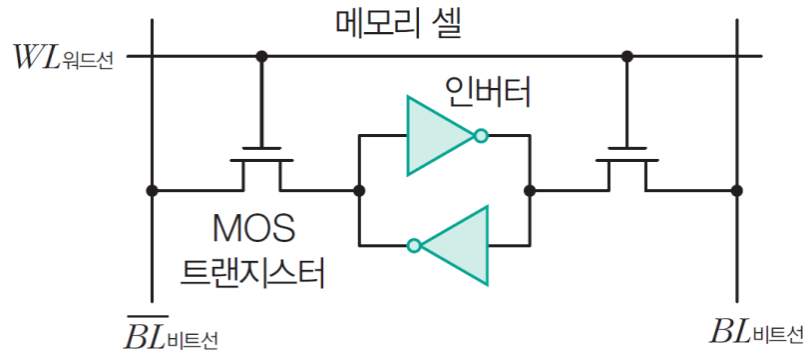
그림 6-12 RAM의 메모리 셀 구조

02 주기억 장치

❖ SRAM과 DRAM의 특징 비교

표 6-4 SRAM과 DRAM의 특징 비교

구분	SRAM	DRAM
구성 소자	플립플롭	커패시터
집적도	낮음	높음
전력 소모	많음	적음
동작 속도	빠름	느림
가격	고가	저가
재충전 여부	필요 없음	필요함
용도	캐시	주기억 장치



(a) SRAM

그림 6-12 RAM의 메모리 셀 구조

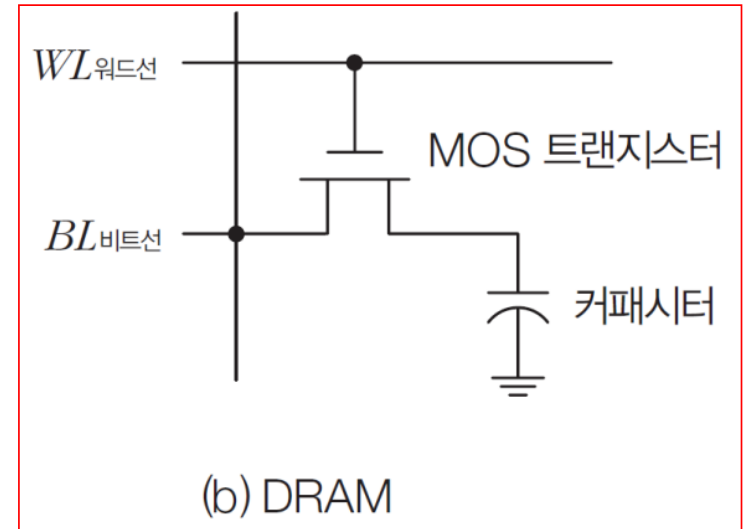


그림 6-12 RAM의 메모리 셀 구조

02 주기억 장치

예제 6-2

내부적으로 1K×1K 정방향 배열로 구성된 1M×1 DRAM에서 충전으로 발생하는 대역폭 손실은 얼마인가? 단, 행 하나를 충전하는 데 100ns가 소요되며, 데이터 유실을 막기 위해 최소한 10ms마다 재충전해야 한다.

풀이

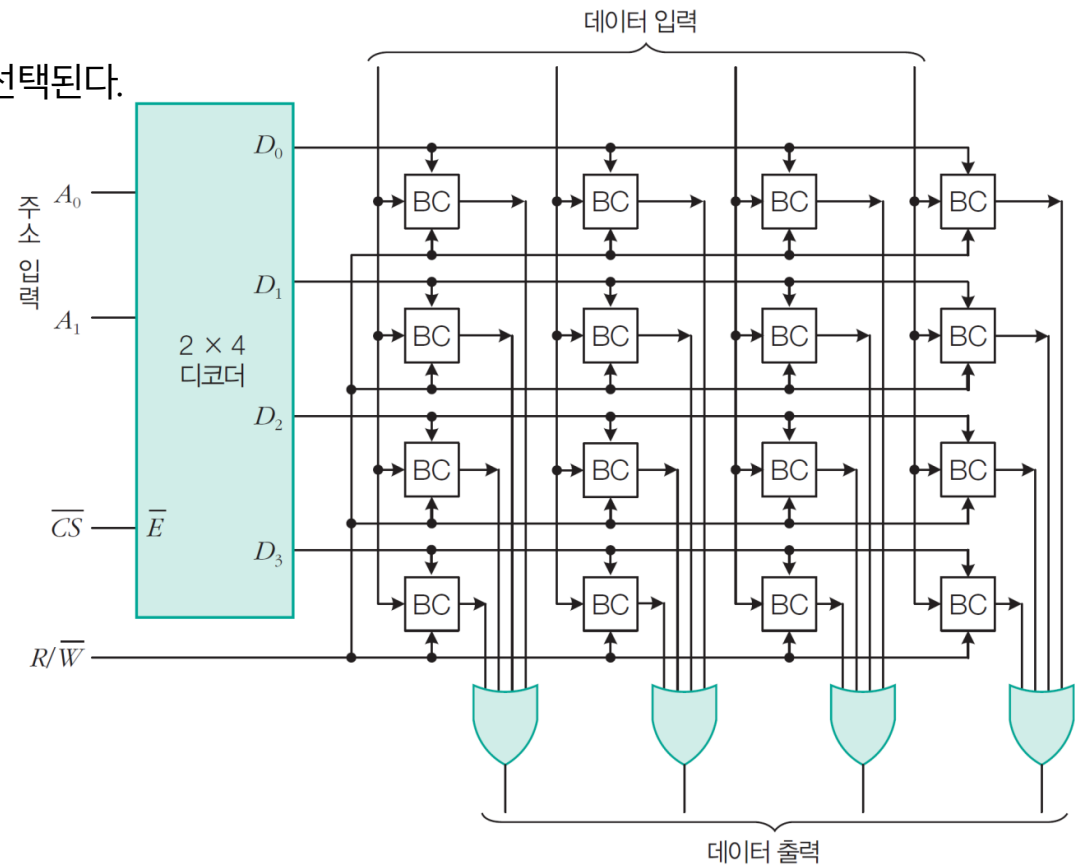
1K×1K 정방향 배열이므로 1K비트로 구성된 열이 1K개 존재한다. 한 번에 하나의 행을 재충전할 수 있으므로 전체 메모리 배열을 충전하는 데 $1K \times 100ns = 0.1ms$ 가 걸린다. 따라서 10ms마다 재충전하기 위해 0.1ms를 사용해야 하므로 전체 대역폭의 1%(=0.1/10)만큼 손실이 발생한다. 행의 수가 많아지면 대역폭 손실은 커질 것이다.

End of Example

02 주기억 장치

❖ SRAM의 내부 구조와 원리(4×4 SRAM)

- 4비트로 이루어진 4개의 기억 장소들로 구성 (실재 존재하지 않으며, 설명의 편의상 사용)
- 주소 비트 수 = 2, 데이터 데이터 입출력 선의 수 = 4
- $\overline{CS} = 0$ 이면 주소 입력 A_1, A_0 값에 따라 워드 4개 중 하나가 선택된다.
- $R/\overline{W} = 1$ 이면 읽기, 0이면 쓰기



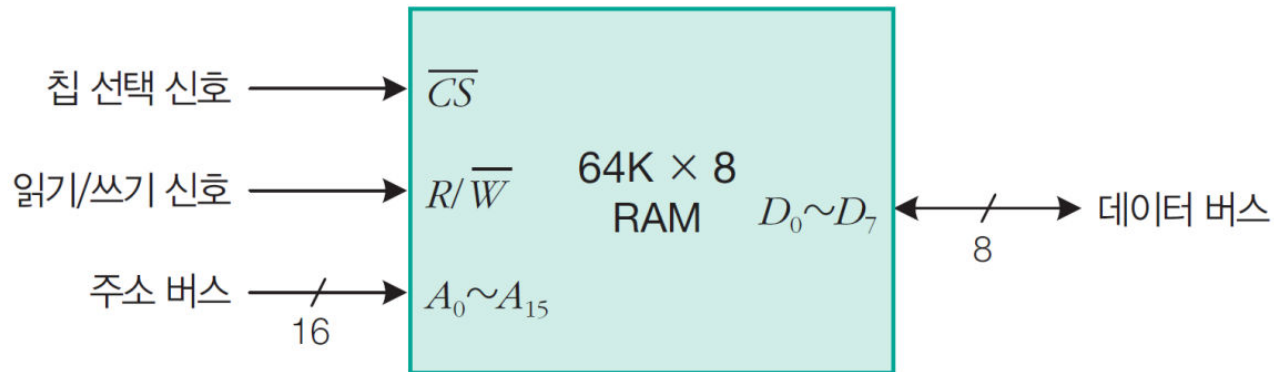
BC: Binary Cell; Bit Cell; 1bit 기억소자

그림 6-13 4×4 SRAM의 기본 구조

02 주기억 장치

❖ 64Kbyte RAM의 블록도

- 64K×8bit는 8비트로 된 기억 장소들이 64K($=2^{16}$)개 배열된 경우이므로 주소선은 16개 필요하다.
- 데이터 선은 8개 필요하다.
- $\overline{CS}=1$ 이면 제어 신호선들이나 입출력선들은 하이임피던스 상태가 된다.



(a) RAM의 블록도

$\overline{CS}$	$R/\overline{W}$	동작
1	×	선택되지 않음
0	1	읽기 동작
0	0	쓰기 동작

(b) 제어 신호에 따른 RAM의 동작

그림 6-14 64Kbyte RAM의 블록도

02 주기억 장치

3 기억 장치 모듈의 설계

□ 워드 길이 확장

- 기억장치 칩의 데이터 I/O 비트 수가 워드 길이보다 적은 경우
→ 여러 개의 칩들을 병렬로 접속하여 기억장치 모듈을 구성
- 각 칩의 주소 수는 기억 장치의 주소 수와 같은 $16(=2^4)$ 개이므로 전체 주소 공간은 0000~1111번지가 된다.

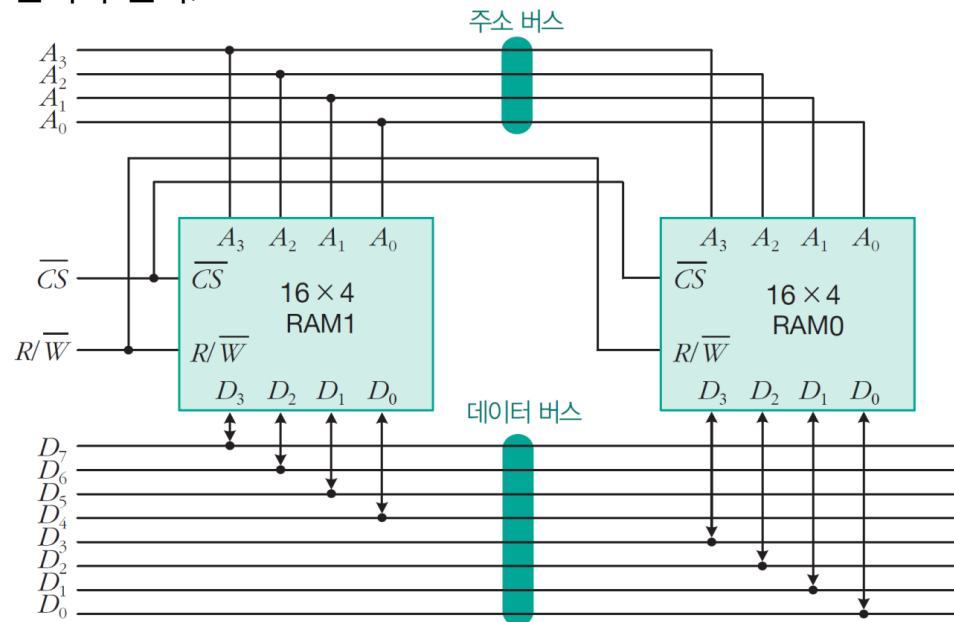


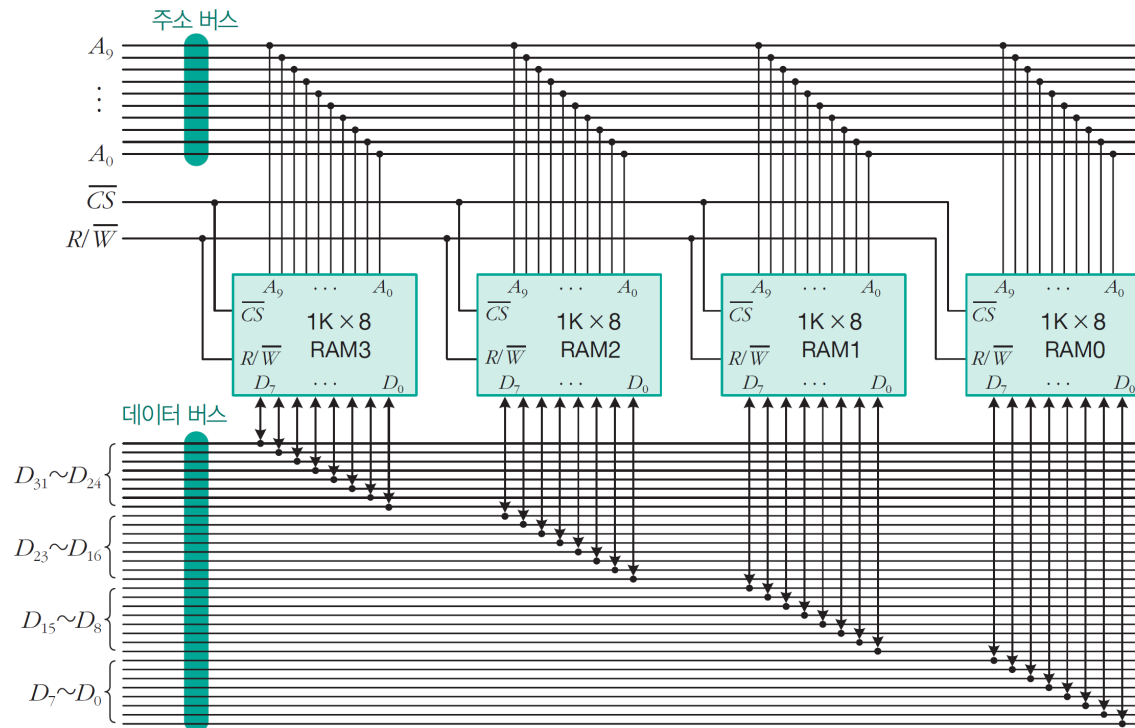
그림 6-15 워드 길이 확장(16×4 RAM 2개를 이용해 16×8 RAM으로 확장)

02 주기억 장치

예제 6-3 1K×8 RAM 4개를 사용해 1K×32 RAM을 구성하여라.

풀이

- 주소선 10개(10비트)가 사용되므로 주소 공간은 000H~3FFH($0000000000_{(2)} \sim 1111111111_{(2)}$)번지다.



End of Example

02 주기억 장치

□ 워드 용량 확장

- 필요한 기억장소의 수가 각 기억장치 칩의 기억장소 수보다 많은 경우
→ 여러 개의 칩들을 직렬 접속하여 기억장치 모듈을 구성
- RAM0의 주소 공간 범위 : $A_4A_3A_2A_1A_0 = 00000 \sim 01111$
- RAM1의 주소 공간 범위 : $A_4A_3A_2A_1A_0 = 10000 \sim 11111$

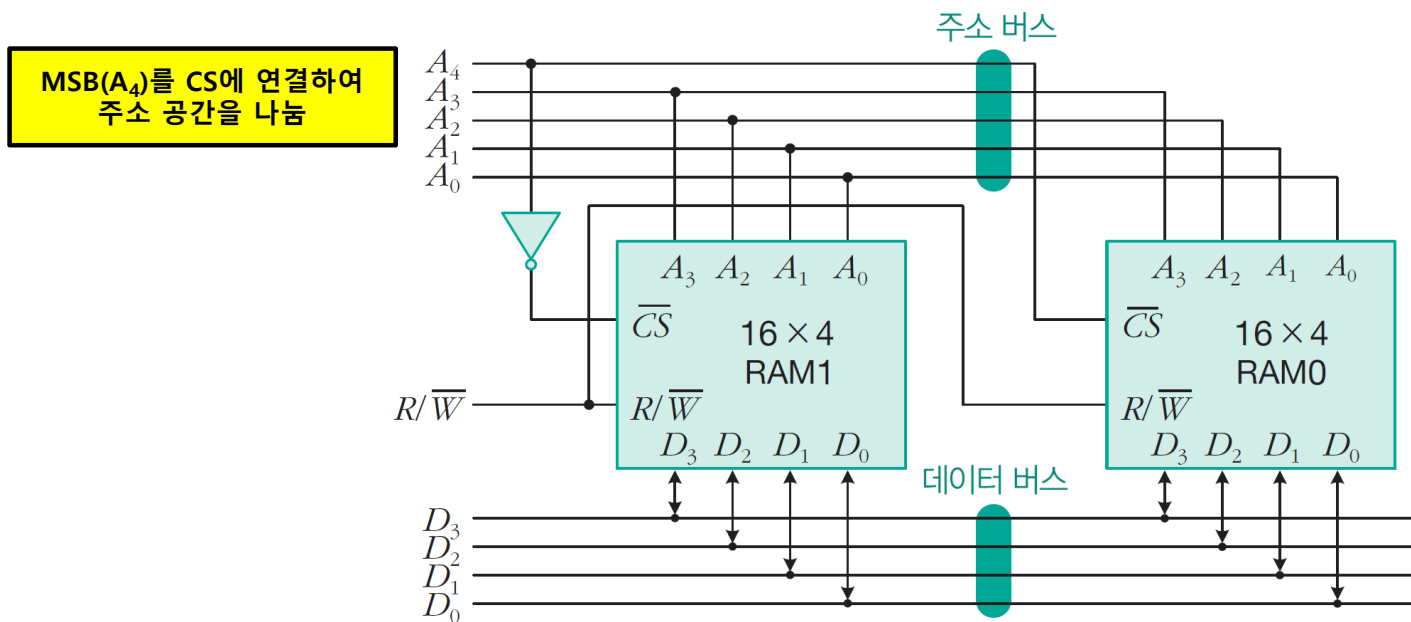


그림 6-16 워드 용량 확장(16×4 RAM 2개를 사용해 32×4 RAM으로 확장)

02 주기억 장치

예제 6-4 1K×8 RAM 4개를 사용해 4K×8 RAM을 구성하여라.

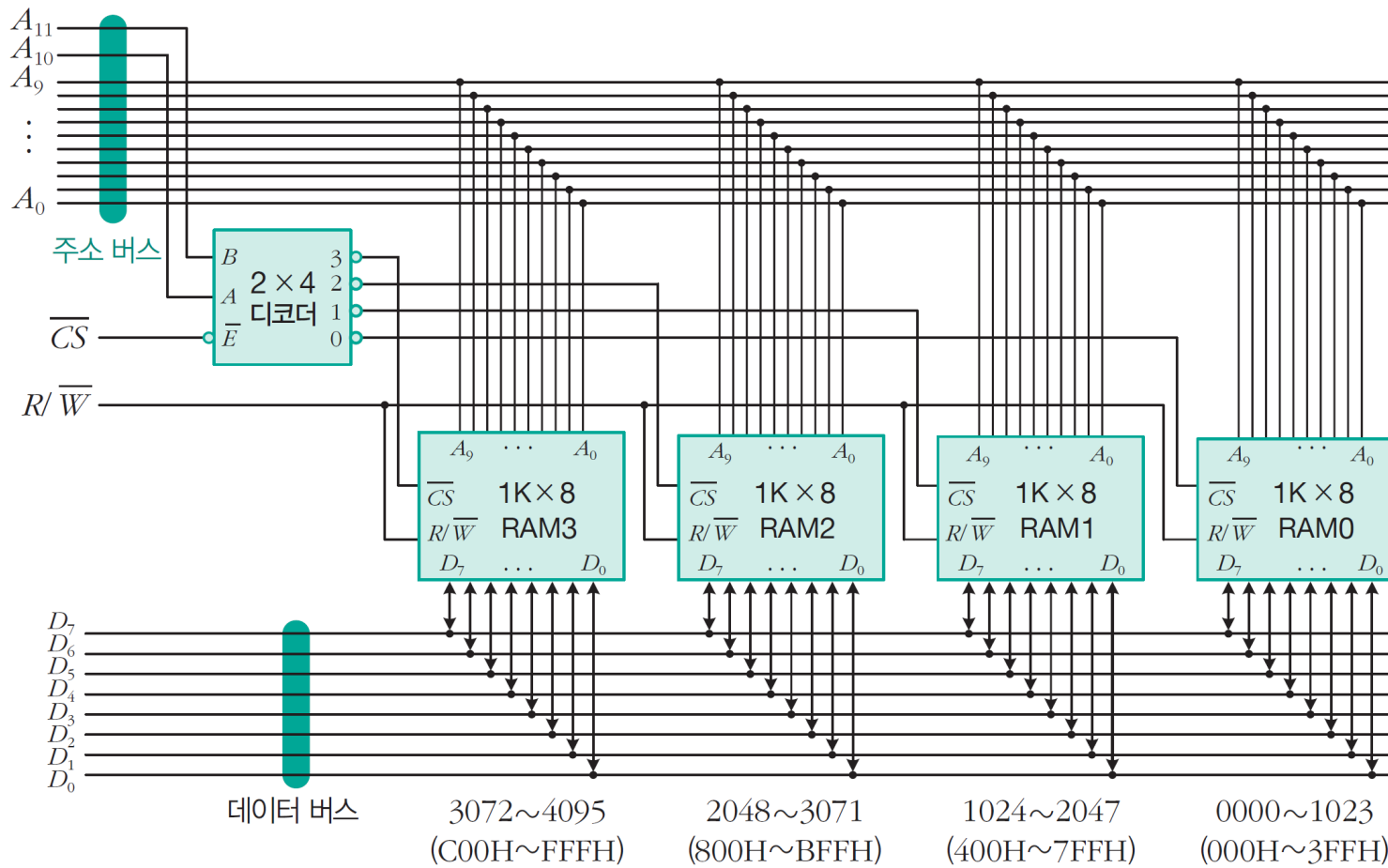
풀이

- 주소 비트(12개: A11~A0) 접속 방법
 - 상위 2비트(A11, A10) : 주소 해독기를 이용하여 4개의 칩 선택(chip select) 신호 발생
 - 하위 10비트 (A9~A0) : 모든 칩들에 공통으로 접속
- 전체 주소 영역 : 000H ~ FFFH
- 데이터 버스 : 모든 기억장치 칩에 공통 접속 → 한 번에 8비트씩 액세스

메모리 칩	주소 공간(16진수)	주소 비트											
		$A_{11} A_{10} A_9 A_8$				$A_7 A_6 A_5 A_4$				$A_3 A_2 A_1 A_0$			
RAM0	000H~3FFH	0	0	0	0	0	0	0	0	0	0	0	0
		0	0	1	1	1	1	1	1	1	1	1	1
RAM1	400H~7FFH	0	1	0	0	0	0	0	0	0	0	0	0
		0	1	1	1	1	1	1	1	1	1	1	1
RAM2	800H~BFFH	1	0	0	0	0	0	0	0	0	0	0	0
		1	0	1	1	1	1	1	1	1	1	1	1
RAM3	C00H~FFFH	1	1	0	0	0	0	0	0	0	0	0	0
		1	1	1	1	1	1	1	1	1	1	1	1

각 RAM에 지정되는 주소 영역

02 주기억 장치



End of Example

02 주기억 장치

□ 8비트 CPU의 주기억 장치 설계

❖ 기억장치 모듈의 설계 순서

- ① 컴퓨터시스템에 필요한 기억장치 용량 결정
- ② 사용할 칩들을 결정하고, 주소 표(address map)를 작성
- ③ 세부 회로 설계

❖ 8-비트 CPU를 위한 기억장치의 설계

- 용량 : 1Kbyte ROM, 2Kbyte RAM
- 주소 영역 : ROM = 0번지부터, RAM = 400H 번지부터
- 사용 가능한 칩들 : 1K×8bit ROM, 512×8bit RAMs

표 6-5 간단한 주기억 장치의 메모리 맵

메모리 칩	주요 공간(16진수)	주소 비트											
		A ₁₁	A ₁₀	A ₉	A ₈	A ₇	A ₆	A ₅	A ₄	A ₃	A ₂	A ₁	A ₀
ROM	000H~3FFH	0	0	×	×	×	×	×	×	×	×	×	×
RAM0	400H~4FFH	0	1	0	0	×	×	×	×	×	×	×	×
RAM1	500H~5FFH	0	1	0	1	×	×	×	×	×	×	×	×
RAM2	600H~6FFH	0	1	1	0	×	×	×	×	×	×	×	×
RAM3	700H~7FFH	0	1	1	1	×	×	×	×	×	×	×	×

02 주기억 장치

표 6-5 간단한 주기억 장치의 메모리 맵

메모리 칩	주요 공간(16진수)	주소 비트											
		A ₁₁	A ₁₀	A ₉	A ₈	A ₇	A ₆	A ₅	A ₄	A ₃	A ₂	A ₁	A ₀
ROM	000H~3FFH	0	0	x	x	x	x	x	x	x	x	x	x
RAM0	400H~4FFH	0	1	0	0	x	x	x	x	x	x	x	x
RAM1	500H~5FFH	0	1	0	1	x	x	x	x	x	x	x	x
RAM2	600H~6FFH	0	1	1	0	x	x	x	x	x	x	x	x
RAM3	700H~7FFH	0	1	1	1	x	x	x	x	x	x	x	x

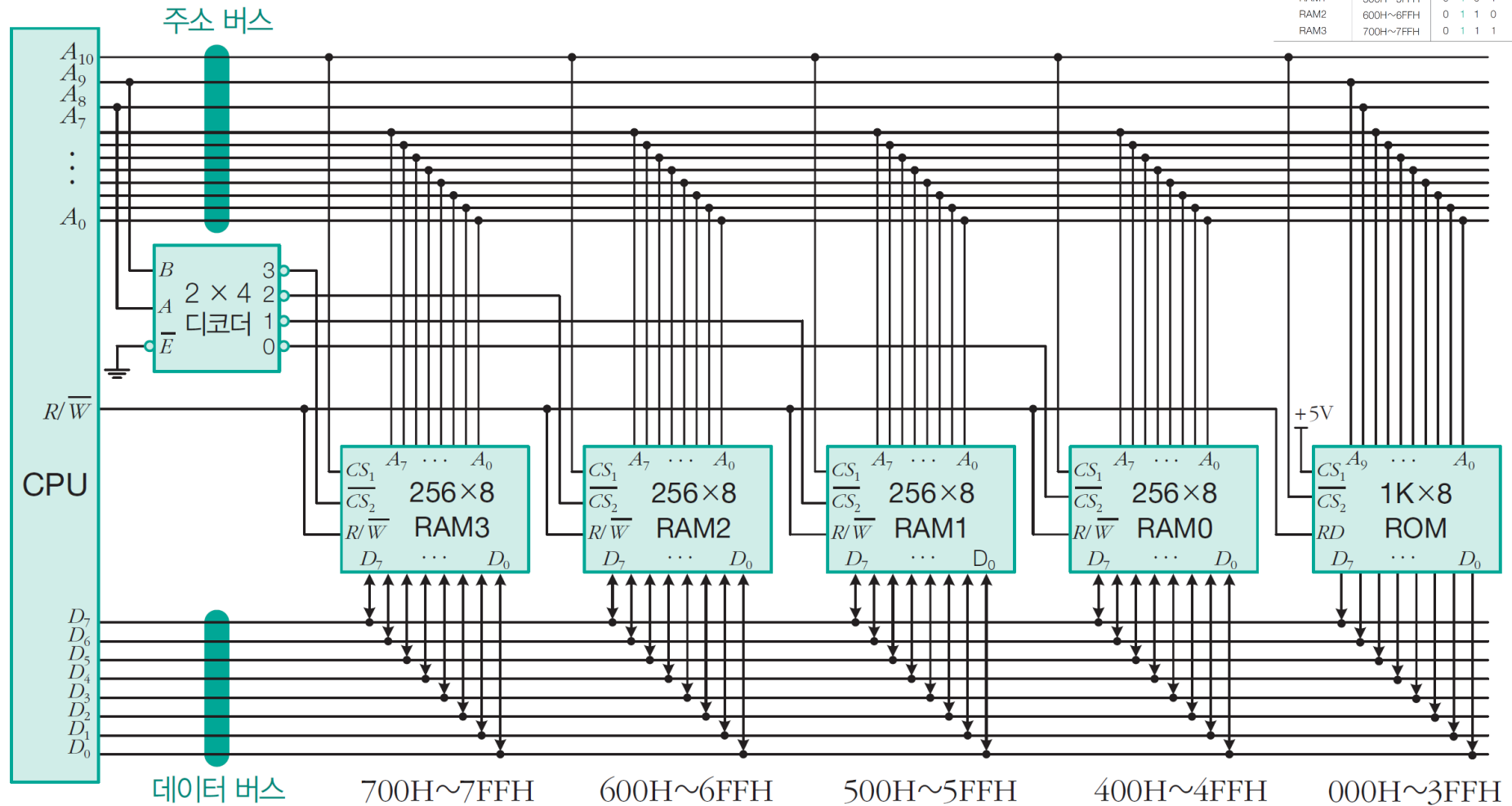


그림 6-17 8비트 CPU의 주기억 장치 설계

02 주기억 장치

지금은
사용안함



SIMM



DIMM



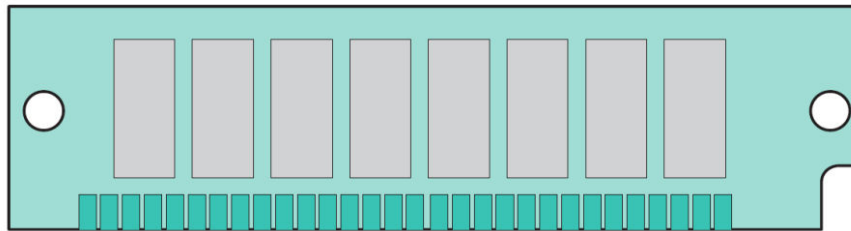
SODIMM

□ SIMM과 DIMM

❖ SIMM(Single-In-line Memory Module)

- SIMM은 30핀과 72핀 두 종류가 있다. 주요 차이점은 데이터 경로의 크기에 있다.
- SIMM의 메모리 용량은 256Kbyte에서 32Mbyte까지 다양하다.
- 일반적으로 30핀 SIMM은 8비트 데이터 버스용으로 설계되었다.
- 72핀 SIMM은 32비트 데이터 버스를 수용할 수 있으며, 64비트 데이터 버스의 경우에는 SIMM 2개가 필요하다.

(a) 30핀 SIMM



(b) 72핀 SIMM

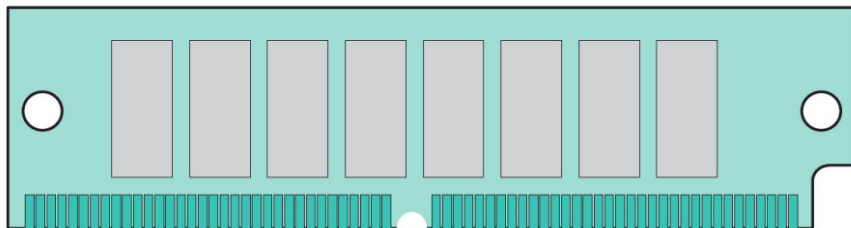


그림 6-18 30핀과 72핀 SIMM

02 주기억 장치

❖ DIMM(Dual-In-line Module Memory)

- DIMM은 입력핀과 출력핀을 보드 양면에 분산시켰다.
- DIMM은 보통 72핀, 100핀, 144핀, 168핀을 가지고 있으며, 32비트와 64비트 데이터 경로를 모두 수용할 수 있다.
- 일반적으로 DIMM의 메모리 용량은 4Mbyte에서 512Mbyte이다.
- SIMM용 소켓과 DIMM용 소켓은 물론 다르며, 바꾸어서 사용할 수 없다.

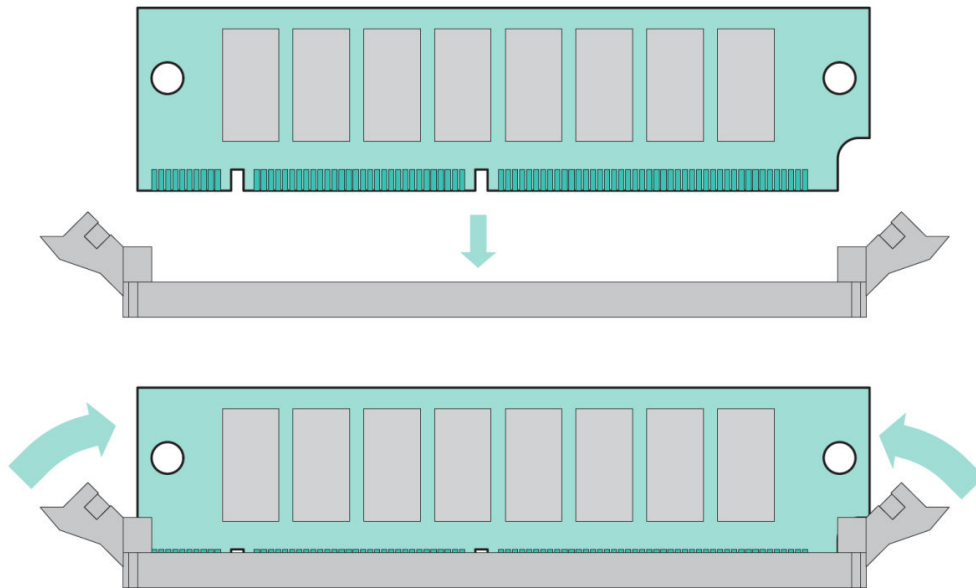


그림 6-19 DIMM의 시스템 보드 장착



SIMM



DIMM

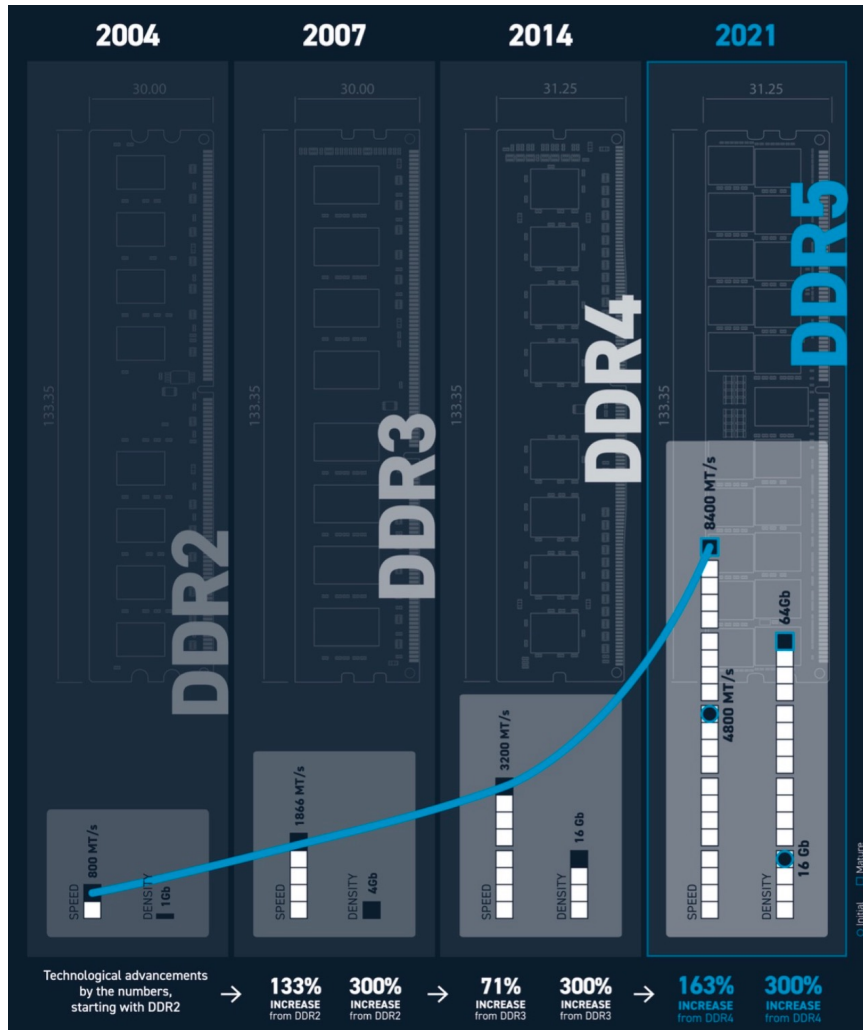
#데스크탑
메모리



SODIMM

#노트북
메모리

최신 메모리 현황: DDR4 vs. DDR5



삼성전자 노트북 DDR4 PC4-25600

8G

최저가29,570원

가격비교(419)

상품설명

상품평(6472)

요약설명

용도	노트북용(SO-DIMM)
용량	8G
램개수	1개
메모리규격	DDR4
동작속도	3,200MHz(PC4-25,600)



삼성전자 노트북 DDR5 PC5-38400

8G

최저가40,840원

가격비교(31)

상품설명

상품평(131)

요약설명

용도	노트북용(SO-DIMM)
용량	8G
램개수	1개
메모리규격	DDR5
동작속도	4,800MHz(PC5-38,400)

03 캐시 기억 장치

❖ 캐시의 사용 목적

- CPU와 주기억 장치의 속도 차이로 인한 CPU 대기 시간을 최소화 시키기 위하여 CPU와 주기억 장치 사이에 설치하는 고속 반도체 기억장치

❖ 캐시의 특징

- 주기억 장치보다 액세스 속도가 높은 칩 사용
- 가격 및 제한된 공간 때문에 용량이 적다.

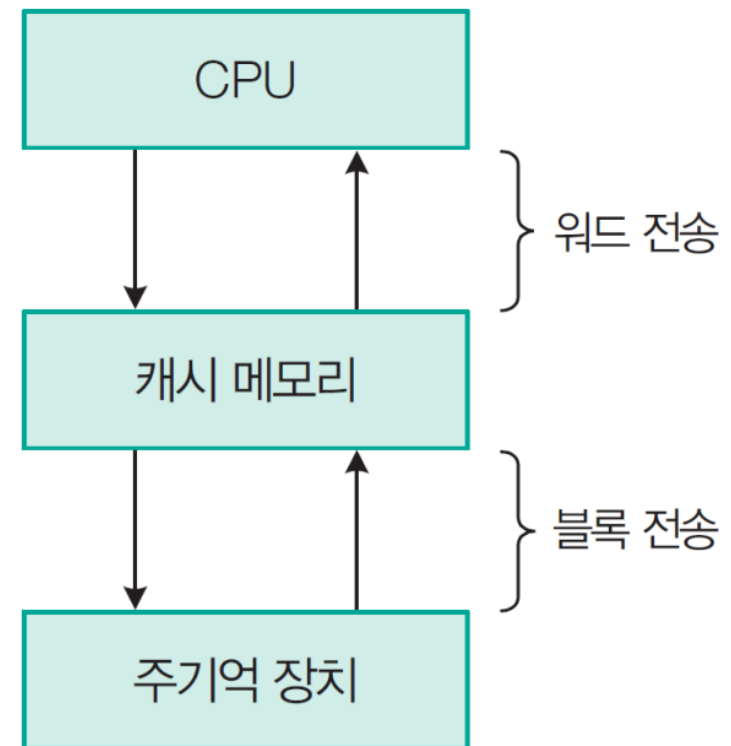
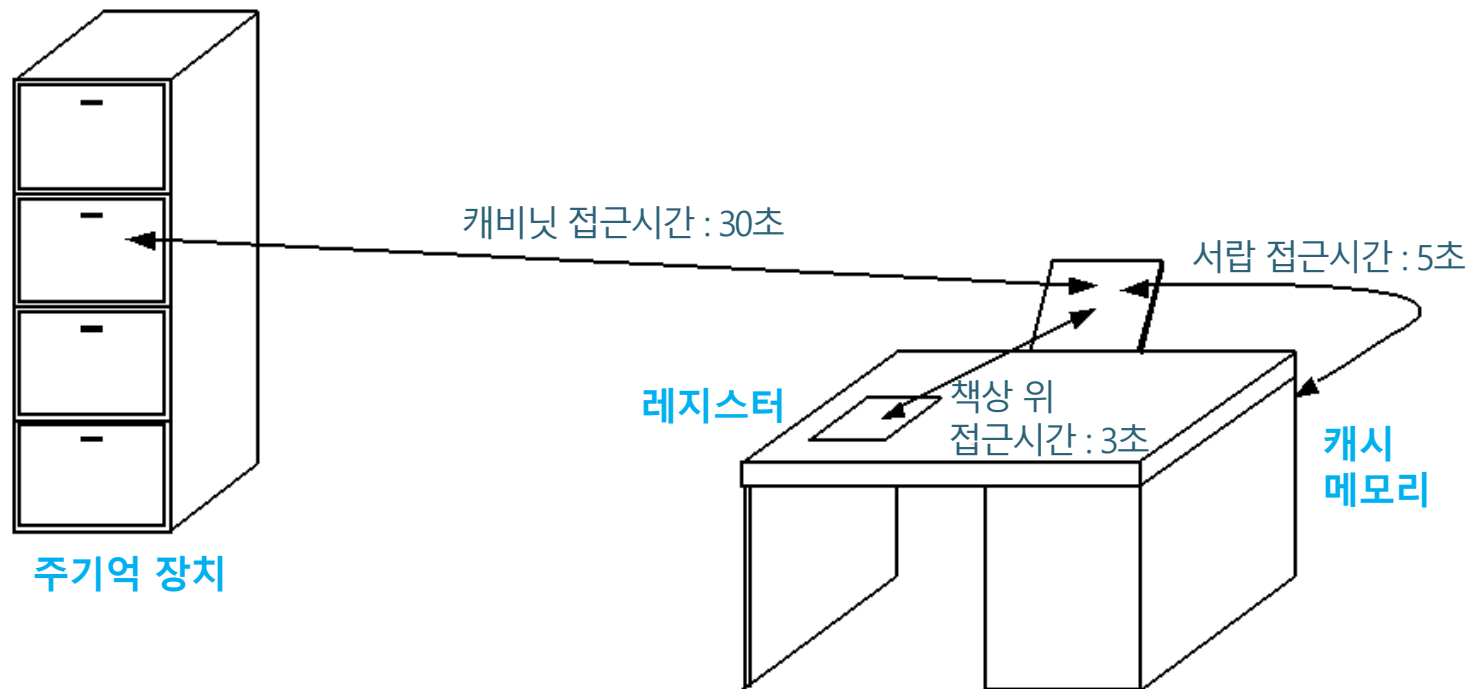


그림 6-20 캐시 위치

03 캐시 기억 장치

❖ 책상 위(레지스터), 서랍(캐시), 파일 캐비닛(주기억 장치)의 비교



03 캐시 기억 장치

❖ 캐시의 기본적인 동작 흐름도

- CPU가 기억 장치에서 어떤 정보(명령어 또는 데이터)를 읽으려는 경우 **먼저 해당 정보가 캐시에 있는지 검사**한다.
- 있다면 해당 정보를 즉시 읽어 들이고, **없다면 해당 정보를 주기억 장치에서 캐시로 적재**한 후 읽어 들인다.

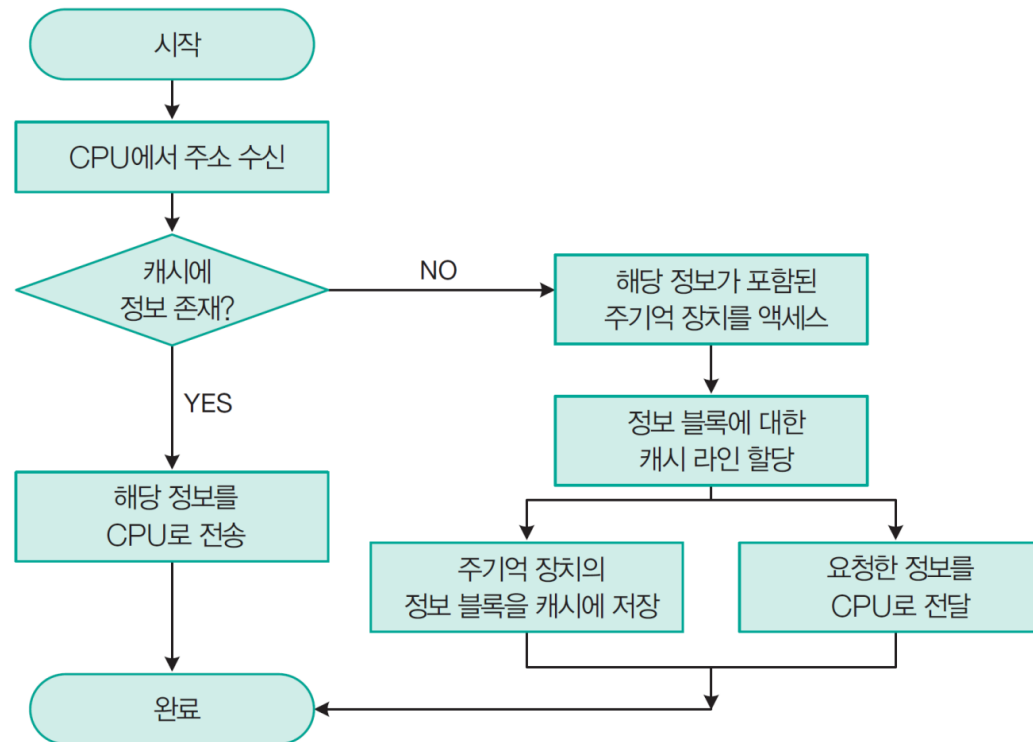


그림 6-21 캐시 읽기 동작 순서

03 캐시 기억 장치

❖ 캐시의 히트율 및 평균 액세스 시간

- 캐시 히트(cache hit): CPU가 원하는 데이터가 캐시에 있는 상태
- 캐시 미스(cache miss): CPU가 원하는 데이터가 캐시에 없는 상태. 이 경우에는 주기억 장치로부터 데이터를 읽어온다.
- 히트률(hit ratio): 캐시에 히트되는 정도(H)

$$H = \frac{\text{캐시에 히트되는 횟수}}{\text{전체 기억장치 액세스 횟수}}$$

- 캐시의 미스율(miss ratio) = $(1 - H)$
- 평균 기억장치 액세스 시간(T_a)

$$\begin{aligned} T_a &= H \times T_c + (1 - H) \times (T_c + T_m) \\ &\approx H \times T_c + (1 - H) \times T_m \end{aligned}$$

$$T_c + T_m \approx T_m$$

단, T_c 는 캐시 액세스 시간, T_m 은 주기억 장치 액세스 시간

03 캐시 기억 장치

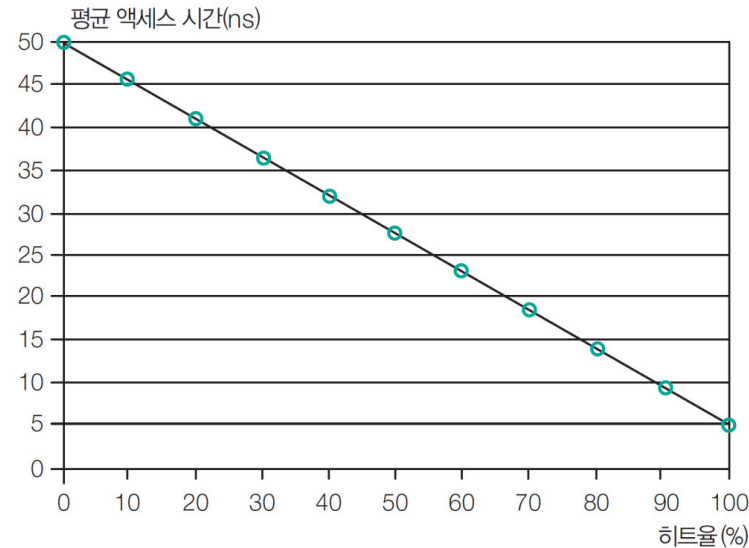
예제 6-5

캐시의 액세스 시간이 $T_c=5\text{ns}$ 이고, 주기억 장치의 액세스 시간이 $T_m=50\text{ns}$ 인 기억 장치 시스템에서 캐시 히트율이 0%부터 100%까지 10% 간격으로 변할 때 평균 액세스 시간 T_a 를 구하고 이를 그래프로 나타내어라. 결과도 설명하여라. 단, 그래프의 x 축은 히트율, y 축은 평균 액세스 시간으로 한다.

풀이

히트율(%)	평균 액세스 시간(ns)
0	50.0
10	45.5
20	41.0
30	36.5
40	32.0
50	27.5
60	23.0
70	18.5
80	14.0
90	9.5
100	5.0

캐시의 히트율이 높아질수록 평균 기억장치 액세스시간은 캐시 액세스 시간에 접근



$$T_a = H \times T_c + (1 - H) \times (T_c + T_m)$$

$$\approx H \times T_c + (1 - H) \times T_m$$

End of Example

03 캐시 기억 장치

❖ 참조 지역성(locality of reference)

- **공간적 지역성**(spatial locality) : 기억장치 내에 인접하여 저장되어 있는 데이터들이 연속적으로 액세스될 가능성이 높다. 예를 들어 표나 배열의 데이터들이 저장되어 있는 기억 장치 영역은 관련 연산이 수행되는 동안 자주 액세스된다.
- **시간적 지역성**(temporal locality) : 최근에 액세스된 프로그램이나 데이터가 가까운 미래에 다시 액세스될 가능성이 높다. 예를 들어 서브루틴이나 루프 프로그램들은 반복적으로 호출되며, 공통 변수들도 자주 액세스된다.

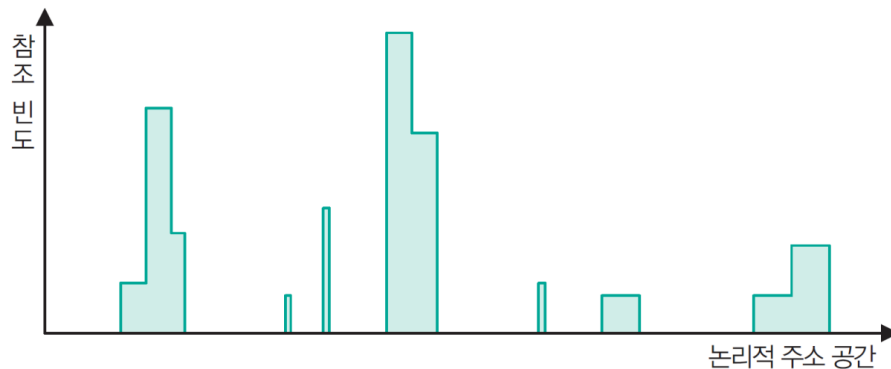


그림 6-22 주소 참조의 전형적인 비균등 분포

전형적인 많은 프로그램을 분석해 보면 그림처럼 주어진 시간 동안에 메모리 참조는 한정된 영역에서만 이루어지는 경향이 있음.
이러한 현상을

참조 지역성(locality of reference)
- 공간적 지역성(spatial locality)
- 시간적 지역성(temporal locality)

예) 도서관에서 캐시에 대한 내용을 찾기 위해 컴퓨터 구조 책을 찾아왔다면 그 책 주위에 이와 유사한 책이 많다(**공간적 지역성**).

또 그 책을 책상으로 가지고 왔다면 머지않아 그 책을 다시 찾아볼 가능성이 매우 크다(**시간적 지역성**)

Writing Cache-Friendly Code

- <http://web.cecs.pdx.edu/~jrb/cs201/lectures/cache.friendly.code.pdf>

Writing Cache Friendly Code

Repeated references to variables are good (temporal locality)

Stride-1 reference patterns are good (spatial locality)

Examples:

- cold cache, 4-byte words, 8-word cache blocks

```
int sumarrayrows(int a[M][N])
{
    int i, j, sum = 0;

    for (i = 0; i < M; i++)
        for (j = 0; j < N; j++)
            sum += a[i][j];
    return sum;
}
```

Miss rate = $1/8 = 12.5\%$

```
int sumarraycols(int a[M][N])
{
    int i, j, sum = 0;

    for (j = 0; j < N; j++)
        for (i = 0; i < M; i++)
            sum += a[i][j];
    return sum;
}
```

Miss rate = 100%

Concluding Observations

Programmer can optimize for cache performance

- How data structures are organized
- How data are accessed
 - Nested loop structure
 - Blocking is a general technique

All systems favor “cache friendly code”

- Getting absolute optimum performance is very platform specific
 - Cache sizes, line sizes, associativities, etc.
- Can get most of the advantage with generic code
 - Keep working set reasonably small (temporal locality)
 - Use small strides (spatial locality)

03 캐시 기억 장치

❖ 캐시 설계에 있어서의 공통적인 목표

- 캐시의 히트율을 극대화해야 한다.
- 캐시 히트인 경우 캐시에서 정보를 읽어 오는 시간을 최소화해야 한다.
- 캐시 미스인 경우 주기억 장치에서 캐시로 정보를 가져오는 데 걸리는 시간을 최소화해야 한다.
- 캐시의 내용이 변경되었을 경우 주기억 장치에 해당 내용을 갱신하는 데 소요되는 시간을 최소화해야 한다.

❖ 캐시 설계의 설계 요소

표 6-6 캐시의 설계 요소

캐시 용량		쓰기 정책	write-through
사상 방식	직접 사상		write-back
	연관 사상	라인 크기	
	세트-연관 사상	캐시 수	단일 레벨 또는 다중 레벨
교체 알고리즘	Least Recently Used _{LRU}		단일 캐시 또는 분리 캐시
	First In First Out _{FIFO}		
	Least Frequently Used _{LFU}		
	Random		

03 캐시 기억 장치

1 캐시 용량

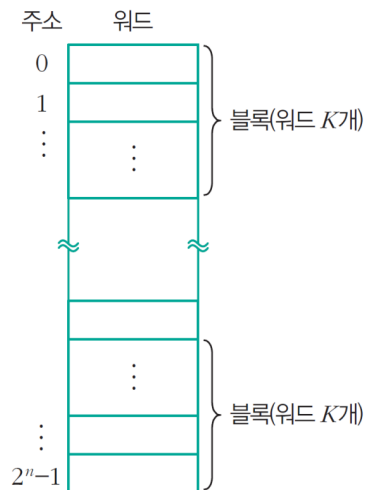
- 캐시 메모리 용량이 커질수록 히트율은 높아지지만, 비용이 증가
- 용량이 커질수록 주소 해독 및 정보 인출을 위한 주변 회로가 복잡해지므로 액세스 시간이 다소 더 길어진다.
- CPU 칩 또는 메인보드의 공간에도 제한을 받는다.

03 캐시 기억 장치

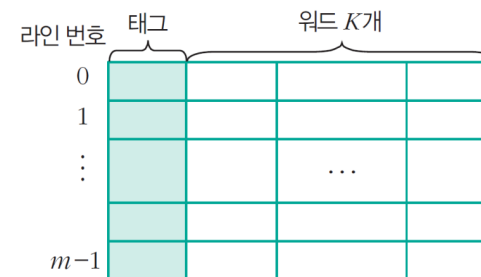
2 사상 방식

- **블록(block)**: 주기억 장치와 캐시 사이에 이동되는 정보 단위
- 주기억 장치 용량 = 2^n 워드, 블록 = K 개 워드 → 블록의 수 = $2^n/K$ 개
- **라인(line)**: 캐시에서 각 블록이 저장되는 장소. 라인 수 m 개, 각 라인에는 워드 K 개가 저장된다.
- **태그(tag)**: 라인에 적재된 블록을 구분해주는 정보

주기억 장치의 블록 중에서 일부만이
캐시에 적재될 수 있으므로
캐시의 각 라인은 여러 블록이 공유한다.



(a) 주기억 장치



(b) 캐시

그림 6-23 주기억 장치와 캐시의 구성

03 캐시 기억 장치

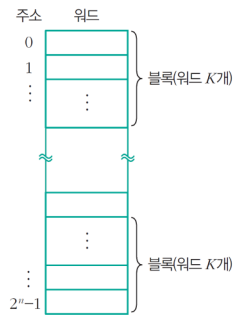
❖ 사상 방식(mapping scheme)

사상 방식이란 주기억 장치의 블록이 어느 캐시 라인에 들어갈 것인지 결정하는 방법

- 직접 사상(direct mapping)
- 완전-연관 사상(fully-associative mapping)
- 세트-연관 사상(set-associative mapping)

❖ 사상 방식을 설명하기 위한 모델 예

- 주기억 장치의 용량은 $128(=2^7)$ 바이트, 캐시 용량은 $32(=2^5)$ 바이트
- 주기억 장치 주소 = 7비트 (바이트 단위로 주소가 지정, 워드 길이는 1바이트)
- 블록 크기 = 4바이트($=2^2$) → 주기억 장치에는 $128/4=32$ 개($=2^5$)의 블록이 있다.
- 캐시 라인의 크기 = 4바이트(블록 크기와 동일)
- 전체 캐시 라인의 수, $m = 32/4 = 8$ 개 ($=2^3$)



(a) 주기억 장치

그림 6-23 주기억 장치와 캐시의 구성



(b) 캐시

주기억장치 주소의 각 필드 (비트)를 어떤 의미로 사용할 것인지 결정하는 것

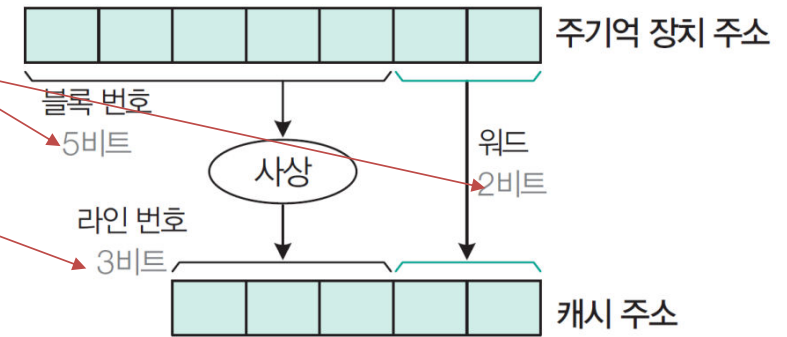
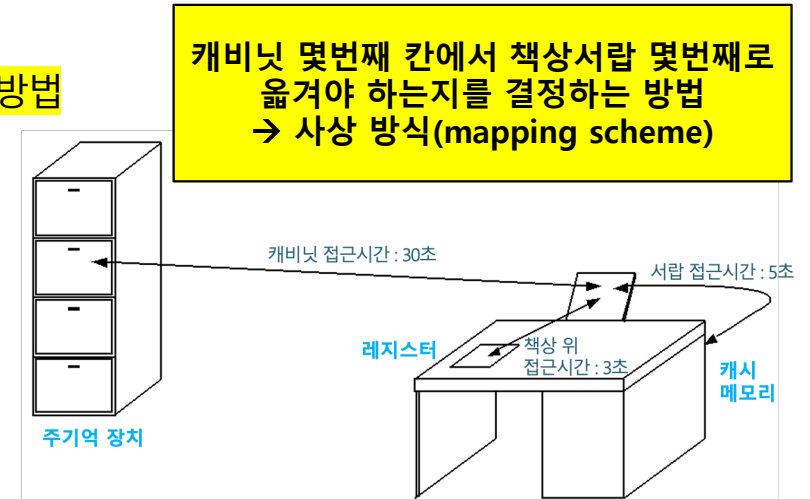


그림 6-24 주기억 장치와 캐시의 주소 사상

03 캐시 기억 장치

❑ 직접 사상 (direct mapping)

- 주기억 장치의 블록들이 **지정된 하나의 캐시 라인으로만 적재됨**
- 주기억 장치 주소는 필드 3개로 구성된다. 주기억 장치의 주소 지정에는 7비트가 필요하므로 $t+s+w=7$ 이다.
- 한 블록 내에는 워드 4개가 들어가므로 각 워드를 구별하기 위해 $w=2$ 비트가 필요하고,
- 캐시 라인의 수가 8개이므로 각각을 구별하기 위해 $s=3$ 비트,
- 그리고 태그 필드는 나머지 $t=2(=7-3-2)$ 비트가 된다.
- **태그 필드 값은 캐시 라인을 공유하는 주기억 장치 블록들을 서로 구분하는 데 사용된다.**

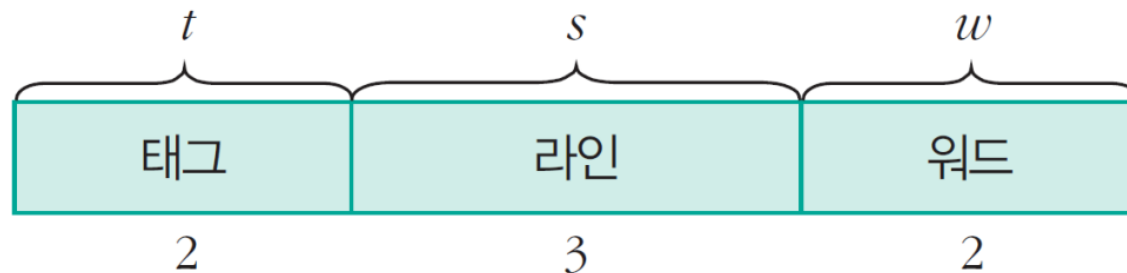


그림 6-25 직접 사상에서 주소 필드의 구성

직접사상:
7비트 주소를
상위 2비트는 태그로 해석,
3비트는 라인으로,
하위 2비트는 워드 구분용으로
사용

03 캐시 기억 장치

- 주기억 장치의 블록 $j(=0, 1, \dots, 31)$ 가 적재될 수 있는 캐시 라인 번호는 다음 연산으로 결정된다.

$$i = j \bmod m \quad \text{나머지 연산자}$$

- 예) 캐시라인수 $m=8$, 주기억 장치 10번째 블록 $j=10$ 인 경우, $10 \bmod 8 = 2$ 이므로 2번째 캐시 라인에 들어감
- [표 6-7]에는 각 캐시 라인을 공유하는 블록들의 번호 $4(=2^2=2^2)$ 개가 나열되어 있다. 여기서 블록 번호는 주기억 장치 주소의 상위 5비트에 해당한다.

표 6-7 직접 사상에서 각 캐시 라인을 공유하는 주기억 장치 블록들

캐시 라인	주기억 장치 블록 번호			
0(000)	00000	01000	10000	11000
1(001)	00001	01001	10001	11001
2(010)	00010	01010	10010	11010
3(011)	00011	01011	10011	11011
4(100)	00100	01100	10100	11100
5(101)	00101	01101	10101	11101
6(110)	00110	01110	10110	11110
7(111)	00111	01111	10111	11111

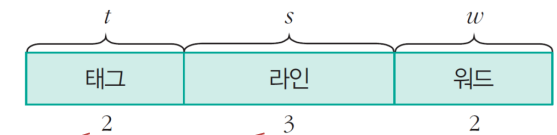


그림 6-25 직접 사상에서 주소 필드의 구성

직접사상:
7비트 주소를
상위 2비트는 태그로 해석,
3비트는 라인으로,
하위 2비트는 워드 구분용으로
사용

03 캐시 기억 장치

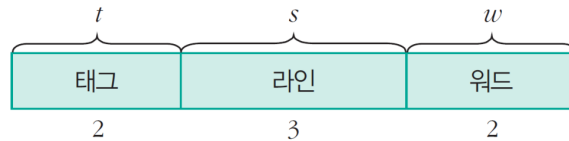


그림 6-25 직접 사상에서 주소 필드의 구성

표 6-7 직접 사상에서 각 캐시 라인을 공유하는 주기억 장치 블록들

캐시 라인	주기억 장치 블록 번호			
0(000)	00000	01000	10000	11000
1(001)	00001	01001	10001	11001
2(010)	00010	01010	10010	11010
3(011)	00011	01011	10011	11011
4(100)	00100	01100	10100	11100
5(101)	00101	01101	10101	11101
6(110)	00110	01110	10110	11110
7(111)	00111	01111	10111	11111

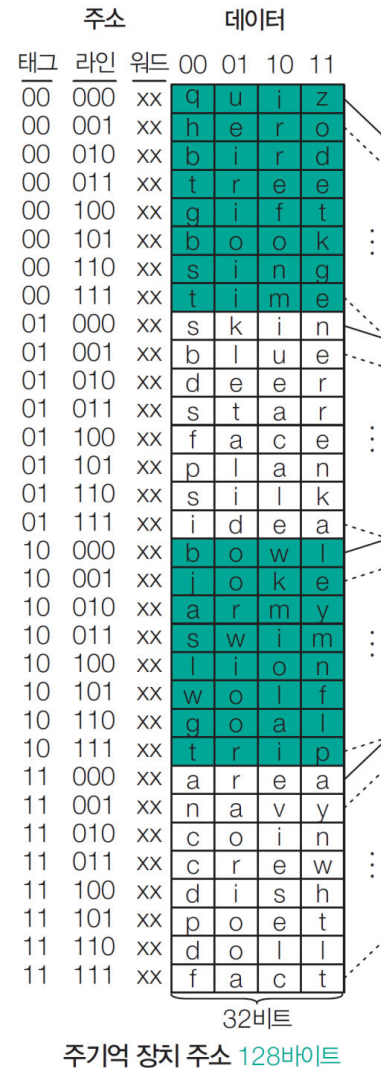
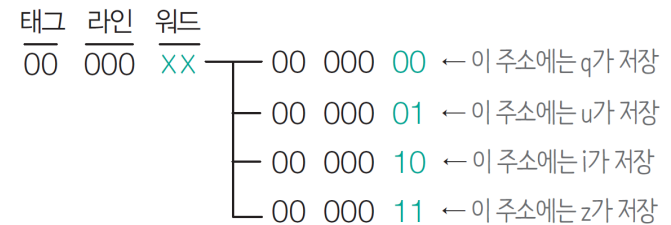


그림 6-26 직접 사상의 예

주기억 장치에 저장되는 데이터는 이해하기 쉽게 워드 4개(ASCII 코드)로 구성되는 것으로 가정했다.



03 캐시 기억 장치

주소			데이터			
태그	라인	워드	00	01	10	11
00	000	xx	q	u	i	z
00	001	xx	h	e	r	o
00	010	xx	b	i	r	d
00	011	xx	t	r	e	e
00	100	xx	g	i	f	t
00	101	xx	b	o	o	k
00	110	xx	s	i	n	g
00	111	xx	t	i	m	e
01	000	xx	s	k	i	n
01	001	xx	b	l	u	e
01	010	xx	d	e	e	r
01	011	xx	s	t	a	r
01	100	xx	f	a	c	e
01	101	xx	p	l	a	n
01	110	xx	s	i	l	k
01	111	xx	i	d	e	a
10	000	xx	b	o	w	l
10	001	xx	j	o	k	e
10	010	xx	a	r	m	y
10	011	xx	s	w	i	m
10	100	xx	l	i	o	n
10	101	xx	w	o	l	f
10	110	xx	g	o	a	l
10	111	xx	t	r	i	p
11	000	xx	a	r	e	a
11	001	xx	n	a	v	y
11	010	xx	c	o	i	n
11	011	xx	c	r	e	w
11	100	xx	d	i	s	h
11	101	xx	p	o	e	t
11	110	xx	d	o	i	l
11	111	xx	f	a	c	t

32비트

주기억 장치 주소 128바이트

❖ 직접 사상에서 캐시 내부 구성 및 읽기 동작

- ① 캐시로 주기억 장치 주소 11 101 01이 보내진다(태그 필드=11, 라인 필드=101, 워드 필드=01).
- ② 라인 필드가 101이므로 5번 캐시 라인이 선택된다.
- ③ 선택된 5번 라인의 태그 비트 11을 읽어서, ④ 주기억 장치 주소의 태그 필드인 11과 비교한다.
- ⑤ 두 태그 비트가 일치하므로 캐시가 히트된 것이다.
- ⑥ 다음에는 주소의 워드 필드가 01이므로 poet 중에서 o(1110 1111)가 인출되어 CPU로 전송된다.
- ⑦ 그러나 태그 비트가 일치하지 않으면 캐시가 미스된 것이므로 주소 전체가 주기억 장치로 보내져서 해당 블록을 인출해 온다. 인출된 블록은 지정된 캐시 라인에 저장된다. 그런데 그 라인을 공유하는 다른 블록이 이미 저장되어 있는 상태라면 원래 블록은 지워지고 새로 인출된 블록이 저장된다.

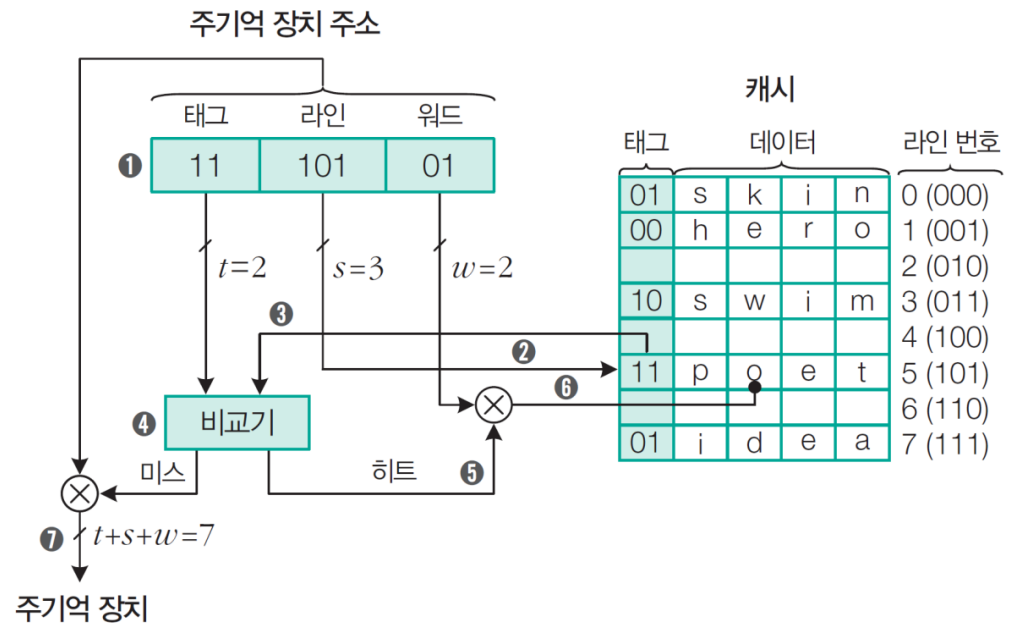


그림 6-27 직접 사상에서 캐시 내부 구성 및 동작

그림 6-26 직접 사상의 예

03 캐시 기억 장치

주소			데이터			
태그	라인	워드	00	01	10	11
00	000	xx	a	u	r	e
00	001	xx	h	e	r	o
00	010	xx	b	i	r	d
00	011	xx	t	r	e	e
00	100	xx	g	i	f	t
00	101	xx	b	o	o	k
00	110	xx	s	i	n	g
00	111	xx	t	i	m	e
01	000	xx	s	k	i	n
01	001	xx	b	l	u	e
01	010	xx	d	e	e	r
01	011	xx	s	t	a	r
01	100	xx	f	a	c	e
01	101	xx	p	l	a	n
01	110	xx	s	i	l	i
01	111	xx	i	d	e	a
10	000	xx	b	o	w	i
10	001	xx	j	o	k	e
10	010	xx	a	r	m	y
10	011	xx	s	w	i	m
10	100	xx	f	a	c	t
10	101	xx	w	o	l	f
10	110	xx	g	o	a	l
10	111	xx	t	r	i	p
11	000	xx	a	r	e	a
11	001	xx	n	a	v	y
11	010	xx	c	o	i	n
11	011	xx	c	r	e	w
11	100	xx	d	i	s	h
11	101	xx	p	o	e	t
11	110	xx	d	o	i	l
11	111	xx	f	a	c	t

태그	데이터	라인 번호
01	s k i n	0 (000)
00	h e r o	1 (001)
10	s w i m	2 (010)
11	p o e t	5 (101)
01	i d e a	7 (111)

2비트 32비트
캐시 32바이트

32비트
주기억 장치 주소 128바이트

예제 6-6

직접 사상 캐시의 라인들이 [그림 6-26]과 같이 저장되어 있다고 가정하자. 이때 CPU에서 발생한 주소들이 다음과 같은 경우 히트인지 또는 미스인지 구별하여라. 미스인 경우 주기억 장치에서 데이터를 인출하여 해당 라인에 적재된 후의 결과를 설명하라.

(a) 00 001 01 (b) 01 010 11 (c) 10 011 10 (d) 11 111 00

풀이

- (a) **히트**: 이 블록은 1번 라인에 적재될 수 있는데, 현재 태그가 00이므로 일치한다. 인출되는 데이터는 hero 중에서 e다.
- (b) **미스**: 이 블록이 적재될 수 있는 2번 라인이 비어 2번 라인의 데이터 필드에는 deer가 적재되고, 태그는 01로 세트된다. 최종적으로 인출되는 데이터는 deer 중에서 r이다.
- (c) **히트**: 이 블록은 3번 라인에 적재될 수 있는데, 현재의 태그가 10이므로 일치한다. 인출되는 데이터는 swim 중에서 i다.
- (d) **미스**: 주소의 태그 비트는 11이고, 이 블록이 적재될 수 있는 7번 라인의 현재 태그는 01이므로 일치하지 않는다. 따라서 7번 라인의 데이터 필드는 fact로 대체되고, 태그는 11로 변경된다. 최종적으로 인출되는 데이터는 fact 중에서 f다.

End of Example

그림 6-26 직접 사상의 예

03 캐시 기억 장치

❖ 직접 사상 캐시의 장단점

장점	하드웨어가 간단하고, 구현 비용이 적게 든다.
단점	각 주기억 장치 블록이 적재될 수 있는 캐시 라인이 한 개뿐이기 때문에, 그 라인을 공유하는 다른 블록이 적재되는 경우에는 반복적으로 미스되어 히트율이 떨어짐(swap-out)

03 캐시 기억 장치

□ 완전-연관 사상(fully-associative mapping)

- 주기억 장치 블록이 캐시의 어떤 라인으로든 적재 가능
- 태그 필드 = 주기억 장치 블록 번호

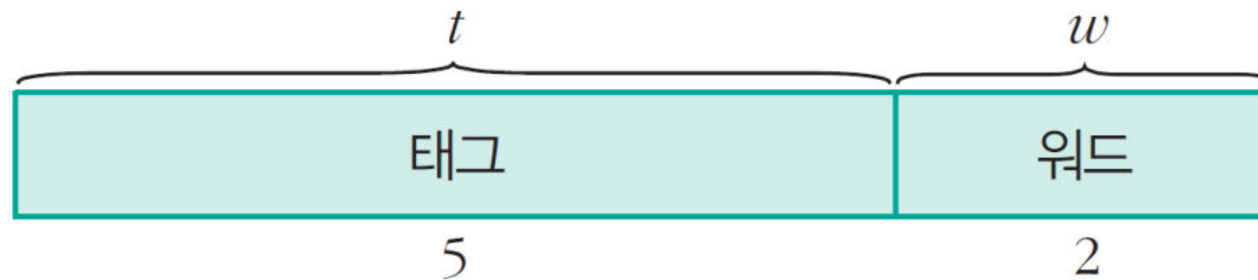
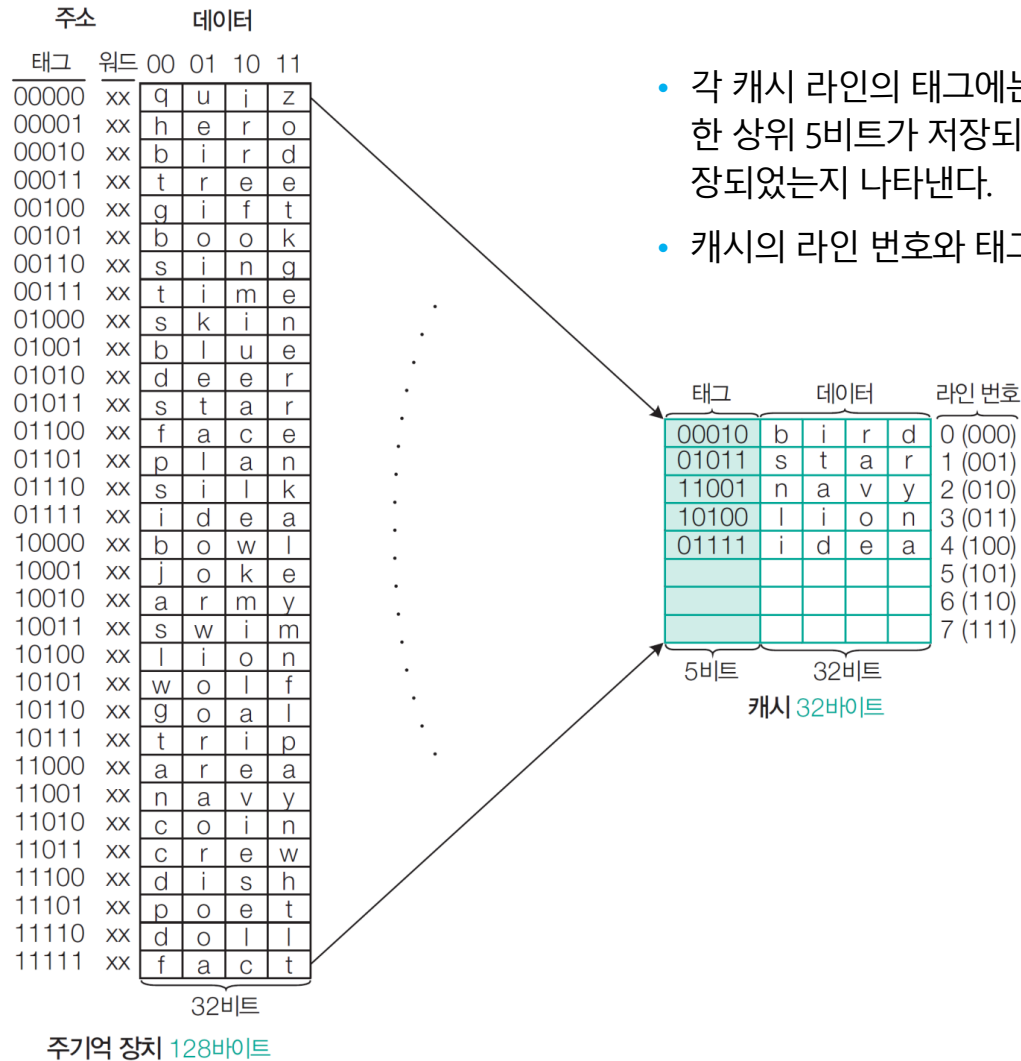


그림 6-28 완전-연관 사상에서 주소 필드의 구성

03 캐시 기억 장치



- 각 캐시 라인의 태그에는 주소의 워드 필드(2비트)를 제외한 상위 5비트가 저장되어 주기억 장치의 어느 블록이 저장되었는지 나타낸다.
- 캐시의 라인 번호와 태그 필드는 관련이 없다.

캐시 태그가 주기억장치 주소
위에서 부터 순차적으로 적재

그림 6-29 완전-연관 사상의 예

03 캐시 기억 장치

주소		데이터			
태그	워드	00	01	10	11
00000	xx	q	u	i	z
00001	xx	h	e	r	o
00010	xx	b	i	r	d
00011	xx	t	r	e	e
00100	xx	g	i	f	t
00101	xx	b	o	o	k
00110	xx	s	i	n	g
00111	xx	t	i	m	e
01000	xx	s	k	i	n
01001	xx	b	l	u	e
01010	xx	d	e	e	r
01011	xx	s	t	a	r
01100	xx	f	a	c	e
01101	xx	p	l	a	n
01110	xx	s	i	l	k
01111	xx	i	d	e	a
10000	xx	b	o	w	l
10001	xx	j	o	k	e
10010	xx	a	r	m	y
10011	xx	s	w	i	m
10100	xx	l	i	o	n
10101	xx	w	o	l	f
10110	xx	g	o	a	l
10111	xx	t	r	i	p
11000	xx	a	r	e	a
11001	xx	n	a	v	y
11010	xx	c	o	i	n
11011	xx	c	r	e	w
11100	xx	d	i	s	h
11101	xx	p	o	e	t
11110	xx	d	o	l	l
11111	xx	f	a	c	t

32비트

주 기억 장치 128바이트

그림 6-29 완전-연관 사상의 예

❖ 완전-연관 사상에서 캐시 내부 구성 및 읽기 동작

- ① 캐시로 주기억 장치 주소 11001 11이 보내진다. 태그 필드는 11001이고, 워드 필드는 11이다.
- ② 캐시의 모든 라인의 태그들과 주기억 장치 주소의 태그 필드 내용을 비교한다.
- ③ 2번 캐시 라인의 11001과 주소의 태그 필드 11001이 일치하므로 **캐시가 히트된 것이다**.
- ④ 다음에는 주소의 워드 필드가 11이므로 navy 중에서 y(0111 1001)가 인출되어 CPU로 전송된다.
- ⑤ 만약, 태그 비트가 일치하지 않는 경우, 캐시가 미스된 것이므로 주소 전체가 주기억 장치로 보내져서 해당 블록을 인출해 온다. 인출된 블록은 5번 캐시 라인에 저장된다.

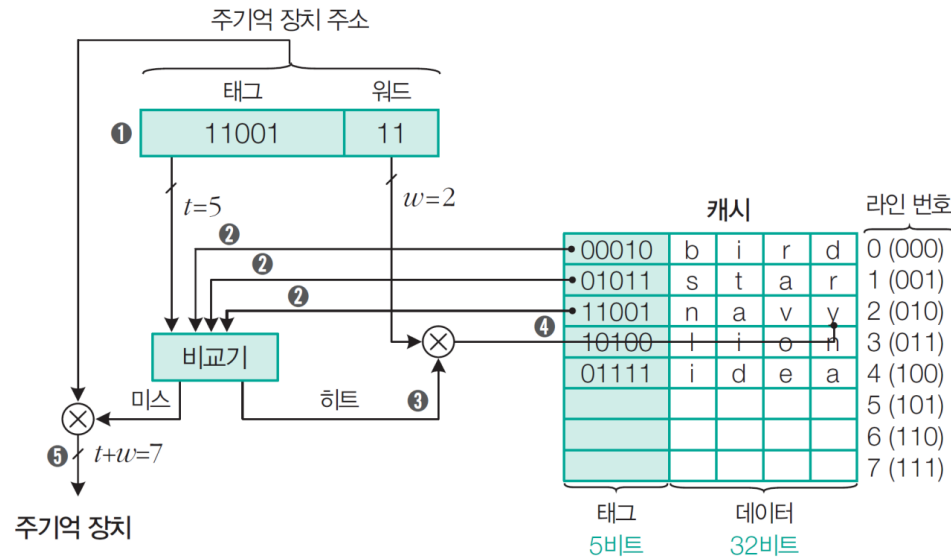


그림 6-30 완전-연관 사상에서 캐시 내부 구성 및 동작

03 캐시 기억 장치

주소		데이터			
태그	워드	00	01	10	11
00000	xx	q	u	i	z
00001	xx	h	e	r	e
00010	xx	b	i	r	d
00011	xx	t	r	e	e
00100	xx	g	i	f	t
00101	xx	b	o	o	k
00110	xx	s	i	n	g
00111	xx	t	i	m	e
01000	xx	s	k	i	n
01001	xx	b	l	u	e
01010	xx	d	e	e	r
01011	xx	s	t	a	r
01100	xx	f	a	c	e
01101	xx	p	l	a	n
01110	xx	s	i	l	k
01111	xx	i	d	e	a
10000	xx	b	o	w	l
10001	xx	j	e	k	e
10010	xx	a	r	m	y
10011	xx	s	w	i	n
10100	xx	l	i	o	n
10101	xx	w	o	l	f
10110	xx	g	o	a	l
10111	xx	t	r	i	p
11000	xx	a	r	e	a
11001	xx	n	a	v	y
11010	xx	c	o	i	n
11011	xx	c	i	e	w
11100	xx	d	i	s	h
11101	xx	p	o	e	t
11110	xx	d	o	l	l
11111	xx	f	a	c	t

32비트

주 기억 장치 128바이트

예제 6-7

완전-연관 사상 캐시의 라인들이 [그림 6-29]와 같이 저장되어 있다고 가정하자. 이때 CPU에서 발생한 주소가 다음과 같은 경우 히트인지 미스인지 구별하여라. 미스인 경우 주기억 장치에서 데이터를 인출하여 해당 라인에 적재된 후의 결과에 대해 설명하라.

- (a) 01011 01 (b) 10010 11
(c) 00010 10 (d) 11100 00

풀이

태그	데이터	라인 번호
00010	b i r d	0 (000)
01011	s t a r	1 (001)
11001	n a v y	2 (010)
10100	l i o n	3 (011)
01111	i d e a	4 (100)
		5 (101)
		6 (110)
		7 (111)

(a) **히트**: 1번 라인에 적재되어 있으며, 인출 데이터는 star에서 t다.

(b) **미스**: 비어 있는 첫 번째 라인인 5번 라인에 적재되므로 데이터 필드에는 army가 적재되고, 태그는 10010으로 변경된다. 최종 인출 데이터는 army에서 y다.

(c) **히트**: 0번 라인에 적재되며, 인출 데이터는 bird에서 r이다.

(d) **미스**: 라인 순으로 6번 라인에 적재되므로 데이터 필드에는 dish가 적재되고, 태그는 11100으로 변경된다. 최종 인출 데이터는 dish에서 d다.

End of Example

그림 6-29 완전-연관 사상의 예

03 캐시 기억 장치

□ 세트-연관 사상(set-associative mapping)

- 직접 사상의 장점(단순성)과 완전-연관 사상의 장점(높은 캐시 히트율) 조합
- 주 기억 장치 블록 그룹이 하나의 캐시 세트를 공유하며, 그 세트는 두 개 이상의 라인들에 적재될 수 있음
- 전체 캐시 라인(m)은 v 개의 세트들로 나누어지며, 각 세트들은 k 개의 라인들로 구성(k -way)
- 주 기억 장치 블록(j)이 적재될 수 있는 캐시 세트의 번호 i :

$$m = v \times k$$
$$i = j \bmod v$$

- 앞의 예에서 캐시 라인의 수는 $m=8$ 이다. 세트당 캐시 라인의 수가 $k=2$ 라면 세트 수는 $v=8/2=4$ 개다. 따라서 캐시는 세트 4개로 구성되고, 각 세트에 캐시 라인이 2개 있다.

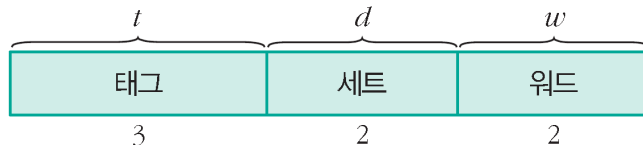


그림 6-31 세트-연관 사상에서 주소 필드의 구성

캐시 라인의 수는 $m=8$, 세트당 캐시 라인의 수가 $k=2$ 라면 세트 수는 $v=8/2=4$ 개다.

그러므로 주 기억 장치의 10번 블록은 $10 \bmod 4 = 2$ 번 세트에 들어간다.

그리고 10번 블록이 들어갈 2번 세트에는 캐시 라인이 2개 있으므로 하나가 채워져 있어도 나머지 캐시 라인에 들어갈 수 있다.

이와 같이 각 세트에 캐시 라인이 2개이면 2-way 세트-연관 사상이라고 하며, 세트당 캐시 라인이 4개이면 4-way 세트-연관 사상이라고 한다.

03 캐시 기억 장치

- 주기억 장치의 블록 $j(=0, 1, \dots, 31)$ 가 적재될 수 있는 세트 번호는 $j \bmod 4$ 로 결정된다.
- 세트-연관 사상 방식에서 세트 4개, 세트당 캐시 라인 2개에 들어갈 수 있는 주기억 장치 블록 32개는 다음과 같다.

표 6-8 세트-연관 사상에서 각 세트를 공유하는 주기억 장치 블록

세트 번호	주기억 장치 블록 번호							
0(00)	00000	00100	01000	01100	10000	10100	11000	11100
1(01)	00001	00101	01001	01101	10001	10101	11001	11101
2(10)	00010	00110	01010	01110	10010	10110	11010	11110
3(11)	00011	00111	01011	01111	10011	10111	11011	11111

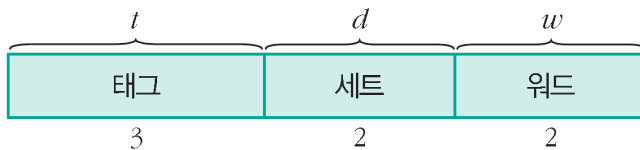


그림 6-31 세트-연관 사상에서 주소 필드의 구성

캐시에는 캐시 라인이 $m=8$ 개 있고, 이들은 $v=4(=2^d)$ 개의 세트로 나누어지므로 세트당 캐시 라인의 수는 $k=2(=8/4)$ 개로 2-way가 된다.

세트 필드의 $d=2$ 개 비트는 캐시 세트 4개 중 하나를 선택하는데 사용된다. 태그 필드의 $t=3$ 개 비트는 세트에 있는 캐시 라인들의 태그들을 비교하여 히트 여부를 확인하는 데 사용된다.

따라서 세트-연관 사상은 직접 사상으로 세트를 찾고, 완전-연관 사상으로 해당 세트 내 캐시 라인을 찾는 방식이라고 할 수 있다.

03 캐시 기억 장치

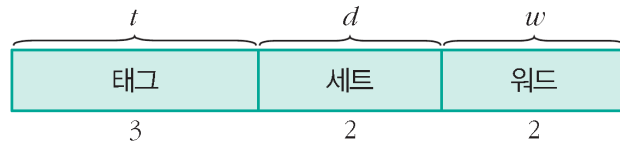


그림 6-31 세트-연관 사상에서 주소 필드의 구성

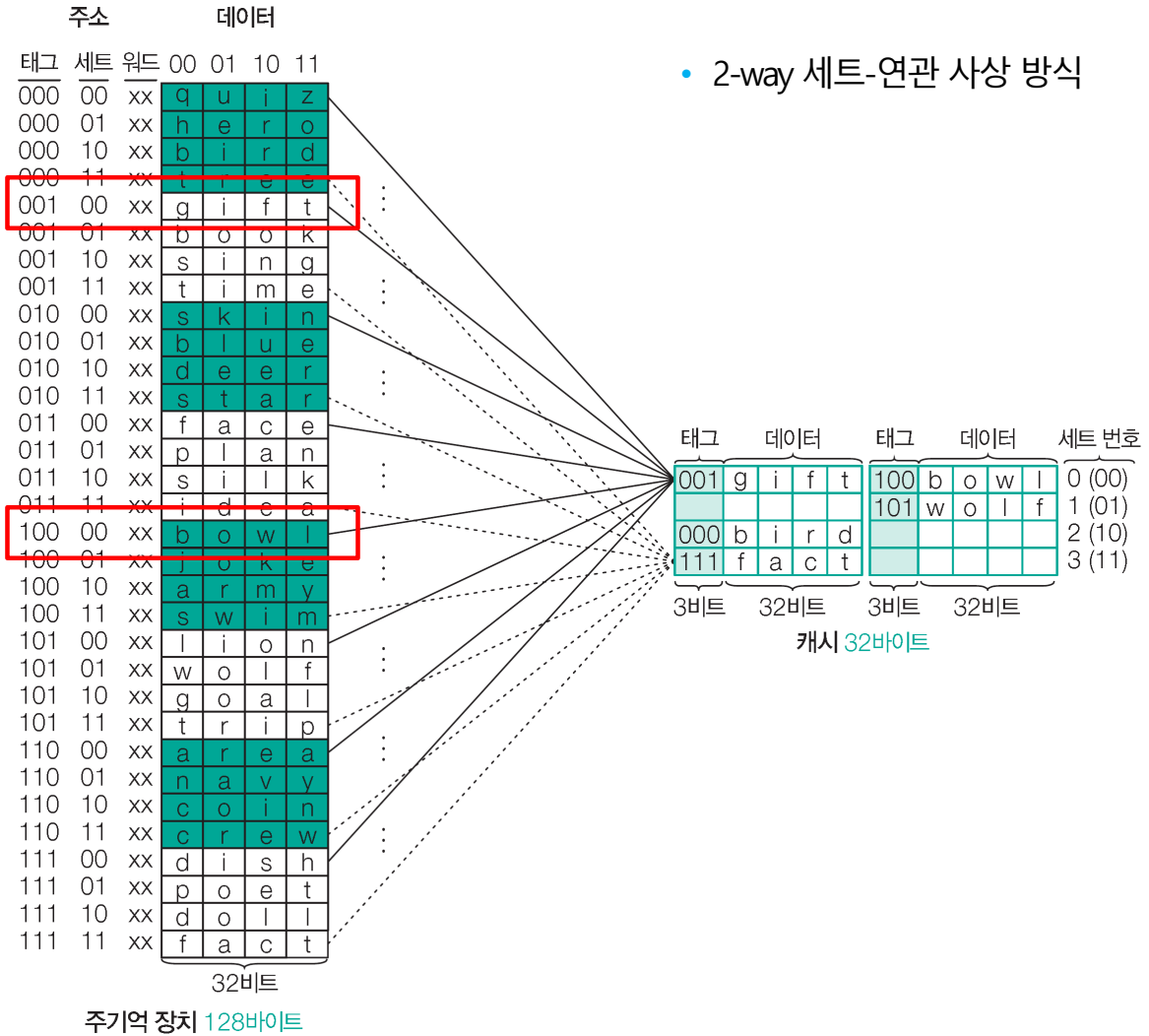


그림 6-32 세트-연관 사상의 예

03 캐시 기억 장치

주소	데이터
태그 세트 워드	00 01 10 11
000 00 xx	q u i z
000 01 xx	h e r o
000 10 xx	b i r d
000 11 xx	f r e e
001 00 xx	g i f t
001 01 xx	b o o k
001 10 xx	s i n g
001 11 xx	t i m e
010 00 xx	s k i n
010 01 xx	b l u e
010 10 xx	d e e r
010 11 xx	s t a r
011 00 xx	f a c e
011 01 xx	p l a n
011 10 xx	s i l k
011 11 xx	i d e a
100 00 xx	b o w l
100 01 xx	j o k e
100 10 xx	a r m y
100 11 xx	s w i m
101 00 xx	l i o n
101 01 xx	w o l f
101 10 xx	g o a l
101 11 xx	t r i p
110 00 xx	a r e a
110 01 xx	n a v y
110 10 xx	c o i n
110 11 xx	c r e w
111 00 xx	d i s h
111 01 xx	p o e t
111 10 xx	d o l l
111 11 xx	f a c t

32비트

주 기억 장치 128바이트

그림 6-32 세트-연관 사상의 예

❖ 세트-연관 사상에서 캐시 내부 구성 및 읽기 동작

- ① 주기억 장치 주소 **001 00 10**이 캐시로 보내진다(태그 필드=001, 세트 필드=00, 워드 필드= 10).
- ② 00 세트 번호를 이용해 캐시의 0번 세트를 선택한다.
- ③ 0 번 세트 라인의 태그에 주기억 장치 주소의 태그 비트 001과 일치하는 것이 있는지 검사한다.
- ④ 0번 세트 내 첫 번째 라인의 태그 비트가 **001이므로 히트되었다**.
- ⑤ 다음에는 주소의 워드 필드가 10이므로 gift 중에서 f(1110 0110)가 인출되어 CPU로 전송된다.
- ⑥ 만약, 태그 비트가 일치하지 않으면 캐시가 미스된 것이므로 주소 전체가 주기억 장치로 보내져서 해당 블록을 인출해 온다. 적절한 교체 알고리즘에 의하여 2개 중 하나를 선택하여 그 라인에 새로운 블록을 적재해야 한다.

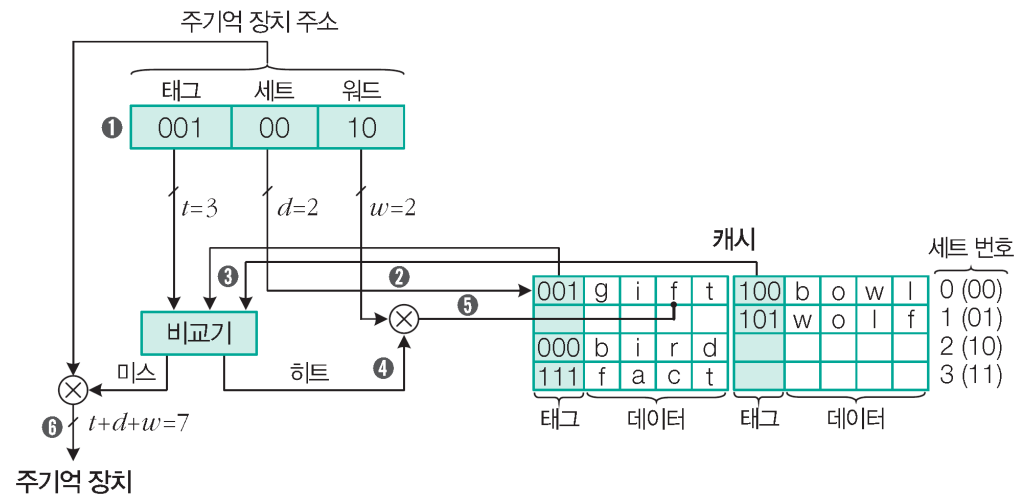


그림 6-33 세트-연관 사상에서 캐시 내부 구성 및 동작

03 캐시 기억 장치

주소	데이터
태그 세트 워드	00 01 10 11
000 00 xx	q u i z
000 01 xx	h e r o
000 10 xx	b i r d
000 11 xx	t r e e
001 00 xx	g i f t
001 01 xx	b o o k
001 10 xx	s i n g
001 11 xx	t i m e
010 00 xx	s k i n
010 01 xx	b i r d
010 10 xx	d e e r
010 11 xx	s t a r
011 00 xx	f a c e
011 01 xx	p l a n
011 10 xx	s i l k
011 11 xx	i d e a
100 00 xx	b o w l
100 01 xx	j o k e
100 10 xx	a r m y
100 11 xx	s w i m
101 00 xx	l i o n
101 01 xx	w o l f
101 10 xx	g o a l
101 11 xx	t r i p
110 00 xx	a r e a
110 01 xx	n a v y
110 10 xx	c o i n
110 11 xx	c r e w
111 00 xx	d i s h
111 01 xx	p o e t
111 10 xx	d o i l
111 11 xx	f a c t

32비트

주기억 장치 128바이트

태그	데이터	태그	데이터	세트 번호
001	g i f t	100	b o w l	0 (00)
000	b i r d	101	w o l f	1 (01)
000	b i r d			2 (10)
111	f a c t			3 (11)

3비트

32비트

캐시 32바이트

그림 6-32 세트-연관 사상의 예

예제 6-8

세트-연관 사상 캐시의 라인들이 [그림 6-32]와 같이 저장되어 있다고 가정하자. 이때 CPU에서 발생한 주소들이 다음과 같은 경우 히트인지 또는 미스인지 구별하여라. 그리고 미스인 경우 주기억 장치에서 데이터를 인출하여 해당 라인에 적재된 후 결과를 설명하여라.

- (a) 100 00 01 (b) 011 10 11
(c) 111 11 10 (d) 010 00 00

풀이

- (a) **히트** : 0번 세트의 두 번째 라인에 적재되어 있으며, 인출 데이터는 bowl에서 o이다.
(b) **미스** : 적재될 수 있는 2번 세트의 첫 번째 라인의 태그 000과 주소의 태그 011이 일치하지 않는다. 따라서 빈 두 번째 라인에 silk가 적재되고, 태그는 011로 변경된다. 최종적으로 인출 데이터는 silk에서 k다.
(c) **히트** : 3번 세트의 첫 번째 라인에 적재되어 있으며, 인출 데이터는 fact에서 c다.
(d) **미스** : 이 블록이 적재될 수 있는 0번 세트의 두 라인의 태그(001, 100)가 주소의 태그 010과 일치하지 않는다. 편의상 새로운 블록이 두 번째 라인에 적재된다고 가정하면 skin이 적재되고, 태그는 010으로 변경된다. 최종 인출 데이터는 skin에서 s다.

End of Example

03 캐시 기억 장치

❖ 사상 방식의 비교

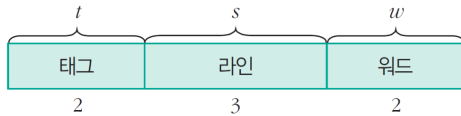


그림 6-25 직접 사상에서 주소 필드의 구성

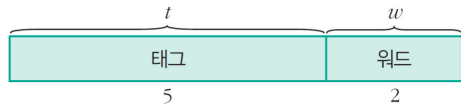


그림 6-28 완전-연관 사상에서 주소 필드의 구성

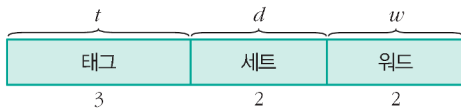


그림 6-31 세트-연관 사상에서 주소 필드의 구성

표 6-9 사상 방식의 비교

사상 기법	단순성	태그 연관 검색	캐시 효율	교체 기법
직접	단순	없음	낮음	불필요
완전-연관	복잡	연관	높음	필요
세트-연관	중간	중간	중간	필요

03 캐시 기억 장치

3 교체 알고리즘

- 세트-연관 사상에서 주기억 장치로부터 새로운 블록이 캐시로 적재될 때, 만약 세트 내 모든 라인들이 다른 블록들로 채워져 있다면, **그들 중의 하나를 선택하여 새로운 블록으로 교체**
- 교체 알고리즘 : **캐시 히트율을 극대화할 수 있도록 교체할 블록을 선택하기 위한 알고리즘**
 - **LRU**(Least Recently Used) : 사용되지 않은 채로 가장 오래 있었던 블록을 교체하는 방식
 - **FIFO**(First-In-First-Out) 알고리즘 : 캐시에 적재된 지 가장 오래된 블록을 교체하는 방식
 - **LFU**(Least Frequently Used) 알고리즘 : 참조되었던 횟수가 가장 적은 블록을 교체하는 방식
 - **Random** : 사용 횟수를 고려하지 않고 후보 캐시 라인 중 임의로 선택하여 교체하는 방식

LRU(Least Recently Used)

각 캐시 라인에 USE 비트를 포함
액세스된 캐시 라인의 USE 비트를 1로 설정하고 다른 캐시 라인은 0으로 설정
새로운 블록으로 교체 시 USE 비트가 0인 캐시라인을 교체함

FIFO(First-In-First-Out)

캐시에 적재된 지 가장 오래된 블록을 교체한다.

LFU(Least Frequently Used)

각 캐시 라인에 카운터를 설치하여 캐시에 적재된 후 액세스 횟수가 가장 적은 블록을 교체한다.

Random

랜덤하게 캐시 라인 중 임의로 선택하여 교체한다.

03 캐시 기억 장치

예제 6-9

FIFO 교체 알고리즘을 사용하는 세트-연관 사상 캐시로 다음 블록을 연속으로 액세스하는 경우 각 캐시 라인에 적재되는 블록을 표시하고 히트율을 구하여라. 단, 각 세트의 라인 수는 (a) 2개, (b) 4개라고 가정한다.

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

풀이

(a) 캐시 라인 수가 2개인 경우 : 히트율=5/20

7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
7	7	1	1	0	0	0	4	4	3	3	3	2	2	2	0	0	0	0	1
	0	0	2	2	3	3	3	2	2	0	0	0	1	1	1	1	7	7	7
						hit					hit			hit		hit		hit	

← FIFO 방식 예제 오류 수정함(2023.06.07)

FIFO(First-In-First-Out)

캐시에 적재된지 가장 오래된 블록을 교체한다.
(Hit와 상관없이 오랜 된 블록이 교체된다)

(b) 캐시 라인 수가 4개인 경우 : 히트율=10/20

7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
7	7	7	7	7	3	3	3	3	3	3	3	3	3	2	2	2	2	2	2
	0	0	0	0	0	0	4	4	4	4	4	4	4	4	4	4	7	7	7
		1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0	0
			2	2	2	2	2	2	2	2	2	2	1	1	1	1	1	1	1
				hit		hit		hit	hit		hit	hit			hit	hit		hit	hit

End of Example

03 캐시 기억 장치

예제 6-10

LRU 교체 알고리즘을 사용하는 세트-연관 사상 캐시로 다음 블록을 연속으로 액세스하는 경우 각 캐시 라인에 적재되는 블록을 표시하고 히트율을 구하여라. 단, 각 세트의 라인 수는 (a) 2개, (b) 4개라고 가정한다.

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

풀이

(a) 캐시 라인 수가 2개인 경우 : 히트율=3/20

7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
7	7	1	1	0	0	0	0	2	2	0	0	2	2	2	2	1	1	0	0
	0	0	2	2	3	3	4	4	3	3	3	3	1	1	0	0	7	7	1
						hit					hit			hit					

(b) 캐시 라인 수가 4개인 경우 : 히트율=12/20

7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
7	7	7	7	7	3	3	3	3	3	3	3	3	3	3	3	3	7	7	7
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
		1	1	1	1	1	4	4	4	4	4	4	1	1	1	1	1	1	1
			2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2
					hit	hit		hit	hit	hit	hit	hit		hit	hit	hit		hit	hit

LRU(Least Recently Used)

각 캐시 라인에 USE 비트를 포함
액세스된 캐시 라인의 USE
비트를 1로 설정하고 다른 캐시
라인은 0으로 설정
새로운 블록으로 교체시 USE
비트가 0인 캐시라인을 교체함

End of Example

03 캐시 기억 장치

4 쓰기 정책 write policy

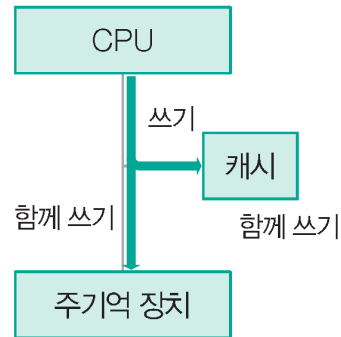
- 캐시의 블록이 변경되었을 때 그 내용을 주기억 장치에 갱신하는 시기와 방법의 결정

❖ Write-through

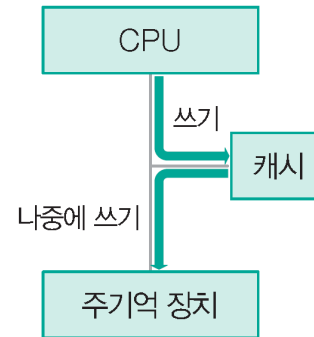
- 모든 쓰기 동작들이 캐시로 뿐만 아니라 주기억 장치로도 동시에 수행되는 방식

❖ Write-back

- 캐시에서 데이터가 변경되어도 주기억 장치에는 갱신되지 않는 방식



(a) write-through



(b) write-back

그림 6-34 쓰기 정책

write-back:

캐시에서 데이터가 변경되어도 주기억 장치에는 갱신하지 않는다. 캐시에서 데이터가 수정되면 해당 블록의 update비트가 셋되어 나중에 그 블록이 교체될 때 블록 전체가 주기억 장치에 갱신된 후 교체된다. 캐시 미스가 발생하여 블록을 교체할 경우 해당 블록의 데이터가 수정된 상태이면 주기억 장치 갱신을 위한 추가 시간이 소요된다.

03 캐시 기억 장치

❖ 각 방식의 장단점

Write-through

장점	• 캐시에 적재된 블록의 내용과 주기억 장치에 있는 그 블록의 내용이 항상 같다.
단점	• 모든 쓰기 동작이 주기억 장치 쓰기를 포함하므로, 쓰기 시간이 길어진다.

Write-back

장점	• 기억장치에 대한 쓰기 동작의 횟수가 최소화되고, 쓰기 시간이 짧아진다.
단점	• 캐시의 내용과 주기억 장치의 해당 내용이 서로 다르다.

⇒ 블록을 교체할 때는 캐시의 상태를 확인하여 주기억 장치에 갱신하는 동작이 선행되어야 하며, 그를 위하여 각 캐시 라인이 상태 비트(status bit)를 가지고 있어야 한다.

캐시 일관성(Cache Coherence)

03 캐시 기억 장치

예제 6-11

주기억 장치 액세스 시간이 50ns, 캐시 액세스 시간이 5ns인 시스템이 있다. 전체 액세스 요구 중에서 70%는 읽기 동작이고, 30%는 쓰기 동작이었으며, 캐시 히트율은 90%였다. 캐시 쓰기 정책이 (a) write-through일 때와 (b) write-back일 때 평균 기억 장치 액세스 시간을 각각 구하여라. 단, write-back에서 미스가 발생한 경우 새로운 블록을 적재하기 위해 교체할 라인들의 20%는 이미 수정된 상태에 있었다고 가정한다.

풀이

(a) Write-through 정책인 경우

- 읽기동작의 평균시간 = $0.9 \times 5\text{ns} + 0.1 \times 50\text{ns} = 9.5\text{ns}$
- 쓰기동작의 평균시간 = 50ns (모든 쓰기 동작은 주기억 장치를 액세스)
- 평균 기억장치 액세스 시간 = $0.7 \times 9.5\text{ns} + 0.3 \times 50\text{ns} = 21.65\text{ns}$

(b) Write-back 정책인 경우 읽기동작과 쓰기동작에 같은 시간이 걸리므로,
평균 기억장치 액세스 시간 = $0.9 \times 5\text{ns} + 0.1 \times (50\text{ns} + 0.2 \times 50\text{ns}) = 10.5\text{ns}$

$$T_a = H \times T_c + (1 - H) \times (T_c + T_m) \\ \approx H \times T_c + (1 - H) \times T_m$$

캐시 미스 발생 전 블록내 일부 데이터가 변경된 경우, 캐시 일관성을 위해 그 데이터들을 주기억장치에 업데이트 해야함 -> 추가 주기억장치 액세스시간 발생

Write-back
방식이 더 빠름

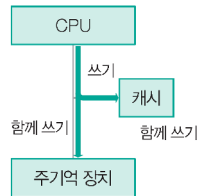
Write-Through (캐시와 주기억장치 동시에 기록): 읽기시간 = 히트율x캐시+(1-히트율)x주기억장치

Write-Back (캐시만 쓰다가 필요시 주기억장치 기록):

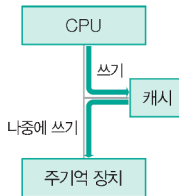
캐시에 히트되었을 때 캐시만 액세스하면 되므로 읽기와 쓰기 동작에 걸리는 시간은 동일하게 5ns.

미스되었을 때에는 모두 주기억 장치를 액세스하고, 캐시 내용 중 20%는 이미 수정되어 있으므로 해당되는 주기억 장치를 추가로 액세스해야 함.

캐시 미스가 발생하여 블록을 교체할 경우 해당 블록의 데이터가 수정된 상태이면 주기억 장치 갱신을 위한 추가 시간이 소요된다.



(a) write-through
그림 6-34 쓰기 정책



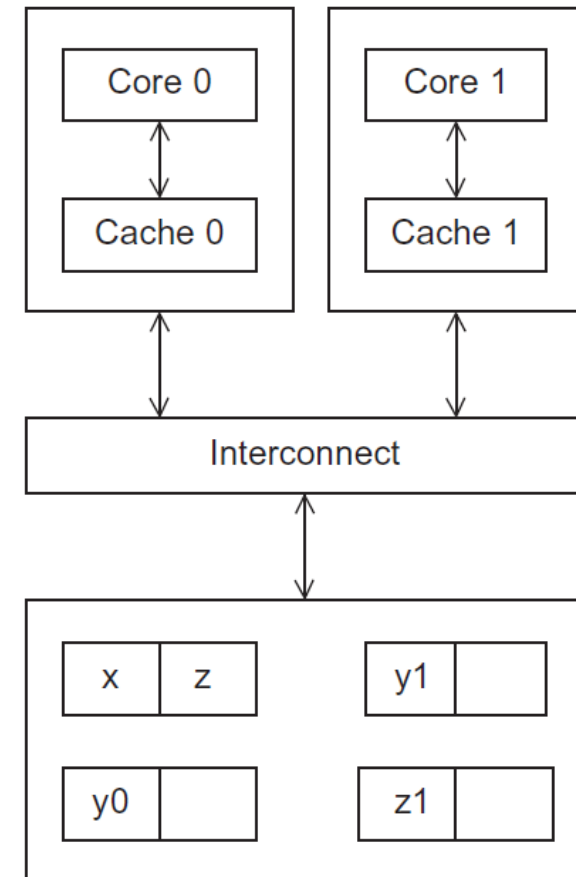
(b) write-back

Caches, Cache-Coherence, and False Sharing

참고(시험범위제외)

- Recall that chip designers have added blocks of relatively fast memory to processors called **cache memory**.
- The use of cache memory can have **a huge impact on shared-memory**.
- Programmers have no control over caches and when they get updated.
- A write-miss occurs when a core tries to update a variable that's not in cache, and it has to access main memory.
- 캐시 메모리 사용효과를 증대시키기 위해서는 'temporal and spatial locality' 시간적 공간적 지역성을 확보하도록 코딩을 해야 함

멀티쓰레드에서 캐쉬 사용 문제



캐시 일관성; Cache Coherence

참고(시험범위제외)

y0 privately owned by Core 0

y1 and z1 privately owned by Core 1

x = 2; /* shared variable */

Time	Core 0	Core 1
0	y0 = x;	y1 = 3*x;
1	x = 7;	Statement(s) not involving x
2	Statement(s) not involving x	z1 = 4*x;

y0 eventually ends up = 2

y1 eventually ends up = 6

z1 = ???

캐시 라인에 어떤 값(x=?)이 쓰여질 때마다
그 라인이 다른 프로세스의 캐시에 저장되어
있다면

(캐시 일관성 위해서) 전체 라인을
무효화(invalid) 시킴

코드에서 x가 업데이트 될 때마다 x를 다시
캐시로 로드하게됨

프로그래머는 캐시가 언제 업데이트될 것인지를 직접 제어하지 못함
공유 변수에 대해 여러 프로세서 캐시된 데이터가 일관적이어야 하는데
그렇지 못한 것을 '캐시 일관성 문제'라고 함

거짓 공유; False Sharing

참고(시험범위제외)

- 실제로 스레드간 공유되지 않는 데이터이지만 캐시 구조상 공유되는 것처럼 행동하는 경우 → 멀티스레드의 성능저하를 발생하게 됨

```
long long num1 = 0;
long long num2 = 0;
alignas(64) long long num1 = 0; //바뀐 부분
alignas(64) long long num2 = 0; //바뀐 부분
```

```
void fun1() {
    for (long long i = 0; i < 1000000000; i++)
        num1 += 1;
}

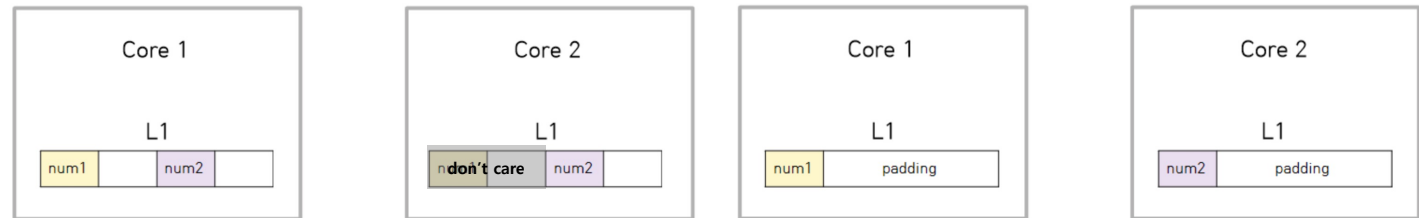
void fun2() {
    for (long long i = 0; i < 1000000000; i++)
        num2 += 1;
}

void fun3() {
    for (long long i = 0; i < 2000000000; i++) {
        num3 += 1;
    }
}
```

```
2000000000
9.34802
-----
2000000000
3.33701
```

싱글스레드가
6초나 빠름

해결방법:
dummy 항목을 갖도록 데이터 padding



Core 1에는 **num1**과 **num2**가 있습니다. 그 이유는 캐시에서 64byte씩 통째로 읽어오기 때문입니다.

Core 2에는 **num2**만 있습니다. **num2**를 시작점으로 64byte를 통으로 읽어왔기 때문입니다.

(이렇게 된 이유는 예제 코드에서의 **num1**과 **num2**가 붙어있기 때문에 64byte씩 읽어오게 될 경우, 데이터에 두개의 변수 정보가 들어갈 확률이 높습니다.)

그래서 Core 2는 문제없이 연산하게 됩니다.

하지만 Core 1은 좀 다릅니다.

CPU에서는 캐시 일관성(cache coherence)이라는 메커니즘이 존재합니다.

CPU는 그냥 계산만 처리하는 기계입니다.

비록 Core 1에서 **num2**의 계산을 하지 않지만, 캐시 입장에선 데이터가 공유되어 무슨일이 일어날지 알 수 없습니다.

따라서 Core 1의 연산을 멈추고 **num2**의 값을 다시 받아옵니다.

일종의 동기화 작업입니다.

캐시 일관성

03 캐시 기억 장치

5 라인 크기

❖ 블록 크기에 따른 특성

- 캐시 용량은 한정되어 있기 때문에 블록 크기가 커질수록 캐시에 들어올 수 있는 블록의 수는 줄어든다. 새로운 블록을 읽어 올 때마다 원래 캐시 내용은 교체되기 때문에 블록 수가 적으면 인출된 직후에 다른 블록과 다시 교체된다.
- 블록 크기가 커질수록 원하는 워드에서 멀리 떨어져 있는 워드들도 같이 읽혀 오며, 그들은 가까운 미래에 사용될 가능성은 낮다.

⇒ 대략적으로 블록 크기가 8~32바이트 정도가 최적에 가까운 것으로 알려져 있다.

03 캐시 기억 장치

6 캐시 수

□ 계층적 캐시

- **온-칩 캐시**(on-chip cache) : 캐시 액세스 시간을 단축시키기 위하여 CPU 칩 내에 포함시킨 캐시 (그림의 L1)
- **계층적 캐시**(hierarchical cache) : 온-칩 캐시를 1차(L1) 캐시로 사용하고, CPU 외부에 더 큰 용량의 2차(L2) 캐시를 설치하는 방식
- **분리 캐시**(split cache) : 캐시를 명령어 캐시와 데이터 캐시로 분리

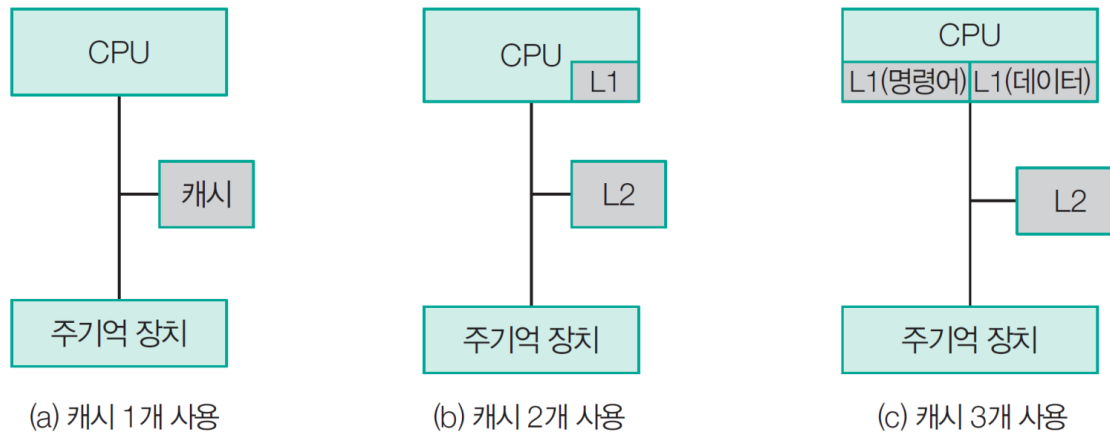


그림 6-35 다양한 계층적 캐시 구조

03 캐시 기억 장치

❖ 계층적 캐시에서의 히트율

- L2의 용량이 L1보다 크며, L1의 모든 내용이 L2에도 존재
- 먼저 L1을 검사하고, 만약 원하는 정보가 L1에 없다면 L2를 검사하며, L2에도 없는 경우에는 주기억 장치를 액세스
- L1은 속도가 빠르지만, 용량이 작기 때문에 L2 보다 히트율은 더 낮다.
- 평균 기억 장치 액세스 시간(T_a)

$$T_a = H_1 \times T_{L1} + (H_2 - H_1) \times T_{L2} + (1 - H_2) T_m$$

L1 캐시의 액세스 시간과 히트율 : T_{L1}, H_1

L2 캐시의 액세스 시간과 히트율 : T_{L2}, H_2

주기억 장치의 액세스 시간 : T_m

H_2 는 전체 기억 장치 액세스에 대한 L2의 히트율을 의미한다.

03 캐시 기억 장치

예제 6-12

L1 캐시, L2 캐시, 주기억 장치의 액세스 시간이 각각 $2ns$, $5ns$, $50ns$ 이고, $H_1=0.7$, $H_2=0.9$ 라고 할 때, 평균 기억 장치 액세스 시간을 구하여라.

풀이

$$\begin{aligned}T_a &= 0.7 \times 2ns + (0.9 - 0.7) \times 5ns + (1 - 0.9) \times 50ns \\&= 1.4ns + 1ns + 5ns = 7.4ns\end{aligned}$$

End of Example

03 캐시 기억 장치

□ 통합 캐시와 분리 캐시

❖ 통합 캐시(unified cache)

- 온-칩 캐시가 처음 출현했을 때, 대부분은 명령어와 데이터를 하나의 캐시에 저장

❖ 분리 캐시(split cache)

- 캐시를 명령어 캐시와 데이터 캐시로 분리
- 명령어 인출 유닛과 실행 유닛 간의 캐시 액세스 충돌 제거
- 고성능 파이프 라인 프로세서들에서 사용

가상 기억 장치

04 가상 기억 장치

❖ 가상 기억 장치(virtual memory)

- 하드 디스크처럼 용량이 큰 보조 기억 장치를 주기억 장치처럼 사용하는 개념
- CPU가 참조하는 가상 주소를 주기억 장치의 실제 주소로 변환하는 주소 매핑이 필요하다.

❖ 주소 매핑(address mapping)

- 프로그래머가 프로그램에 표시한 주소를 가상 주소(Virtual Address) 또는 논리 주소(Logical Address)라 하고, 이들의 주소 집합을 주소 공간이라 한다.
- 실제 프로그램이 적재되는 주기억 장치의 주소를 물리 주소(Physical Address)라 하고, 이들의 집합을 메모리 공간이라 한다.

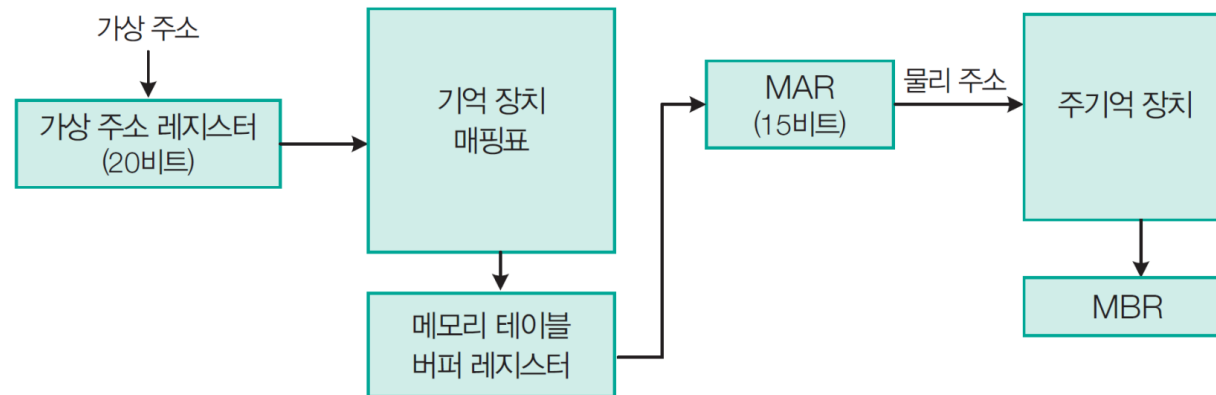


그림 6-36 가상 주소를 물리 주소로 변환하여 정보를 얻는 과정

04 가상 기억 장치

1 가상 기억 장치의 매핑

□ 페이지에 의한 매핑

- **페이지**(page) : (프로그래머가 사용하는) 주소 공간을 고정 크기로 나눈 것
- **블록**(block) : (주 기억 장치인) 메모리 공간을 고정 크기로 나눈 것
- 페이지에 대한 기억 장치 매핑표를 가지고 **페이지를 블록으로 변환**한다.
- 가상 주소 페이지가 주 기억 장치에 존재하지 않는 경우에는 **페이지 오류**(page fault)가 발생되었다고 하며, 이러한 페이지 오류가 자주 발생하는 경우를 **스래싱**(thrashing)이라고 한다.
- 주소 공간이 8K, 메모리 공간이 4K인 컴퓨터 시스템의 예 (최대 페이지 4개가 메모리 공간의 블록 4개 중 하나에 들어간다.)

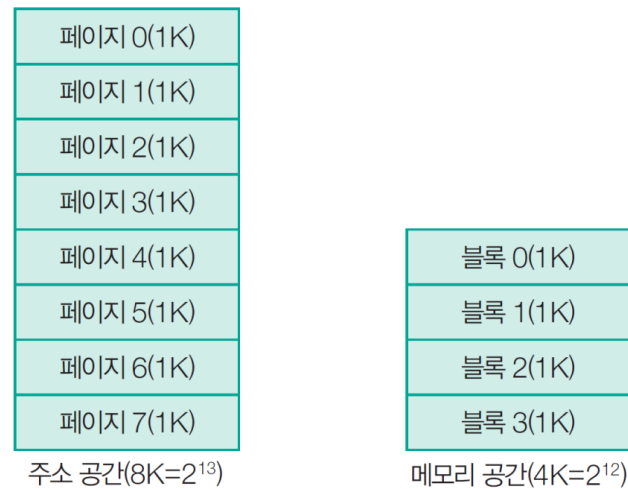


그림 6-37 주소 공간과 메모리 공간의 분리(1K)

가상 기억 장치의 매핑 방식

- 1) 페이지에 의한 매핑
- 2) 연관 기억 장치를 이용한 기억 장치 매핑
- 3) 세그먼트에 의한 매핑

04 가상 기억 장치

❖ 기억 장치 매핑표

- 페이지 번호에서 블록 번호로의 변환만 하면 된다.
- 그림에서 페이지 1, 2, 5, 6은 주기억 장치 블록 2, 0, 1, 3에 각각 저장되어 있다.
- **현존 비트가 1**이면 해당 페이지가 주기억 장치에 존재함을 의미

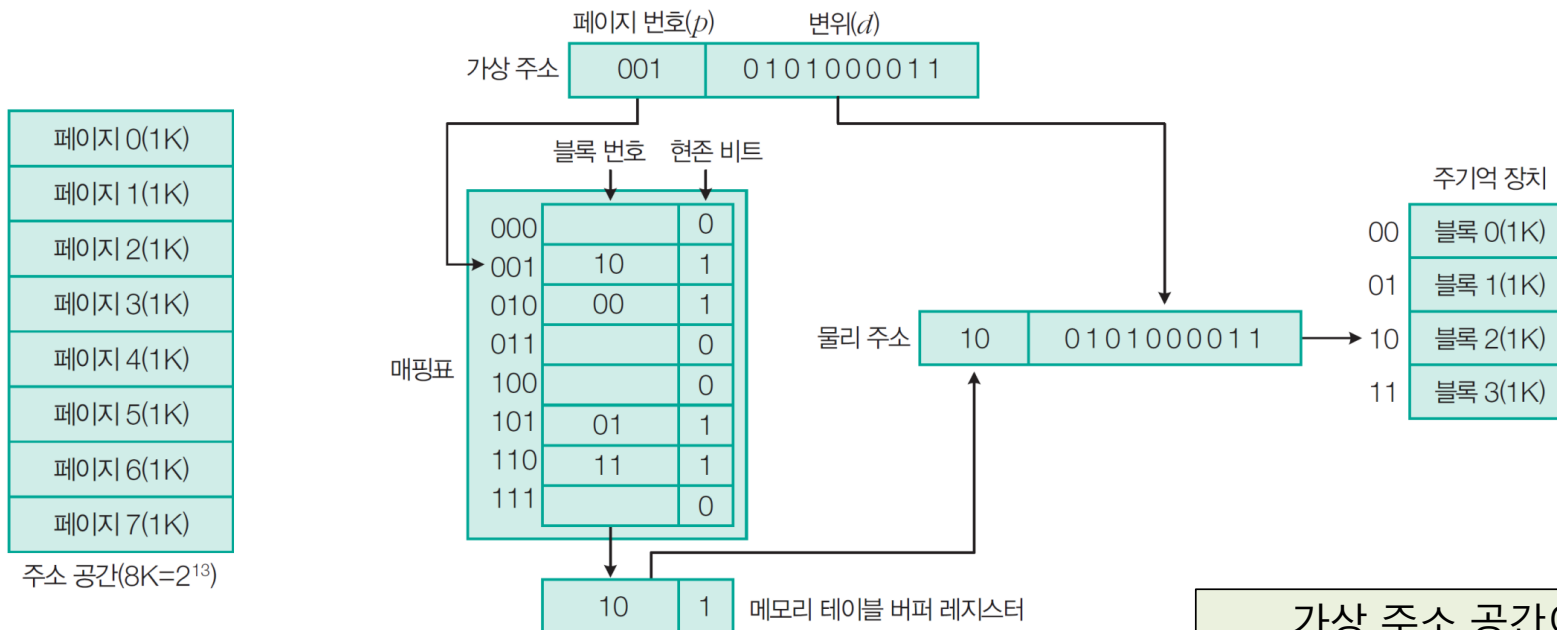


그림 6-38 기억 장치 매핑표 구조

가상 주소 공간이 $8K (= 2^3 2^{10})$
주기억장치 메모리 공간이 $4K (= 2^2 2^{10})$

04 가상 기억 장치

예제 6-13

용량이 1MB ($=2^{20}$)인 주기억 장치를 가진 컴퓨터 시스템에서 32비트의 가상 주소($=2^{32}$)를 사용한다고 하자. 페이지 크기가 1K 워드고, 1워드는 4바이트라고 할 때 가상 주소와 물리 주소의 구성은 어떻게 될까?

풀이

한 페이지는 1K($=2^{10}$) 워드고, 한 워드가 4($=2^2$)바이트이므로
한 페이지의 크기는 4KB($=2^{12}$)이므로 변위는 12비트이어야 한다. 주기억 장치의 용량이 1M($=2^{20}$)바이트고, 한 페이지의 크기는 2^{12} 바이트이므로 주기억 장치에 탑재가능한 페이지수 (=블록 수)는 256($=2^8$)개다.

$$(\text{주기억 장치의 용량})/(\text{한 페이지의 크기})=2^{20}/2^{12}=2^8$$

따라서 물리 주소는 20비트로 구성되므로 페이지 주소에 8비트, 변위에 12비트가 할당된다.



가상 주소는 32비트이므로 페이지 번호에 20비트가 할당된다.



End of Example

04 가상 기억 장치

■ 연관 기억 장치를 이용한 기억 장치 매핑

- 앞 방법에서 기억 장치 매핑표는 기억 장치의 이용상 비효율적이다.
- 이를 위해 연관 기억 장치를 사용해 **기억 장치 매핑표의 워드 수와 주기억 장치의 블록 수를 같게 한다.**
- 가상 주소는 **인자 레지스터**로 전송되며, **키 레지스터**의 마스크 값에 따라 이 레지스터의 페이지 번호 비트는 연관 기억 장치의 모든 페이지 번호 부분과 비교된다.
- 매치가 일어나지 않으면 운영체제에 의해 요구되는 페이지가 보조 기억 장치에서 주기억 장치로 옮겨진다.

가상 기억 장치의 매핑 방식

- 1) 페이지에 의한 매핑
- 2) 연관 기억 장치를 이용한 기억 장치 매핑
- 3) 세그먼트에 의한 매핑

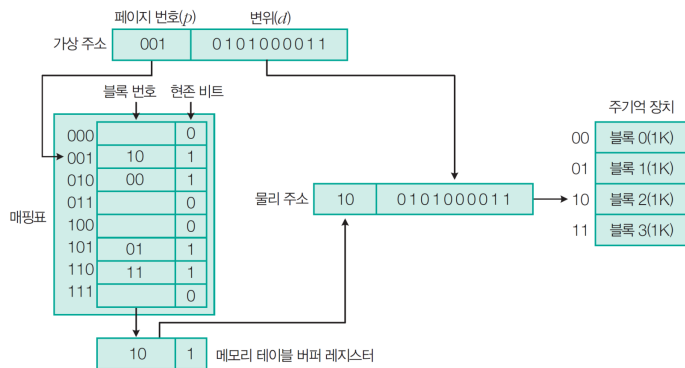


그림 6-38 기억 장치 매핑표 구조

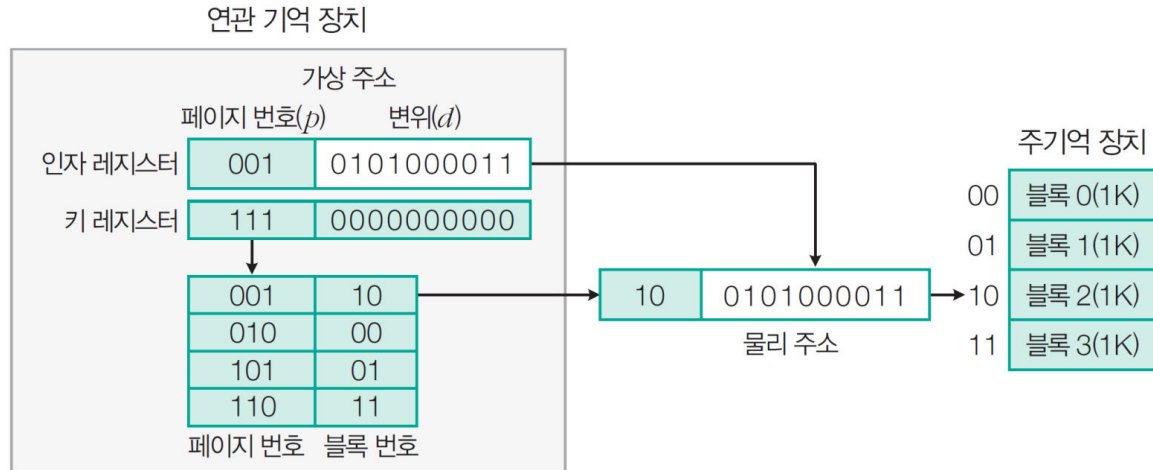


그림 6-39 연관 기억 장치를 이용한 매핑표 구조

04 가상 기억 장치

가상 기억 장치의 매핑 방식

- 1) 페이지에 의한 매핑
- 2) 연관 기억 장치를 이용한 기억 장치 매핑
- 3) 세그먼트에 의한 매핑

□ 세그먼트에 의한 매핑

- 고정 길이의 페이지가 아니라 실행되는 프로그램에 따라 **가변 길이의 세그먼트**로 매핑한다.
- 세그먼트로 된 프로그램에 의해 지정되는 **주소를 논리 주소**라고 한다.
- 프로그래머가 프로그램을 세그먼트화하고, 다시 시스템이 각 세그먼트를 페이지화한다.
- 논리 주소의 세그먼트 번호 → 세그먼트 표 값 + 페이지 번호 → 페이지 표 값(블록 주소) + 변위 → 물리 주소
- 속도 향상을 위해 세그먼트 표나 페이지 표를 접근 시간이 빠른 연관 기억 장치에 저장한다.

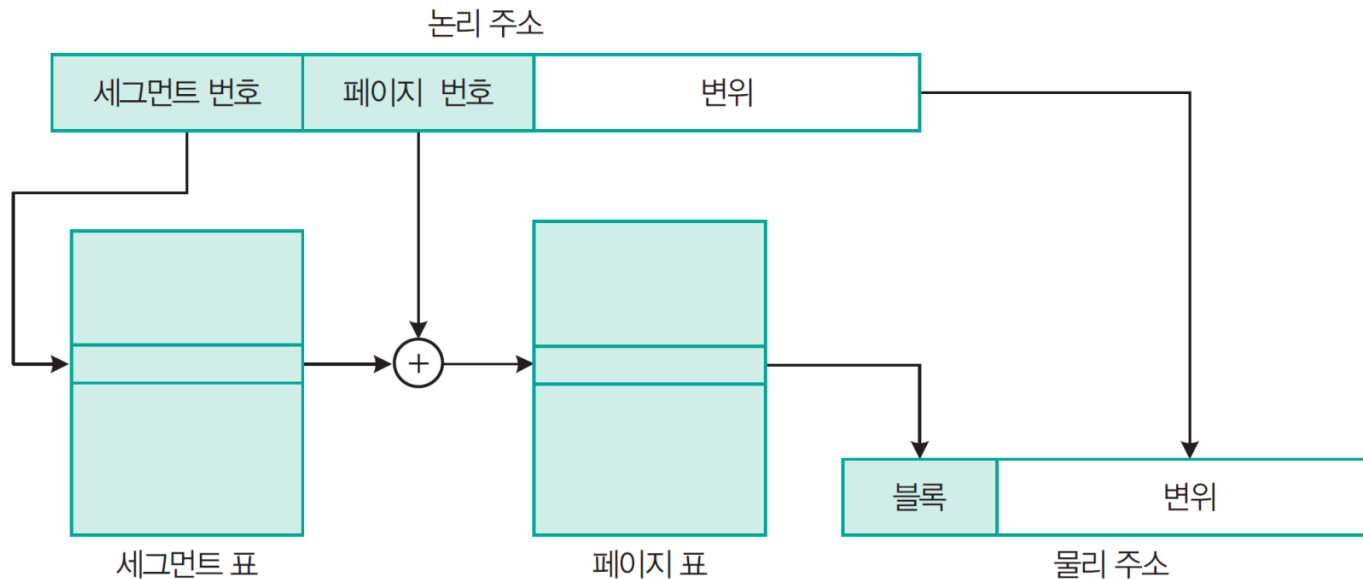


그림 6-40 세그먼트에 의한 매핑에서 논리 주소를 물리 주소로 매핑

04 가상 기억 장치

2 페이지 교체 알고리즘

- 참조하고자 하는 페이지가 주기억 장치에 없을 경우 **페이지 오류**(page fault)라 하고, 이 조건이 발생하면 요구된 페이지가 주기억 장치로 옮겨질 때까지 **프로그램 수행이 중단된다**.
- 보조 기억 장치에서 주기억 장치로 페이지를 전송하는 것은 입출력 동작이므로 운영체제는 이 일을 I/O 프로세서에 맡긴다.
- 페이지 교체 알고리즘**: 새로운 페이지가 주기억 장치로 전송될 때, 주기억 장치가 꽉 차 있으면 제거할 페이지를 선택(FIFO, LRU, LFU 알고리즘 등)

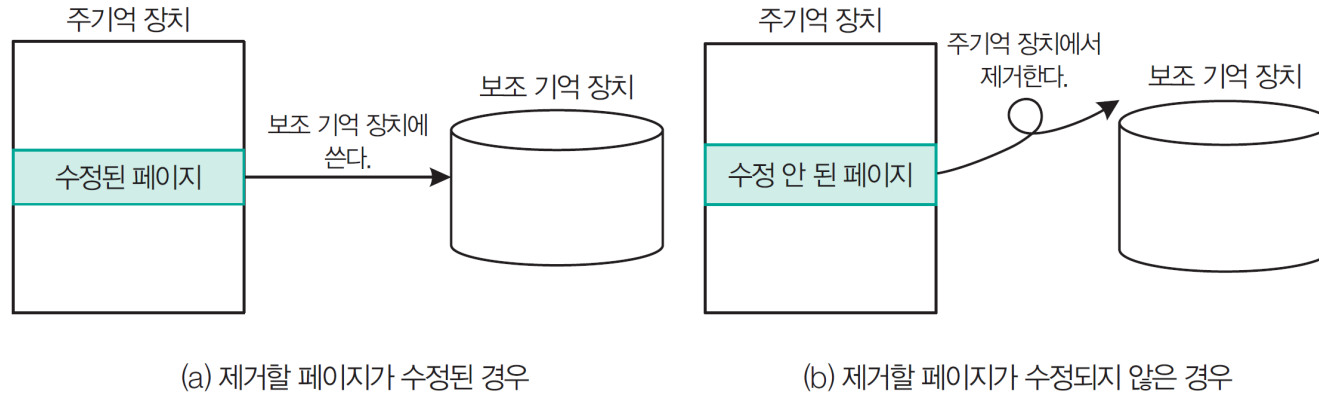


그림 6-41 페이지 교체할 때 동작

04 가상 기억 장치

- 다음 예를 통해 각 알고리즘을 살펴보자.

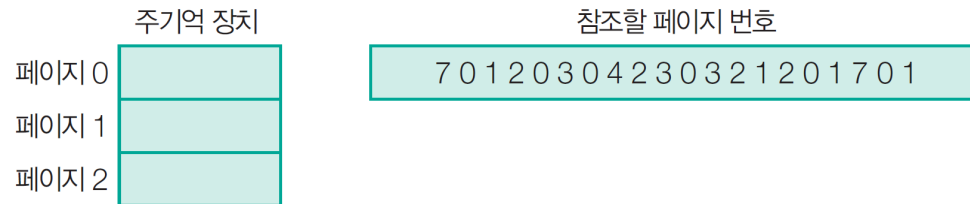


그림 6-42 페이지 교체 알고리즘 동작 예

❖ FIFO(First-In First Out) 알고리즘

- 주 기억 장치에 가장 오래 있었던 페이지를 교체
- 구현하기가 쉽다는 장점이 있으나, 어떤 상황에서는 페이지가 너무 자주 교체되는 단점이 있다.

7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
7	7	7	2	2	2	2	4	4	4	0	0	0	0	0	0	0	7	7	7
	0	0	0	0	3	3	3	2	2	2	2	2	1	1	1	1	1	0	0
		1	1	1	1	0	0	0	3	3	3	3	3	2	2	2	2	2	1

페이지 폴트 15회 발생

04 가상 기억 장치

❖ LRU(Least Recently Used) 알고리즘

- 최근까지 가장 오랫동안 사용되지 않았던 페이지를 선택하여 제거하는 방법

7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
7	7	7	2	2	2	2	4	4	4	0	0	0	1	1	1	1	1	1	1
	0	0	0	0	0	0	0	0	3	3	3	3	3	3	0	0	0	0	0
		1	1	1	3	3	3	2	2	2	2	2	2	2	2	2	7	7	7

페이지 폴트 12회 발생

❖ LFU(Least Frequently Used) 알고리즘

- 사용 빈도가 가장 낮은 페이지를 선택해서 제거하는 방법

7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
7	7	7	2	2	2	2	4	4	3	3	3	3	3	3	3	3	3	3	3
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
		1	1	1	3	3	3	2	2	2	2	2	1	2	2	1	7	7	1

페이지 폴트 13회 발생

04 가상 기억 장치

❖ NUR(Not Used Recently) 알고리즘

- LRU와 비슷한 알고리즘
- 최근의 사용 여부에 따라 **참조 비트**가 세트되고 주기적으로 리셋됨
- 페이지가 수정되면 **변형 비트**가 세트됨
- **참조 비트**와 **변형 비트**를 사용하여 최근에 사용하지 않은 페이지를 교체하는 방법

표 6-10 참조 비트와 변형 비트에 따른 교체 우선순위

참조 비트	변형 비트	교체 순서
0	0	1
0	1	2
1	0	3
1	1	4

04 가상 기억 장치

❖ 캐시 메모리, 주기억 장치, 가상 기억 장치 간의 데이터 이동

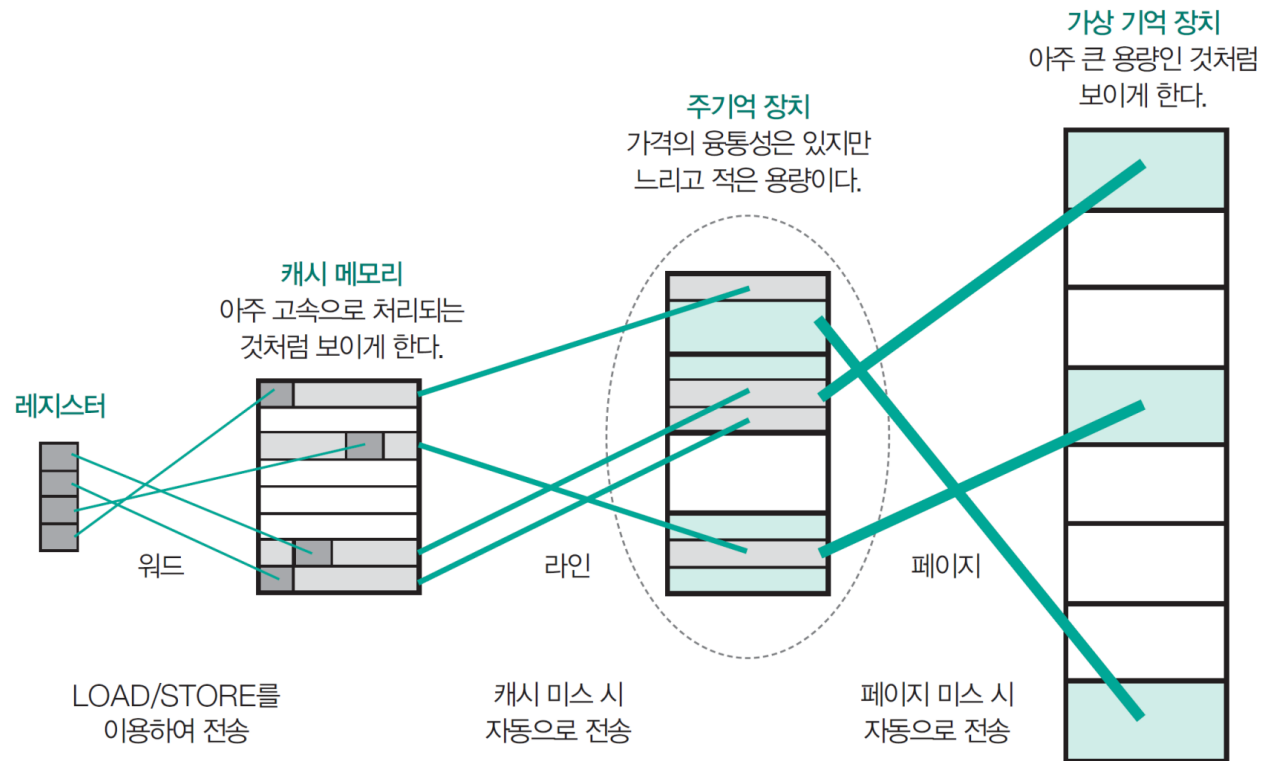


그림 6-43 메모리 계층 구조에서의 데이터 이동

참고자료: 최신 기억 장치 기술

06 최신 기억 장치 기술

- 기억장치의 액세스 속도는 CPU에 비하여 현저히 낮음
- 동영상 편집, 음성/영상 압축과 같은 대규모 데이터 처리 응용 증가
 - ⇒ 주기억 장치 병목 현상 심화
 - ⇒ 새로운 유형의 기억장치(SDRAM, DDR) 개발을 통한 고속화 필요

06 최신 기억 장치 기술

1 SDRAM(Synchronous DRAM)

- SDRAM은 DRAM의 액세스 속도를 향상시키기 위해 개발된 반도체 기억 장치
- SDRAM은 시스템 클록 신호에 맞추어 데이터를 전송하는 동기식 DRAM이다.
- 제어기는 SDRAM의 데이터 입출력과 메모리 셀의 재충전(refresh) 기능을 담당한다.
- 초기 컴퓨터 시스템에서 메모리 제어기는 메인보드 칩셋에 들어 있었지만, 최근에는 CPU가 메모리 제어기를 내장하고 있어 직접 메모리를 제어할 수 있다.

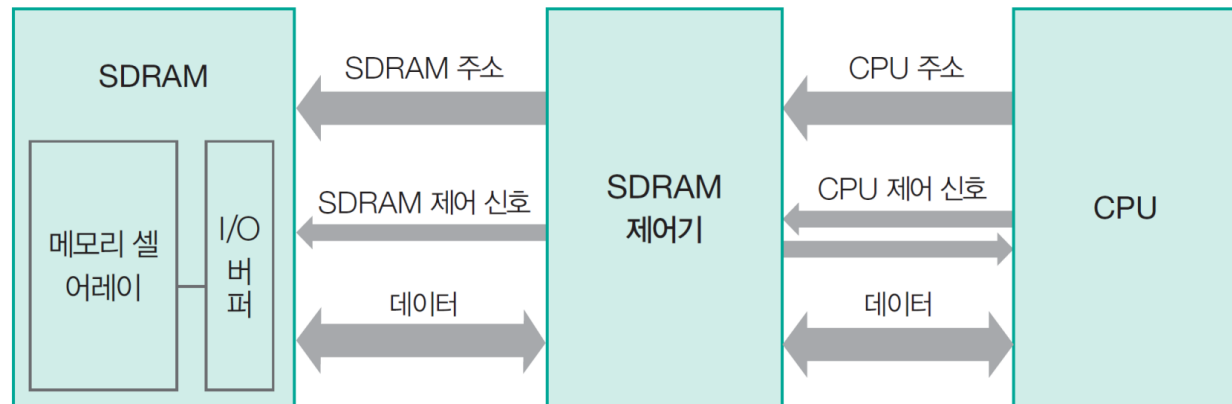


그림 6-49 SDRAM의 인터페이스 구조

06 최신 기억 장치 기술

❖ 256Mbit SDRAM의 내부 구조(K4S560832 SDRAM)

- SDRAM은 데이터를 동시에 액세스하기 위해 내부적으로 여러 개의 **뱅크**로 구성되어 있다.
- 행 주소($A_0 \sim A_{12}$)와 열 주소($A_0 \sim A_9$)를 시간 차이를 두어 $A_0 \sim A_{12}$ 핀에다가 실어 주고, 뱅크 주소는 핀(BA_0, BA_1) 2개를 사용한다.

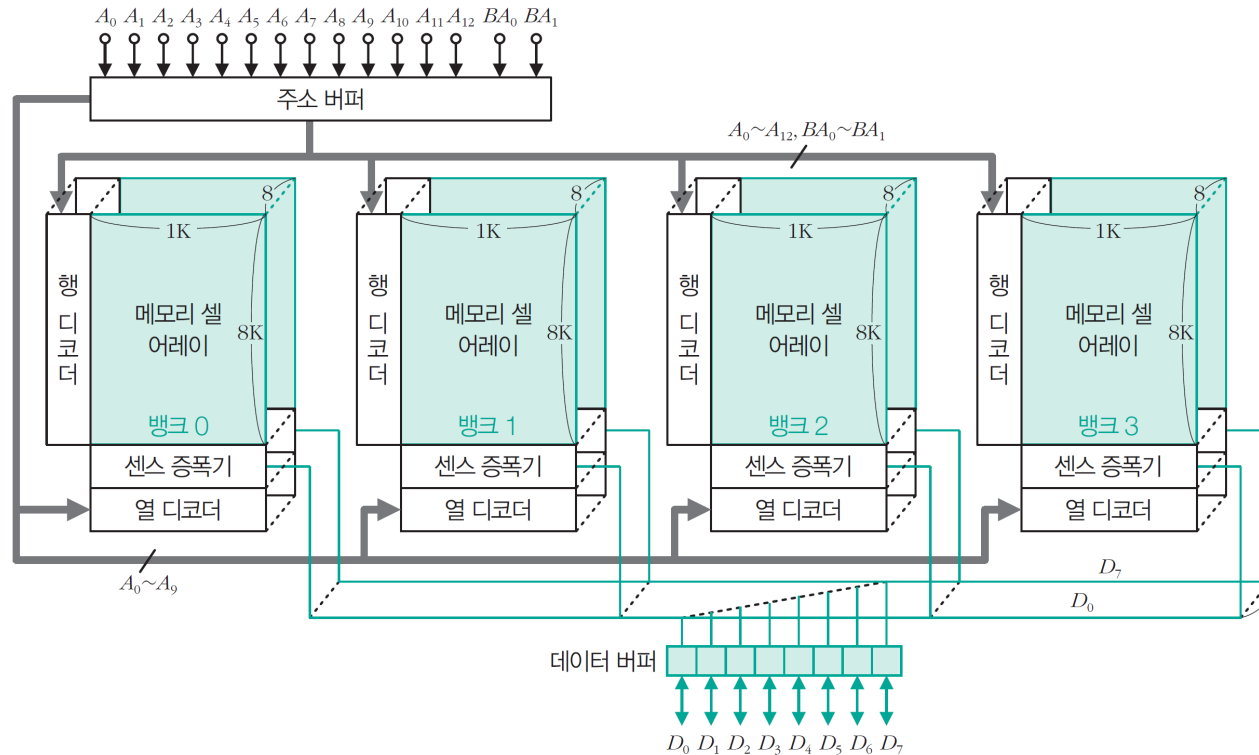


그림 6-50 256Mbit SDRAM의 내부 구조(K4S560832 SDRAM)

06 최신 기억 장치 기술

❖ 256Mbit SDRAM의 용량 계산

- 행(row) : $8K(=2^{13})$ 개의 기억소자 배열
- 각 배열에는 8Kbit (1Kbytes) 씩 저장
⇒ 행 주소 = 13비트($A_0 \sim A_{12}$), 열 주소 = 10비트($A_0 \sim A_9$)
- 뱅크 선택 2개(BA_0, BA_1)
- 칩의 입출력(데이터) 선의 수 : 8개
- 용량 : $2^{13} \times 2^{10} \times 2^2 \times 8 = 2^{28} = 2^8 \times 2^{20} = 256\text{Mbit} = 32\text{Mbyte}$

❖ 버스트 모드(burst mode)

- 행 주소, 열 주소, 뱅크 주소에 의해 해당 행에 저장된 비트 8K개가 센스 증폭기로 모두 이동
- 하나의 행에 포함된 비트들은 열 주소에 의해 8비트 단위(바이트)로 데이터 버스에 실리게 된다.
- **버스트 모드**(burst mode) : 여러 바이트들을 연속적으로 전송하는 동작
- **버스트 길이**(burst length, BL) : 각 버스트 동작 동안에 전송되는 데이터 바이트들의 수(보통 1, 2, 4, 8)

06 최신 기억 장치 기술

❖ SDRAM 액세스에 필요한 신호

- $\overline{CS}$: 칩 선택(Chip Select) 단자
- $\overline{RAS}$: 클록의 상승 에지에서 행 주소를 래치(Row Address Strobe)
- $\overline{CAS}$: 클록의 상승 에지에서 열 주소를 래치(Column Address Strobe)
- $\overline{WE}$: 쓰기 동작을 인에이블(Write Enable), 읽기 동작인 경우에는 $\overline{WE} = 1$

❖ 읽기 동작

- CPU는 한 클록 주기 동안에 시스템 버스를 통하여 주소와 읽기 신호를 기억장치로 보낸 후, 그 결과를 기다리지 않고 내부적으로 다른 연산을 수행
- SDRAM은 주소와 읽기 신호를 받은 즉시 읽기 동작을 시작하며, 그 동작이 완료되면 시스템 버스 사용권을 획득한 후, 다음 클록 주기 동안 버스를 통하여 CPU로 데이터 전송
- CPU는 그 데이터를 받아서, 다음 연산을 수행

06 최신 기억 장치 기술

❖ SDRAM의 읽기 동작 타이밍도

- **CAS 지연**: $\overline{CAS}$ 신호가 입력된 순간부터 데이터가 버스에 실릴 때까지의 시간
- CAS 지연 값은 SDRAM의 클록 속도와 SDRAM의 응답 시간을 고려하여 결정된다.
- SDRAM 메모리는 클록 신호의 상승 에지에서 동기화되어 동작하고 있다.
- **버스트 모드**의 효과: 버스트 사용권을 한 번 획득한 후, 여러 클록 동안 연속 전송 가능

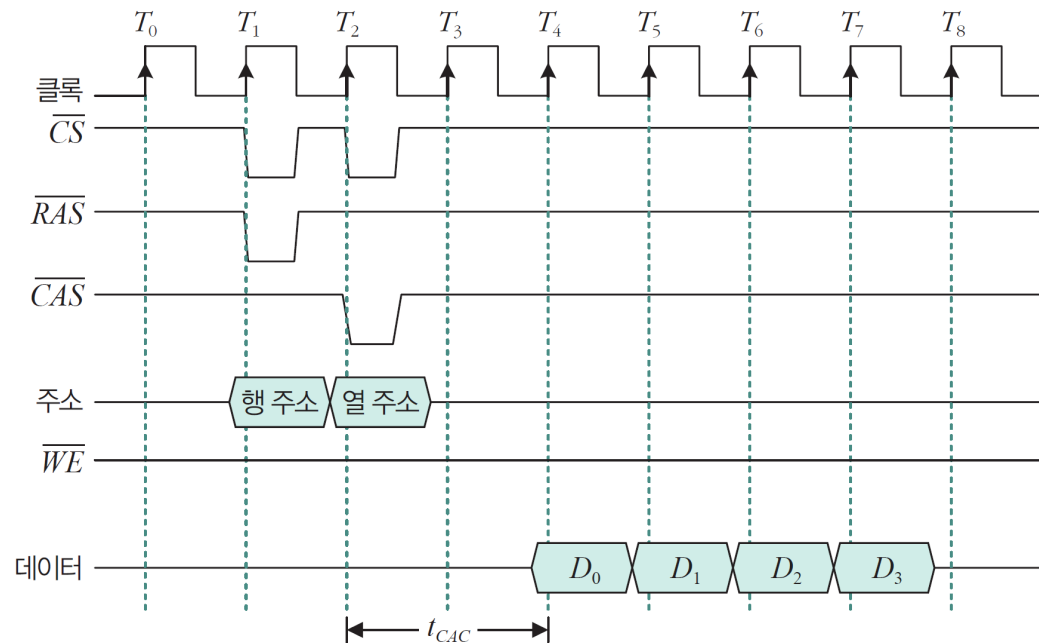


그림 6-51 버스트 읽기 동작의 타이밍도 ($CL=2, BL=4$)

06 최신 기억 장치 기술

❖ SDRAM의 쓰기 동작 타이밍도

- 쓰기 동작에서는 CAS 지연이 항상 0이다.

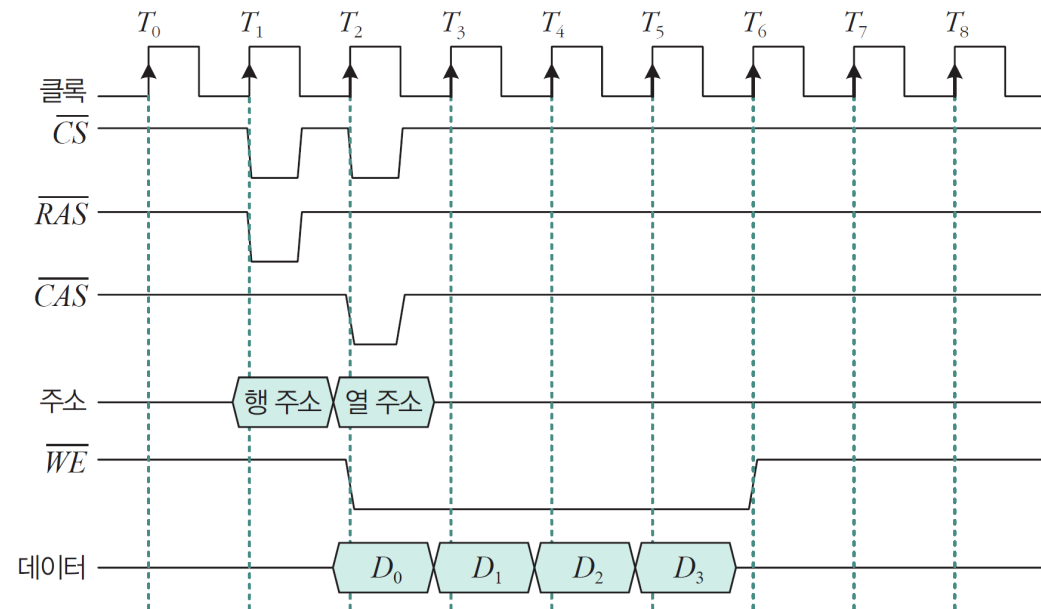


그림 6-52 버스트 쓰기 동작의 타이밍도 ($BL = 4$)

06 최신 기억 장치 기술

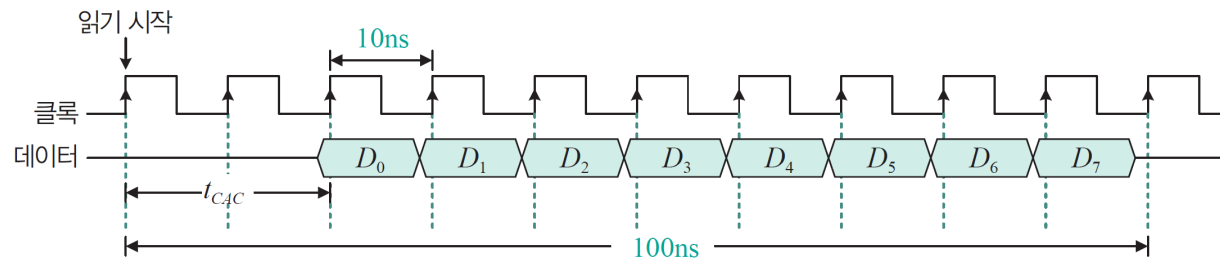
2 DDR SDRAM

- DDR SDRAM은 다음 두 기술을 이용해 DDR→DDR2→DDR3→DDR4로 발전하였다.
 - ① 클록의 상승 에지와 하강 에지에서 데이터를 전송한다.
 - ② 메모리와 제어기 사이의 버스 인터페이스 회로의 전송 속도를 높인다.

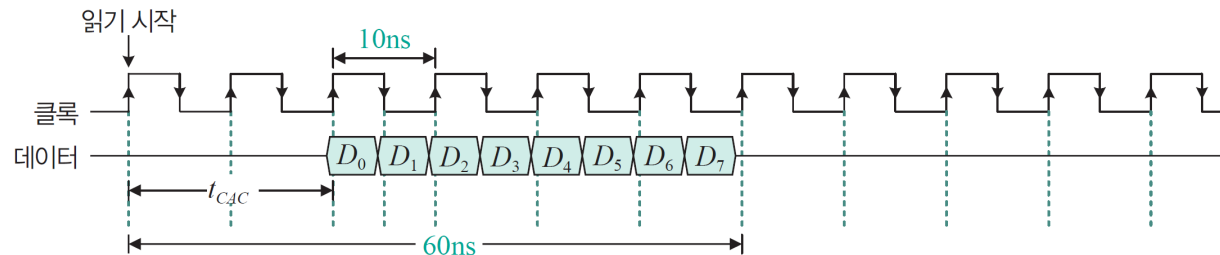
06 최신 기억 장치 기술

❖ DDR SDRAM

- DDR은 SDRAM에 ①번 기술(상승 에지와 하강 에지에서 데이터 전송)을 적용한 것이다.
- 읽기 시간 비교(클록 주파수 100MHz, $BL=8$)
 - SDR SDRAM(PC100) : $100\text{ns}(=20\text{ns}+8 \times 10\text{ns})$
 - DDR SDRAM(DDR-200) : $60\text{ns}(=20\text{ns}+4 \times 10\text{ns})$



(a) SDR SDRAM(PC100)



(b) DDR SDRAM(DDR-200)

그림 6-53 SDR SDRAM과 DDR SDRAM의 읽기 동작 타이밍도 ($CL=2$, $BL=8$)

06 최신 기억 장치 기술

❖ 기억장치 대역폭(memory bandwidth)

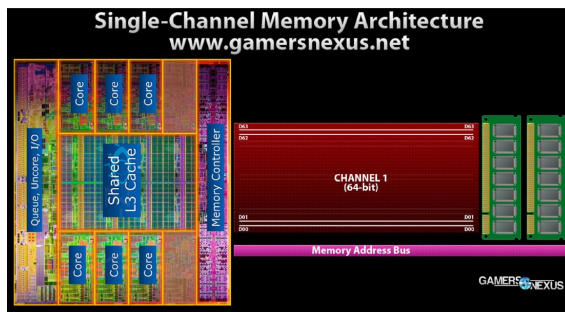
- 단위 시간 당 데이터 전송량 = 버스 폭(bus width) x 클록 주파수
- 예를 들어 데이터 버스 = 64비트, 버스 클록 주파수 = 133MHz인 경우 대역폭은 $(133\text{MHz} \times 64) / 8\text{비트} = 1064 \text{ [Mbytes/sec]}$

❖ 클록 주파수와 버스 주파수

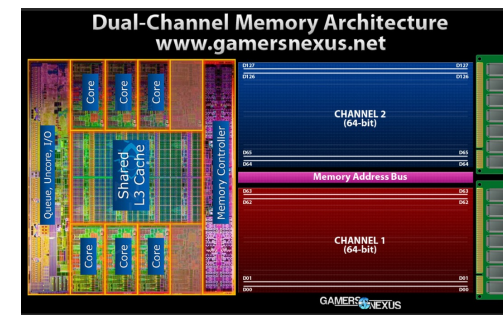
- 클록 주파수(clock frequency) : 메모리 셀 어레이 내부에서 동작하는 주파수
- 버스 주파수(bus frequency) : I/O 버퍼가 동작하는 주파수
- 보통 PCB에서 측정하는 CPU와 메모리 간의 클록은 버스 주파수다.

❖ DDR2, DDR3, DDR4

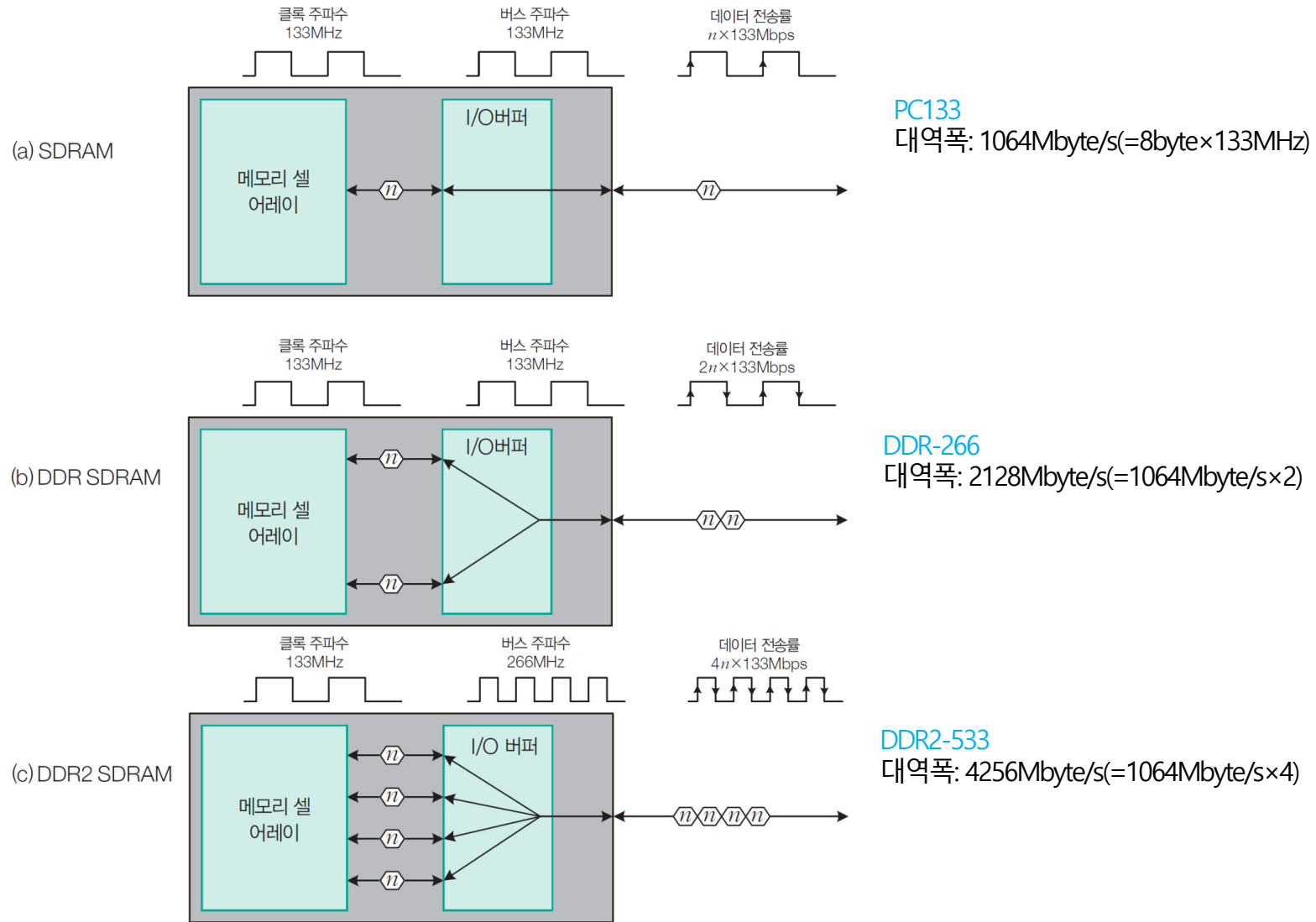
- DDR2부터는 ② 메모리와 제어기 사이의 버스 인터페이스 회로의 전송 속도를 높인 기술이다.
- DDR4 SDRAM은 메모리 내부적으로 듀얼 채널(dual channel)이 구현된 DDR3이라고 볼 수 있다.



대역폭이 2배가 됨



06 최신 기억 장치 기술



06 최신 기억 장치 기술

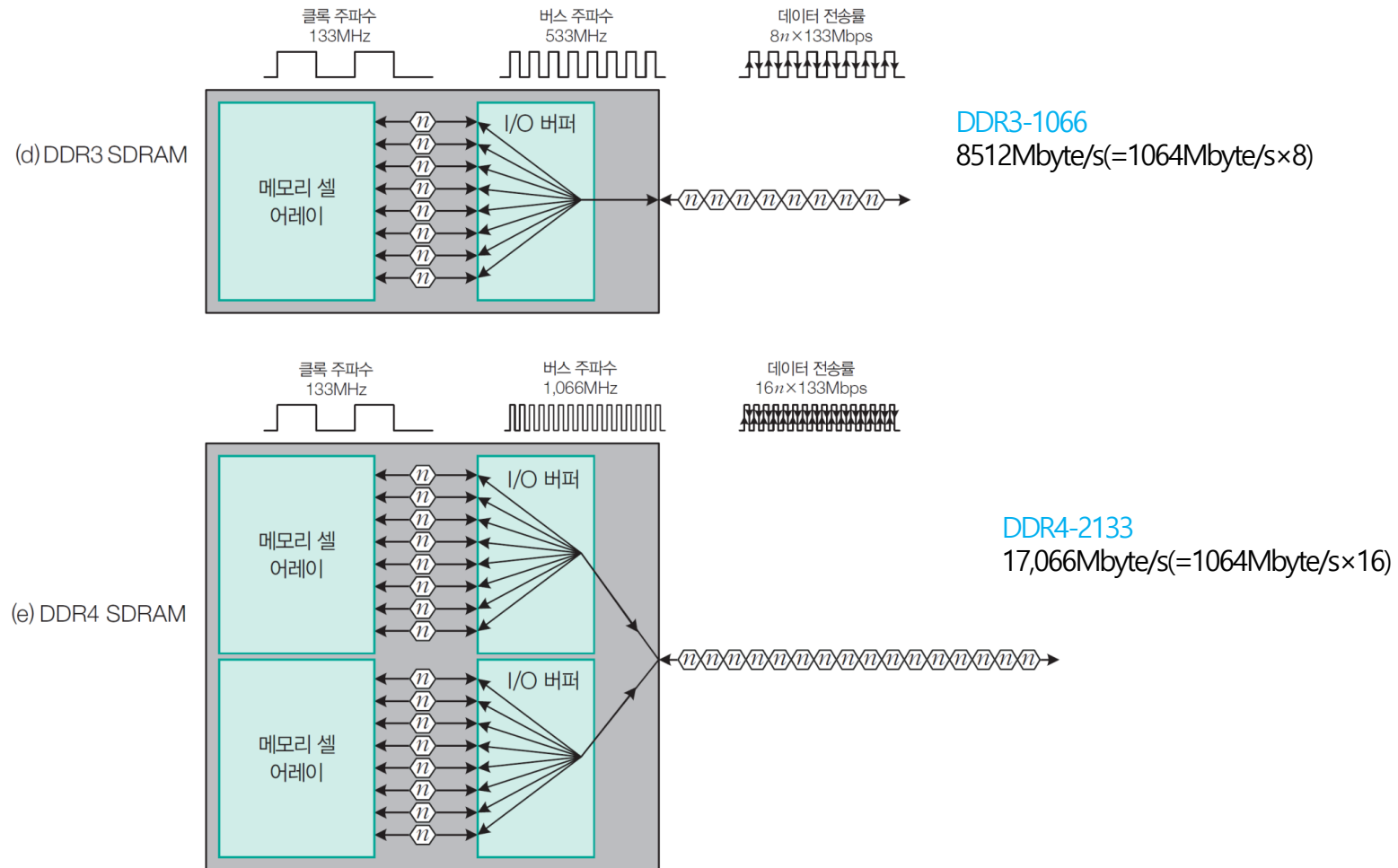


그림 6-54 SDRAM, DDR·DDR2·DDR3·DDR4 SDRAM의 동작 원리

06 최신 기억 장치 기술

❖ SDRAM, DDR SDRAM의 종류(JEDEC)

표 6-12 SDRAM, DDR SDRAM의 종류

규격	모듈 이름	핀 수	전압	클록 주파수(MHz)	버스 주파수(MHz)	대역폭(MB/s)
SDRAM	PC-66	168	3.3V	66	66	528
	PC-100			100	100	800
	PC-133			133	133	1,064
DDR-200	PC-1600	184	2.5V	100	100	1,600
DDR-266	PC-2100			133	133	2,100
DDR-333	PC-2700			166	166	2,700
DDR-400	PC-3200			200	200	3,200
DDR2-400	PC2-3200	240	1.8V	100	200	3,200
DDR2-533	PC2-4200			133	266	4,200
DDR2-667	PC2-5300			166	333	5,300
DDR2-800	PC2-6400			200	400	6,400
DDR3-800	PC3-6400	240	1.5V	100	400	6,400
DDR3-1066	PC3-8500			133	533	8,500
DDR3-1333	PC3-10600			166	666	10,600
DDR3-1600	PC3-12800			200	800	12,800
DDR4-1600	PC4-12800	284	1.2V	100	800	12,800
DDR4-1866	PC4-14933			117	933	14,933
DDR4-2133	PC4-17066			133	1,066	17,066
DDR4-2400	PC4-19200			150	1,200	19,200
DDR4-2666	PC4-21333			166	1,333	21,333
DDR4-3200	PC4-25600			200	1,600	25,600

클록 주파수와 주기

내부 클록 주파수(MHz)	주기 (ns)
66	15
100	10
117	8.5
133	7.5
150	6.6
166	6
200	5

06 최신 기억 장치 기술

❖ DDR, DDR2, DDR3, DDR4의 메모리 모듈

- DDR2와 DDR3의 핀 수는 240핀으로 같지만,中间的의 홈 위치가 달라 혼용이 안 된다.



그림 6-55 DDR, DDR2, DDR3, DDR4의 메모리 모듈 모양

06 최신 기억 장치 기술

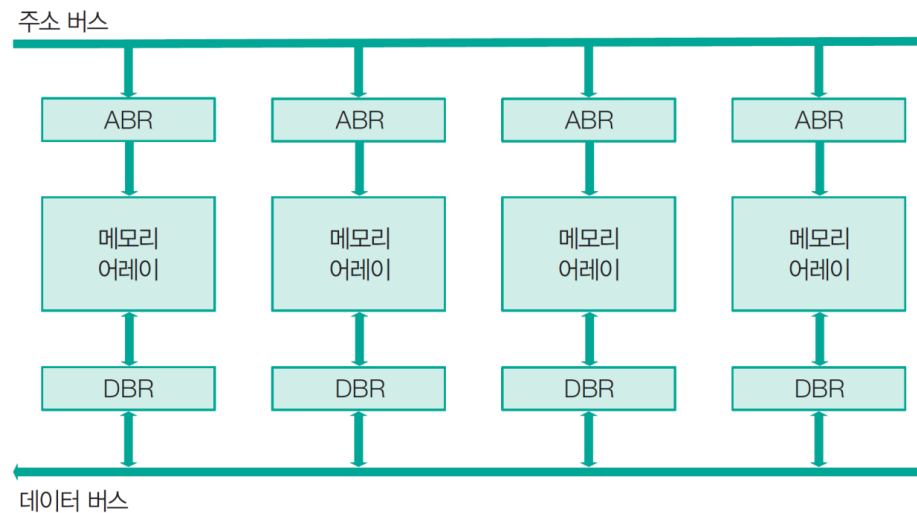
3 인터리브드 메모리

❖ 대역폭을 높이는 방법

- ① 메모리의 입출력 포트를 늘리는 방법
- ② 메모리 자체의 클럭을 높이는 방법
- ③ 메모리를 병렬화하여 액세스하는 **인터리빙(interleaving)** 방법

❖ 메모리 인터리빙

- 메모리의 각 모듈에 연속된 주소를 지정하고 이를 순차적으로 읽는 **파이프라인** 개념을 기반으로 한다.



ABR : Address Buffer Register
DBR : Data Buffer Register

그림 6-56 4-모듈 기억 장치

폰노이만 병목 현상:

주 기억 장치가 CPU보다 속도가 너무 느리고, 주 기억 장치를 사용할 때 CPU와 보조 기억 장치 같은 입출력 장치가 경쟁하므로 기억 장치 버스에 병목 현상이 발생

06 최신 기억 장치 기술

❖ 인터리빙 개념

- 인터리빙은 액세스 시간이 감소되어 대역폭이 늘어나는 효과를 얻는다.
- 메모리 인터리빙은 블록 단위 전송이 가능하므로 DMA(Direct Memory Access)에서 많이 사용한다.

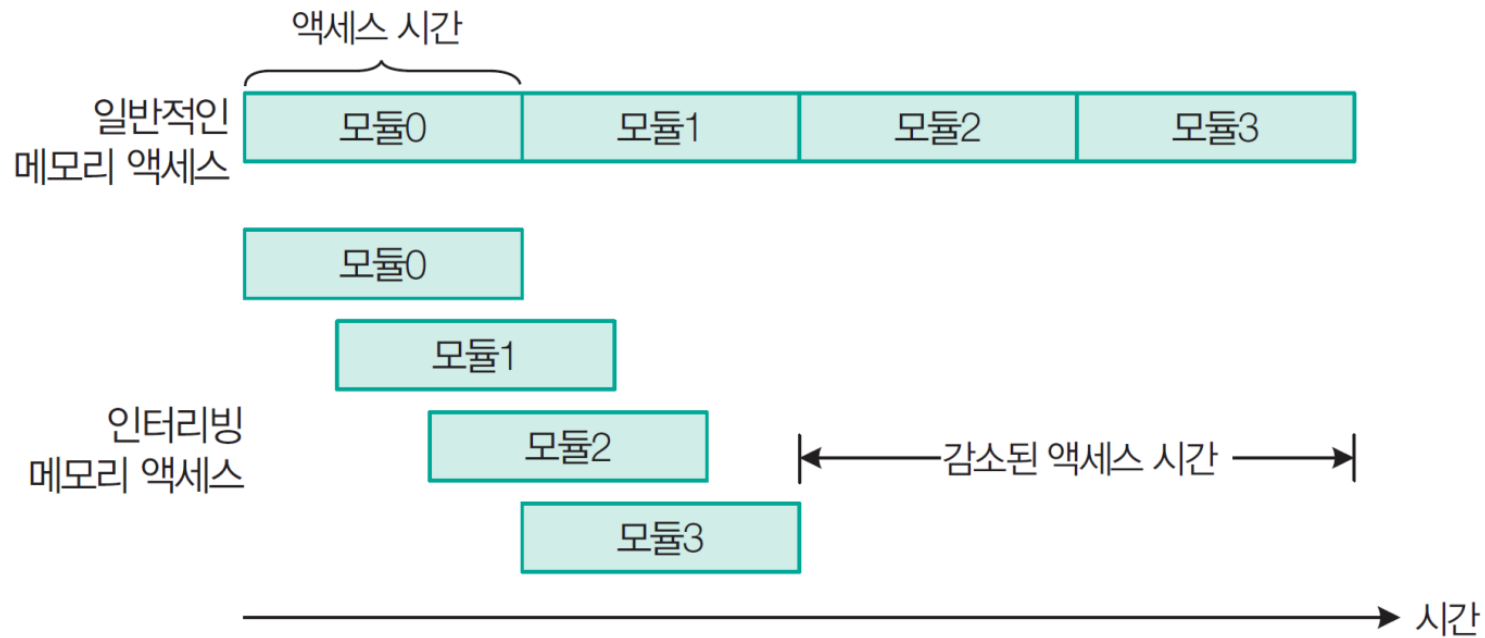


그림 6-57 일반적인 메모리 액세스와 인터리빙 메모리 액세스 비교

06 최신 기억 장치 기술

❖ 인터리빙 메모리의 대역폭

- 메모리 모듈이 동기화되어 병렬 액세스가 가능하도록 하기 위해 주기억 장치의 버스는 각 모듈 간에 시분할(Time-Sharing)하여 동작한다.
- 인터리빙으로 얻어지는 대역폭 향상은 연속적인 메모리 주소에 저장된 정보에 접근하는 경우에만 얻을 수 있다.

인터리빙 메모리의 대역폭 = 한 모듈의 대역폭 × 모듈 수

표 6-13 인터리빙된 주소 할당

(a) 4-모듈인 경우

모듈0	모듈1	모듈2	모듈3
0000(0)	0001(1)	0010(2)	0011(3)
0100(4)	0101(5)	0110(6)	0111(7)
1000(8)	1001(9)	1010(10)	1011(11)
1100(12)	1101(13)	1110(14)	1111(15)

(b) 8-모듈인 경우

모듈0	모듈1	모듈2	모듈3	모듈4	모듈5	모듈6	모듈7
0000(0)	0001(1)	0010(2)	0011(3)	0100(4)	0101(5)	0110(6)	0111(7)
1000(8)	1001(9)	1010(10)	1011(11)	1100(12)	1101(13)	1110(14)	1111(15)

06 최신 기억 장치 기술

상위 인터리빙

- 주기억 장치의 주소 영역을 모듈과 모듈 내 위치를 나타내도록 한다.
- 주기억 장치의 주소 공간을 4K이면 주기억 장치의 주소는 12비트($A_{11} \sim A_0$)
- 모듈 개수가 4개면 모듈 선택에 2비트 ($A_{11} \sim A_{10}$)
- 10비트($=12-2, A_9 \sim A_0$)는 모듈 내 주소를 나타내도록 구성할 수 있다.

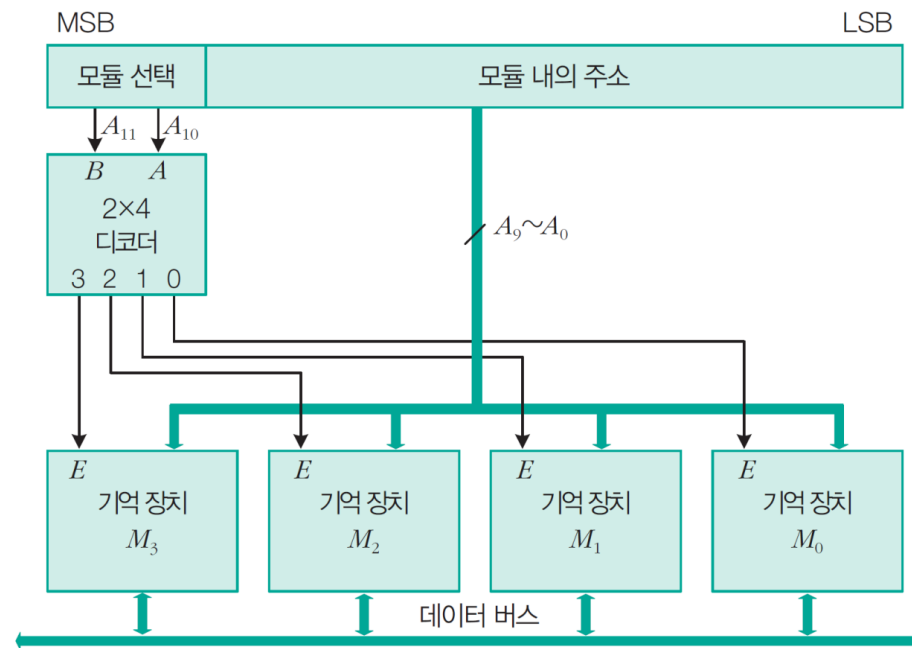


그림 6-58 상위 인터리빙 메모리 구조

06 최신 기억 장치 기술

❖ 상위 인터리빙 주소 할당

- 주기억 장치의 주소는 첫 번째 모듈 내의 모든 기억 장소에 순차적으로 주소가 지정된 후, 두 번째 모듈의 처음 위치부터 이어서 주소가 지정되는 방식이다.
- 명령어와 데이터를 독립적으로 기억 모듈에 저장하는 다중 프로그래밍에서 효과적이다.
- 오류 발생 시 주소 공간의 일부만 영향을 미친다.

표 6-14 상위 인터리빙된 주소 할당

모듈0	모듈1	모듈2	모듈3
00 0000000000(0)	01 0000000000(1024)	10 0000000000(2048)	11 0000000000(3072)
00 0000000001(1)	01 0000000001(1025)	10 0000000001(2049)	11 0000000001(3073)
...	...	...	...
00 1111111111(1023)	01 1111111111(2047)	10 1111111111(3071)	11 1111111111(4095)

06 최신 기억 장치 기술

하위 인터리빙

- 주기억 장치의 주소 영역을 모듈과 모듈 내 위치를 나타내도록 한다.
- 주기억 장치의 주소 공간을 4K이면 주기억 장치의 주소는 12비트($A_{11} \sim A_0$)
- 모듈 개수가 4개면 모듈 선택에 2비트 ($A_1 \sim A_0$)
- 10비트($=12-2, A_{11} \sim A_2$)는 모듈 내 주소를 나타내도록 구성할 수 있다.

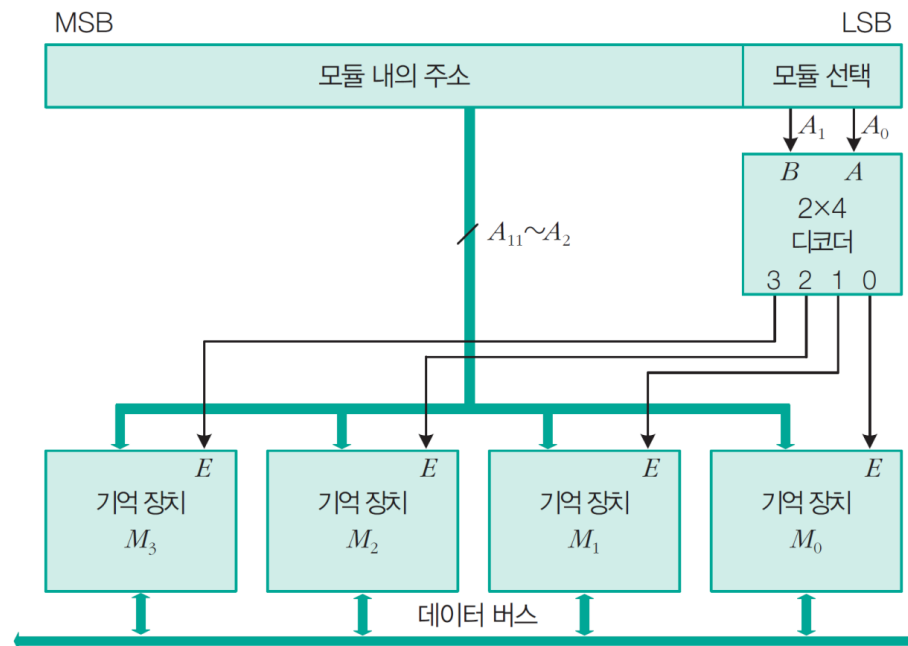


그림 6-59 하위 인터리빙 메모리 구조

06 최신 기억 장치 기술

❖ 하위 인터리빙 주소 할당

- 주기억 장치의 주소는 모든 모듈의 첫 번째 기억 장소에 순서대로 연속된 주소가 지정된 후 다시 첫 번째 모듈로 돌아와서 두 번째 기억 장소에 그 다음 주소가 지정된다.
- 연속된 주소가 연속된 모듈에 따라 다수의 모듈이 동시에 동작하므로 액세스 속도가 향상된다.
- 확장이 어렵고 어느 한 모듈의 오류 시 기억 장치 전체에 영향을 미친다는 단점이 있다.

표 6-15 하위 인터리빙된 주소 할당

모듈0	모듈1	모듈2	모듈3
0000000000 00(0)	0000000000 01(1)	0000000000 10(2)	0000000000 11(3)
0000000001 00(4)	0000000001 01(5)	0000000001 10(6)	0000000001 11(7)
...	...	...	...
1111111111 00(4092)	1111111111 01(4093)	1111111111 10(4094)	1111111111 11(4095)

06 최신 기억 장치 기술

■ 혼합 인터리빙

- 두 방식을 혼합한 방식으로 전체 모듈을 그룹으로 나누고 각 그룹 내 모듈 간에 인터리빙을 한다.
- 기억 모듈로 이루어지는 그룹을 **기억 장치 बैं크(memory bank)**라고 한다.
- 주기억 장치의 주소는 상위 비트들은 बैं크를 지정하고, 하위 비트는 बैं크 내의 모듈을 선택하는데 사용하며, 중간 비트들은 बैं크 내의 모듈에 대한 기억 장소 주소 지정을 한다.

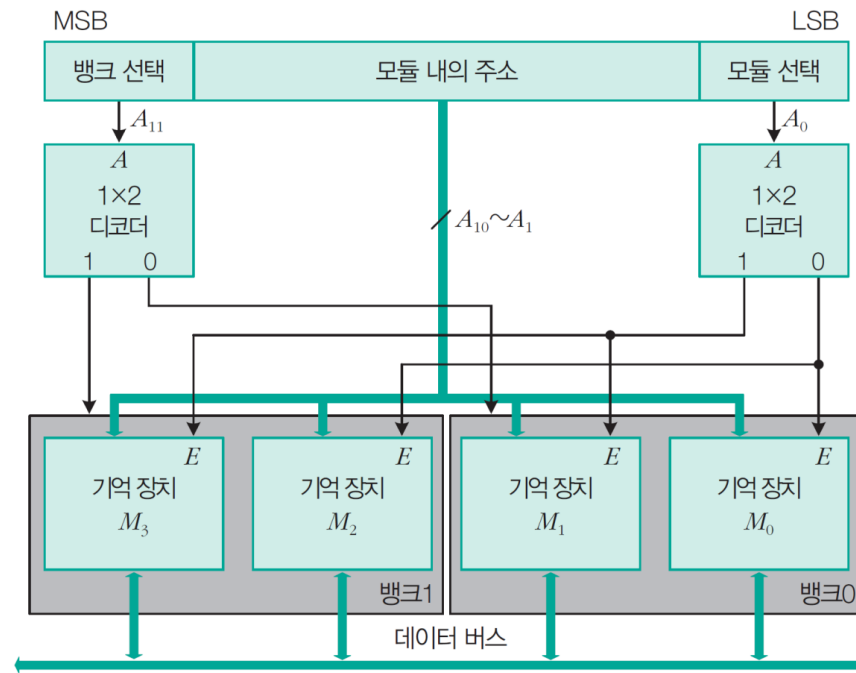


그림 6-60 혼합 인터리빙 메모리 구조

06 최신 기억 장치 기술

❖ 혼합 인터리빙 주소 할당

- 상위 인터리빙 방식을 이용하여 전체 모듈을 뱅크 2개로 나눈다. A_{11} 에 의해 0번지부터 2047번지까지를 뱅크0, 2048번지부터 4095번지까지를 뱅크1로 지정한다.
- 모듈 선택을 위하여 하위 인터리빙 방식으로 A_0 을 이용한다.
- $A_{10} \sim A_1$ 을 이용하여 주소를 지정한다.

표 6-16 혼합 인터리빙된 주소 할당

뱅크0		뱅크1	
모듈0	모듈1	모듈0	모듈1
0 0000000000 0(0)	0 0000000000 1(1)	1 0000000000 0(2048)	1 0000000000 1(2049)
0 0000000001 0(2)	0 0000000001 1(3)	1 0000000001 0(2050)	1 0000000001 1(2051)
...	...	...	...
0 1111111110 0(2044)	0 1111111110 1(2045)	1 1111111110 0(4092)	1 1111111110 1(4093)
0 1111111111 0(2046)	0 1111111111 1(2047)	1 1111111111 0(4094)	1 1111111111 1(4095)

06 최신 기억 장치 기술

4 플래시 메모리

❖ 플래시 메모리의 셀 구조

- 플래시 메모리는 FG MOS(Floating Gate MOSFET)라는 특별한 구조의 MOSFET에 전하(electrical charge)를 축적하여 데이터를 기억한다.
- 플로팅 게이트에 축적된 전하의 유무에 따라 0과 1의 데이터를 저장하며, 축적된 전하는 전원 공급이 없어도 5~10년 동안 전하를 저장할 수 있다.
- 플로팅게이트에 저장된 전자가 많으면 논리 0이 저장되고, 전자가 적거나 없을 경우에는 논리 1이 저장된다.

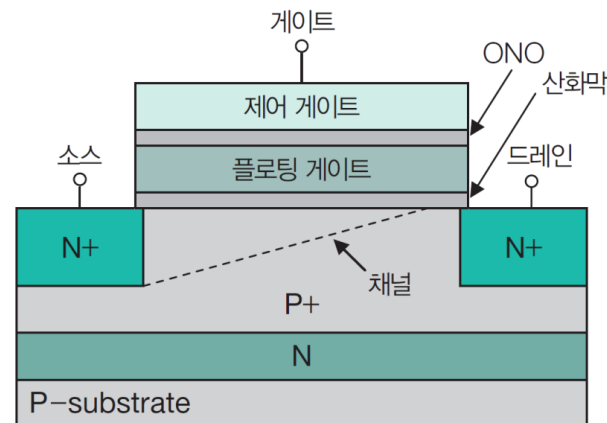


그림 6-61 플래시 메모리의 셀 구조

06 최신 기억 장치 기술

□ 쓰기 동작

- 플래시 메모리를 구입했을 때나 지우기 동작 후에는 논리 1 상태(전자 없음)에 있다.
- 제어 게이트에 약 12~19V의 전압을 인가하면 소스에서 드레인으로 흘러가던 전자가 얇은 산화물 층을 뚫고 플로팅 게이트로 끌려 들어가게 된다.(NAND 플래시인 경우 tunnel injection, NOR 플래시인 경우 hot-electron injection)
- 산화물로 둘러 쌓인 플로팅 게이트는 외부와 차단되어서 플로팅 게이트에 들어온 전자는 큰 전기장의 영향을 받지 않는 이상 외부로 빠져 나갈 수 없다.
- 플로팅 게이트로 들어온 전자는 쉽게 빠져나갈 수 없어서 전원이 끊긴 상태에서도 데이터가 지워지지 않는다.

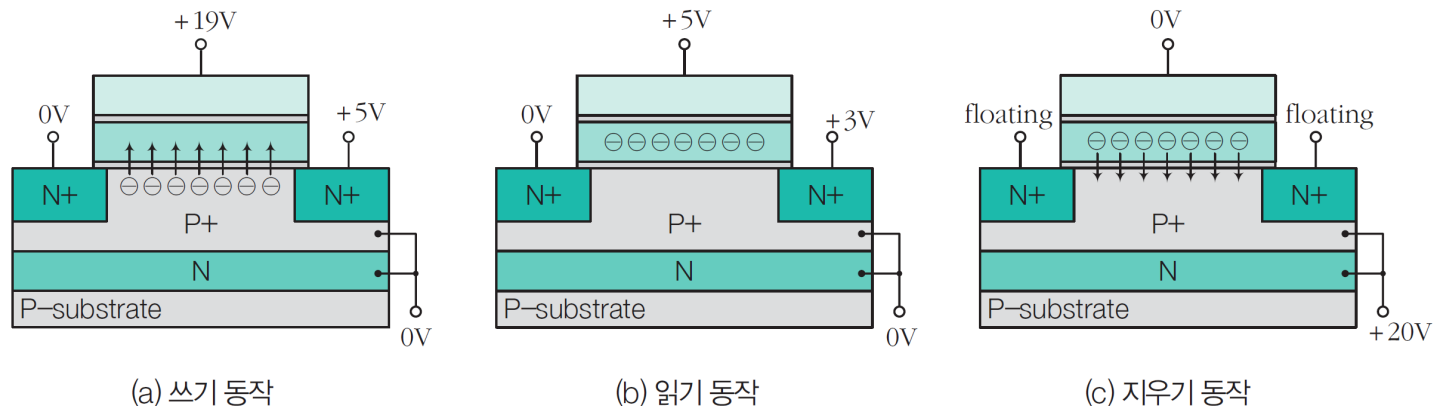


그림 6-62 플래시 메모리의 동작(NAND 플래시인 경우)

06 최신 기억 장치 기술

□ 읽기 동작

- **문턱 전압(threshold voltage)** : FG MOS가 off 상태에서 on 상태로 변하는 게이트 전압
- 플로팅 게이트에 전자가 채워져 있으면(논리 0) 제어 게이트 전압이 플로팅 게이트에 저장되어 있는 음전하(전자)를 극복하기에 불충분하므로 FG MOS는 off된다.
- 플로팅 게이트에 전자가 없으면(논리 1) 제어 게이트 전압은 FG MOS를 on시키기에 충분하다.
- FG MOS가 on되면 드레인에서 소스로 전류가 흐르는데 이 전류가 감지되면 논리 1이고, off되어 전류가 감지되지 않으면 논리 0이 된다.

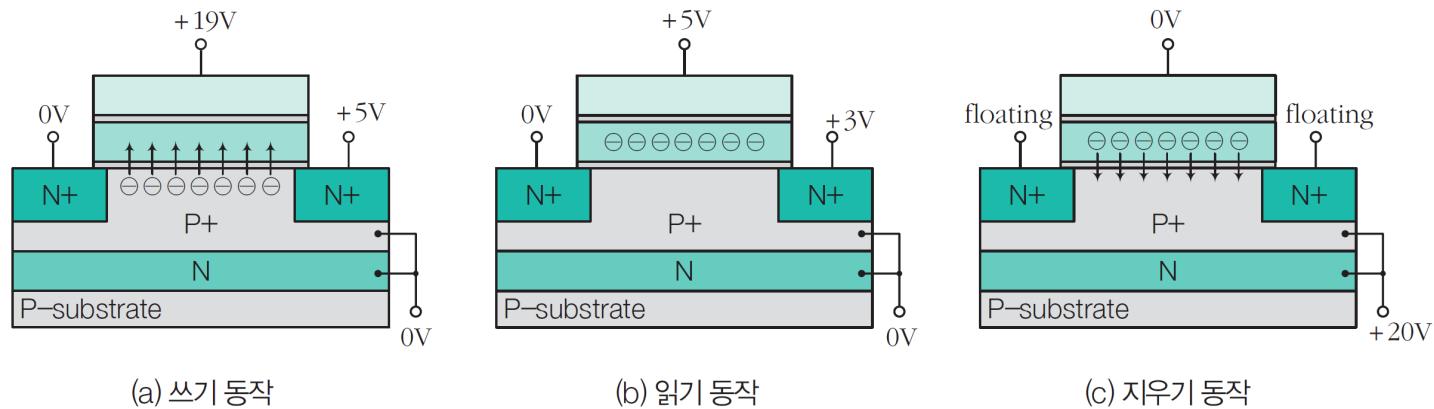


그림 6-62 플래시 메모리의 동작(NAND 플래시인 경우)

06 최신 기억 장치 기술

□ 지우기 동작

- 지우기 동작에서는 모든 메모리 셀의 전하를 제거한다. P+와 N층에 약 +20V 전압을 인가하면 쓰기 동작과는 반대로 플로팅 게이트에 있던 전자들이 강한 전기장의 힘에 이끌려 산화물 층을 통과하여 밖으로 나오게 되고 플로팅 게이트에는 전자가 사라져서 논리 1의 상태로 바뀌게 된다.
- 이런 과정을 NAND 플래시인 경우에는 터널 릴리즈(tunnel release)라고 하며, NOR 플래시인 경우에는 F-N 터널링(Fowler-Nordheim Tunneling)이라고 한다.
- 플래시 메모리는 쓰기 동작 전에 항상 지우기 동작을 수행한다.

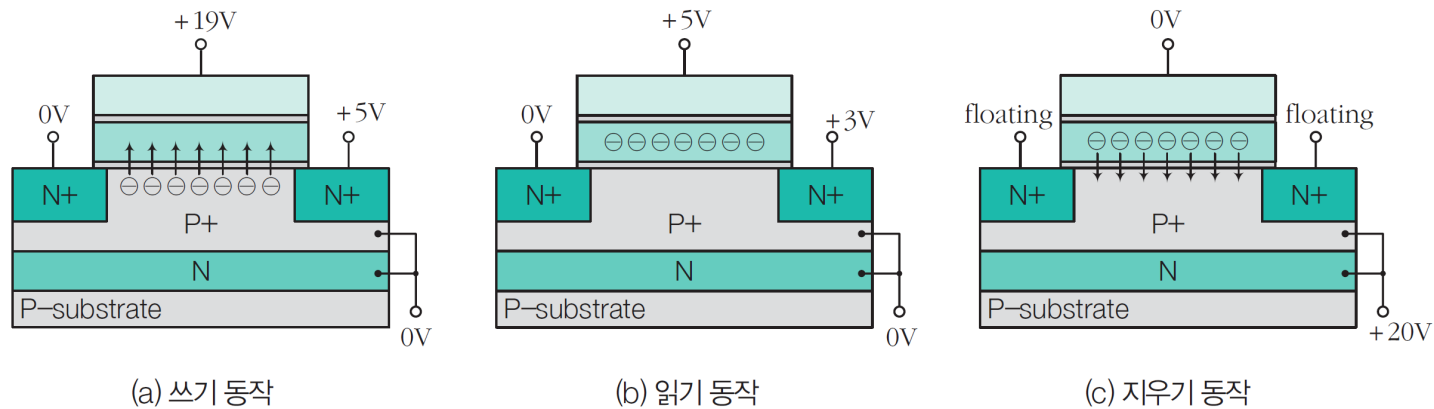


그림 6-62 플래시 메모리의 동작(NAND 플래시인 경우)

06 최신 기억 장치 기술

□ NAND 플래시의 구조와 특징

- 소스와 드레인을 공유하는 형태로 접속되어 있어 실리콘 면적을 작게 할 수 있어 고밀도 대용량의 플래시 메모리가 가능하다.(DRAM의 약 50%, NOR 플래시의 80%)
- 블록(block)은 스트링과 페이지로 구성된다.
- 페이지 단위로 읽기/쓰기 동작이 가능하지만, 특정 페이지를 지우려면 해당 페이지가 포함된 블록 전체를 지워야 한다.

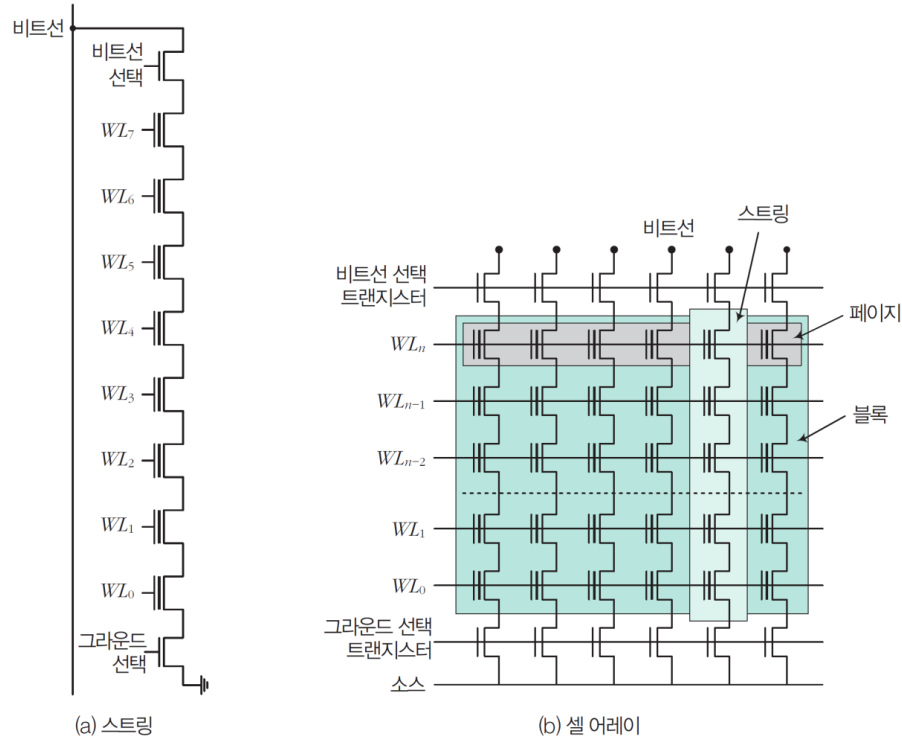


그림 6-63 NAND 플래시의 스트링 및 메모리 셀 어레이 구조

06 최신 기억 장치 기술

□ NOR 플래시의 구조와 특징

- 메모리 셀에 비트선과 소스선 접속부가 각각 존재하므로 NAND 플래시에 비해 저장 밀도가 낮다.
- 워드선(WL)과 비트선(BL) 이외에 소스선(source line, SL)이 있다.
- random access 방식으로 바이트 또는 워드 단위로 읽기/쓰기 동작이 가능하지만 덮어쓰기와 지우기 동작은 임의로 접근할 수 없다.

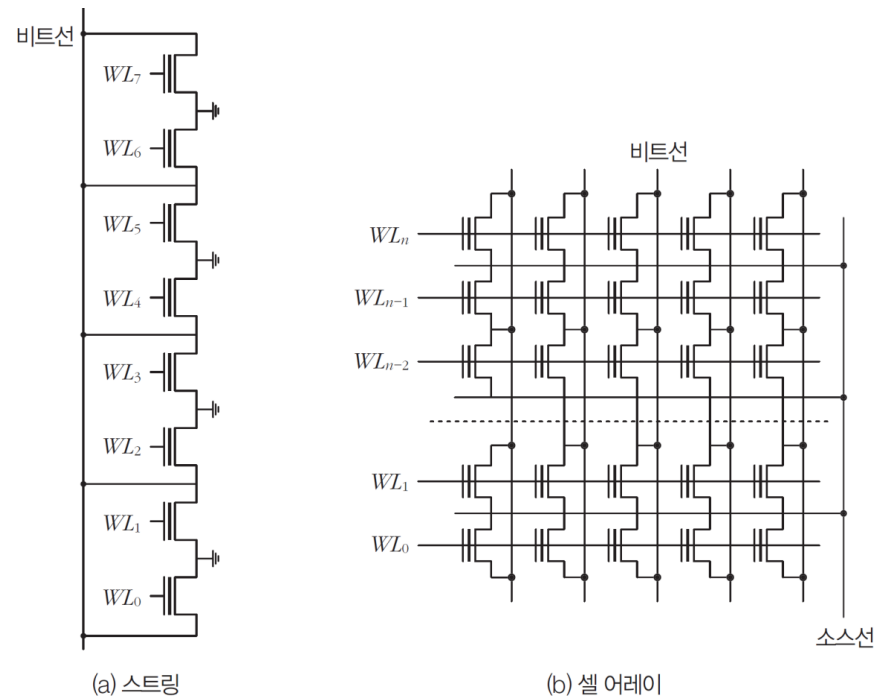


그림 6-64 NOR 플래시의 스트링 및 메모리 셀 어레이 구조

06 최신 기억 장치 기술

□ NAND 플래시와 NOR 플래시의 비교

표 6-17 NAND 플래시와 NOR 플래시 특성 비교

구분	NAND 플래시	NOR 플래시
용도	데이터 저장용	프로그램 코드 저장용
읽기 속도	느리다.	빠르다.
쓰기 속도	빠르다.	느리다.
지우기 속도	빠르다.	느리다.
구조	셀이 직렬로 연결 데이터/주소 통합 구조	셀이 병렬로 연결 데이터/주소 분리 구조
액세스 단위	페이지 및 블록	워드 및 바이트
랜덤 액세스	데이터 읽기 시 불가능	데이터 읽기 시 가능
불량 섹터	있다.	없다.
단가	단가 낮다.	단가 높다.
저장 용량	대용량	소용량
사용 기기	USB 드라이브, 메모리 카드에 이용	스마트폰, 셋톱박스용 칩에 사용
주도 업체	삼성전자, 도시바	인텔, AMD

06 최신 기억 장치 기술

□ SLC, MLC, TLC의 차이와 특성

- 플로팅 게이트에 주입된 전자의 양에 따라 데이터를 구분하는 개념으로 셀을 운용

SLC: 셀 하나에 1비트 정보 저장 0, 1

MLC: 셀 하나에 2비트 정보 저장 00, 01, 10, 11

TLC: 셀 하나에 3비트 정보 저장 000, 001, 010, 011, 100, 101, 110, 111

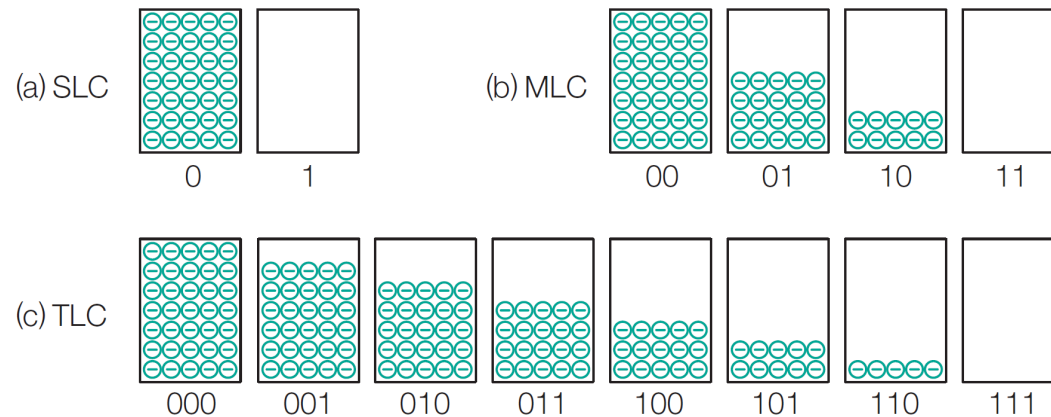
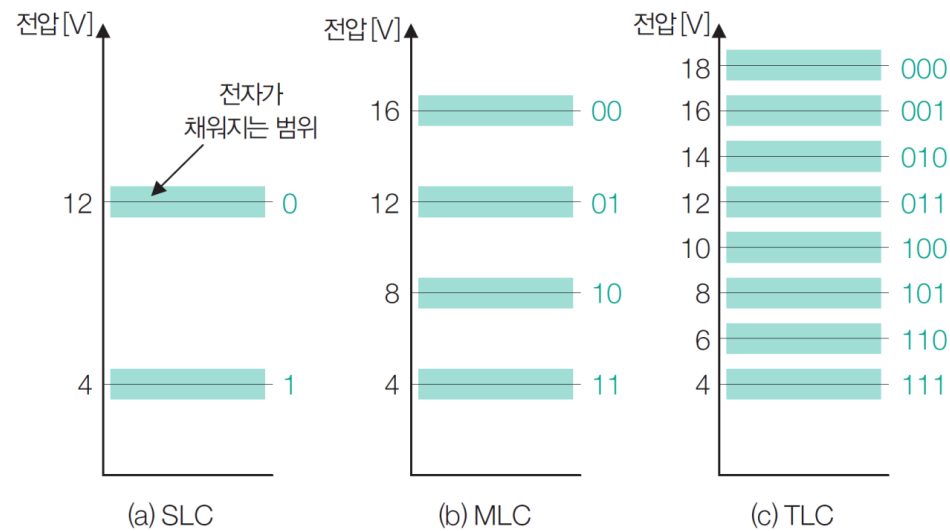


그림 6-65 메모리 셀의 데이터 저장 방식

06 최신 기억 장치 기술

□ SLC, MLC, TLC에서 전자가 채워지는 범위

- 제어 게이트에 특정 전압을 인가하면 플로팅 게이트에 일정 범위 내의 전자가 채워지며, 데이터를 읽는 것도 플로팅 게이트의 전하량이 일정 범위 내에 있으면 해당 데이터로 읽는다.
- 제어 게이트에 인가하는 전압에 비례하여 원하는 수준의 전자가 플로팅 게이트에 정확히 주입되지 않는 문제가 있다.



전압은 임의로 설정

그림 6-66 제어 게이트에 특정 전압을 인가했을 때 전자가 채워지는 범위

06 최신 기억 장치 기술

□ SLC, MLC, TLC의 특성 비교

- NAND 플래시에서는 에러를 검출하고 정정할 수 있도록 데이터와 함께 에러 정정용 **ECC 코드** (Error Correcting Code)를 함께 저장
- SLC는 에러 발생 확률이 적어 ECC 코드도 간단하지만, MLC와 TLC로 갈수록 에러 발생 빈도가 높아서 강력하고 복잡한 ECC 코드가 사용된다.
- 전체적으로 데이터의 읽기, 쓰기, 지우기 작업이 느려지므로 SLC→MLC→TLC 순으로 성능이 떨어진다.
- 상태 별로 전압 간 차이가 다르므로 SLC→MLC→TLC 순으로 수명이 줄어든다.

표 6-18 SLC, MLC, TLC의 특성 비교

구분	SLC	MLC	TLC
셀당 비트 수(bit/cell)	1	2	3
재기록 가능 횟수	100,000	3,000~10,000	1,000
읽기 시간 (read time)	25 μ s	50 μ s	~75 μ s
쓰기 시간(write time)	200~300 μ s	600~900 μ s	~900~1350 μ s
지우기 시간(erase time)	1.5~2ms	3ms	4.5ms



PNY Turbo USB3.0
슬라이드 소켓형의 PNY TURBO USB-러
USB메모리 / USB3.0(USB3.2 Gen1) / MLC / 색상:실버
★ 4.8점 (10) | 등록일 2014.06 | PNY

128GB 95원/GB 65,940원 1개

MLC

전체상품 435 | 가격비교 | 알바상품 | 해외직구 | 카테고리 내 검색 | 30개씩 보기

✓ 인기순 • 최저가순 • 최고가순 • 신제품순 • 판매량순 • 상품평 많은순

☐ 중고/오픈마켓 제외 ☐ 배송비 포함



SanDisk Cruiser Blade Z50
식물 줄 모르는 인기! 5년 AS는 기본! 암호설정 유틸리티로 안전한 USB!
USB메모리 / USB2.0 / TLC / 부가기능:충전코디 / 유틸리티(다운로드) 암호설정 / 무상AS-5년
/ 길이4.2cm / 무게2.5g
★ 4.8점 (41,999) | 등록일 2013.04 | SanDisk

128GB 81원/GB 10,330원 648개
64GB 82원/GB 5,230원 560개
241GB 81원/GB 2,580원 999+개
16GB 148원/GB 2,370원 776개
141GB 260원/GB 2,080원 514개

TLC

감사합니다.